MEASURE — can we believe a number

Owns: `engine/mew.js` , `engine/sprt.js` , `engine/provenance.js` , `engine/status.js` , `engine/backtest_winrate.js` , `engine/paired_h2h.js` , `engine/feature_engine_contrast.js` , the noise floor, the corpus stamps, and the MAG refit.

Its one number: leaf calibration — when the leaf says 90%, is it 90%.

May not: change a policy, a search knob or an engine mechanic. This division builds the rulers; it does not compete on them.

<!-- GENERATED: engine/status.js -->

```
MEASURE — can we believe a number
  leaf calibration: QUARANTINED — the figure is withheld, not annotated.
    data/winrate-backtest.json is downstream of MEDICHAM: its generator engine/backtest_winrate.js is in t
    MEDICHAM is not correct — 5 of 8 gate clauses fail (deliberate roster / items; deliberate roster / abi
    it becomes quotable again when the gate opens AND this is re-run: node engine/backtest_winrate.js
  engine correctness -> leaf: QUARANTINED — the figure is withheld, not annotated.
    data/leaf-engine-contrast.json is downstream of MEDICHAM: its generator engine/leaf_engine_contrast.js
    MEDICHAM is not correct — 5 of 8 gate clauses fail (deliberate roster / items; deliberate roster / abi
    it becomes quotable again when the gate opens AND this is re-run: node engine/leaf_engine_contrast.js
  provenance: 197 unsafe, 2 void (declared), 16 possibly stale, 22 ok, 0 missing
  click censoring: QUARANTINED — the figure is withheld, not annotated.
    data/click-censoring-census.json is downstream of MEDICHAM: its generator engine/click_census.js is ir
    MEDICHAM is not correct — 5 of 8 gate clauses fail (deliberate roster / items; deliberate roster / abi
    it becomes quotable again when the gate opens AND this is re-run: node engine/click_census.js
  the weights are QUARANTINED — data/policy-weights.json and the joint weights were fitted on features com
  REFIT OWED — weights fitted 2026-08-05 00:00
    feature_fixture --check FAILED:   or restamp with: node engine/feature_fixture.js --stamp <file> |   G
    moved after the fit: engine/medicham2-browser.js  2026-08-28 05:46
    moved after the fit: engine/board.js  2026-08-23 15:27
    moved after the fit: data/engine-data.js  2026-08-22 01:46
    moved after the fit: data/abra-tags.js  2026-08-28 04:30
```

stamped 2026-08-28 14:54

<!-- /GENERATED -->

THE DOCUMENTS ARE UNFROZEN AND FIVE GATE CLAUSES WENT BLANK BETWEEN 09:58Z AND 10:06Z. THE CAUSE IS A LINE ENDING. 2026-08-28, CHANGELOG 5.205.0.

Will paused MEDICHAM development and asked for the docs to be unfrozen. The living-docs deferral of 2026-08-10 is discharged: 274 sprint rows folded into the white paper, the deck, the technical docs, SUMMARY, MODELS and DAMAGE-STAGES; all six restamped to 5.205.0; the six `MEDICHAM SPRINT PAUSE` version pins retired from `data/docs-currency-baseline.json` ; `docs/MEDICHAM-SPRINT-NOTES.md` deleted, which is what re-arms the full rule in `engine/docs_scan.js` and `.githooks/pre-commit` .

THIS DIVISION'S FINDING IS NOT THE DOCUMENTS. It is that the gate read 7 of 8 clauses PASS a few hours ago and reads 5 of 8 now, with NO ENGINE BYTE CHANGED. The three deliberate-roster stages, the whole-game differential and the staged-mechanics comparison all record engine release `5f3f7141227c` and were written at 09:56–09:58Z. The tree now hashes to a different release, so `engine/status.js` correctly calls every count in them *"an answer about other bytes"* and withholds it.

MEASURED, NOT ASSUMED — AND THE DISCRIMINATOR IS THE STRONGEST ONE AVAILABLE. Exactly ONE of the twenty-six frozen sources moved: `data/tags.json`, mtime 10:06Z. The release keeps a COPY rather than a checksum, so the two files can be compared directly:

```
data/releases/5f3f7141227c/data/tags.json   799,498 bytes
data/tags.json                              842,110 bytes
raw equal?                 false
CRLF-normalised equal?     true
JSON deep-equal?           true
```

A checkout under `core.autocrlf = true` rewrote a generator's LF output as CRLF between the runs and now. `tag_dex.js` writes LF; git hands back CRLF; the sha256 moves and the content does not.

THIS IS THE SECOND OCCURRENCE AND IT IS ALREADY IN THE RECORD. `docs/ENGINE.md` documents the identical event on 2026-08-26 — *"the reason is a line ending, which is worth writing down because it will happen again to whoever next regenerates* `data/tags.json` *"* — where the remedy was to cut the release over the bytes a checkout actually produces. It happened again. **The hazard is standing, not closed:** `git diff` currently warns *"LF will be replaced by CRLF the next time Git touches it"* on every one of the living documents as well, so any tracked LF file that a generator writes is one checkout away from stranding whatever measured against it.

WHAT THIS DIVISION DID NOT DO, DELIBERATELY. It did not restore the file to LF to make the gate read 7 of 8. Editing an input so a ruler prints the wanted number is the failure this division exists to prevent, and it would have produced a release id that no checkout reproduces. **The five clauses are WITHHELD and a re-run is OWED.** No rate, no diverged count and no roster column from those five artifacts was written into any document this pass — checked afterwards against `data/quarantine-stamp.json`'s `citation_sites` ratchet, which is unmoved at three entries.

WHAT WAS PUBLISHED, WITH ITS BOUNDS STATED IN ALL SIX DOCUMENTS.

figure	artifact	bound written beside it
780 probed / 780 live / 0 missing	`data/mechanics-census.json`	a LAB — one staged scenario per mechanic, usage-blind; answers *is this correct*, never *does this matter*
6000 compared / 6000 agreed / 0 disagreed, and 0 at each of the sixteen band indices	`data/engine-diff.json`	its own `scope` is damage only — no items, no abilities, no turn order, no status duration, no switching — and `skipped_multihit` 134 with `skipped_ability_multihit` 17 means **it has never applied a multi-hit move**
1696/1696 exact, 0 at the wrong stage	`tests/test-damage-stages.js`, re-run green this pass	the stage chain only; the crit die and the event die are the battle loop's
33 compared / 4 declared / 43 in neither list, 25 live at the boundary	`tests/probe_uncompared_leaves.js` over 500 moves, 201 abilities, 148 items	**"the boards match" is a claim about 33 leaves of 80** — recorded as the largest caveat in the document set

AND THE VOID RULING IS STATED IN ALL SIX RATHER THAN CAPTIONED. Every figure that passed through the event die before 2026-08-27 is VOID, not stale — two engines agreed over a narrower slice of outcome space than the comparison claimed, so an agreement is not evidence of one. The old paragraphs stay in place as dated history, under one governing sentence in each document that none of them may be cited as current. The alternative — a per-figure asterisk — is the thing CLAUDE.md already forbids.

OWED AND NOT RUN THIS PASS: the three roster stages, `engine/game_differential.js` and `engine/all_mechanics_fire.js` (a re-run is the only thing that lifts the stranding); the MAG refit, which `engine/status.js` still reports OWED with `feature_fixture --check` failing on both the fixture identity and the damage table; and the standing item — `data/winrate-backtest.json`, leaf calibration, which stays QUARANTINED and unmeasured behind the gate.

A CHECK THAT READS GREEN WITHOUT RUNNING IS WORSE THAN ONE THAT READS RED. 2026-08-28, CHANGELOG 5.201.0.

ROADMAP `#521`, `#522`, `#523`, `#524`, `#525`. **This division builds the rulers, and tonight the ruler was the thing that was wrong — for the sixth time in a day.**

THE HEADLINE IS A NEGATIVE RESULT AND IT IS STILL THE ANSWER. Three probes were counted as ACCOUNTED FOR by `tests/run-all.js`, in prose that asserted `engine/register_reality.js` ran their markers. It could not: `SAFE` required a marker to begin `node <script>` and permitted flags only, and all three begin `node -r ./tests/_live_release.js`. Run at last, **all three are GREEN**, each with a knob-cleared control that moves the arm it names. There was no hidden defect. The accounting was wrong and the engine was not — which is worth exactly as much as a finding, because it is the difference between a ruler that is trusted and one that is trusted correctly.

THE DECISION THAT MATTERED WAS WHICH SIDE OF THE FENCE TO EDIT, AND IT WAS MEASURED RATHER THAN ARGUED. `tests/_live_release.js` is load-bearing: `engine/game_differential.js:196` CUTS A REAL RELEASE at require time when no `--release` is given, repointing `data/engine-release.json` under whatever else is measuring. All three probes detect the preload themselves and refuse with exit 2 without it. Rewriting the markers to satisfy the old regex would have bought three refusals and one moved release pointer per pass. So the REGEX moved.

AND IT MOVED NARROWLY, WHICH IS THE PART TO KEEP. `SAFE` still refuses BARE VALUES. That is not laziness: widening it would silently admit `tests/roster.js --stage moves`, `engine/all_mechanics_fire.js --kind abilities` and two `game_differential.js --team-store` markers — multi-minute game-playing runs, three of which REWRITE artifacts other readers hold. A gate that runs those on every pass is a gate that rewrites the corpus it is auditing. `probe_trace_list.js` learned the `--cells=60` spelling instead; the parser moved, not the gate.

THE PREFIX WAS NOT DECORATIVE ON ALL FOURTEEN, AND THE ONE EXCEPTION IS THE POINT. The brief said thirteen were one deletion from running and warned that one probe running with the variable empty is evidence for that probe. Checked one at a time: `tests/probe_endturn_clock_order.js` never required `engine/showdown_path.js`, so it asked the raw variable and exited 2 with `NOT RUN` under exactly the environment thirteen siblings ran to completion in. Worse, a literal paste of its marker sets the variable to the three characters `...`, which `showdown_path.js` HONOURS by design — so the documentation was not merely useless, it was a working way to break the probe.

A REFUSAL IS STILL PUBLISHED AS A PASS, AND THE WORSE HALF IS AT EXIT ZERO — NOT EXIT 2. The source report ranked the `ABRA-EXIT` gap by counting exit-2 paths. Exit 2 is UNDECLARED, which `register_reality.js` reads as NOT A VERDICT and counts as BAD: noisy, but honest. The unmeasured half is the staging refusal spelled `process.exit(0)` — 12 paths in 4 of 74

`tests/probe_*.js` — which the register reads as VERDICT-GREEN and a CLOSED row reads as CONFIRMED. **A COULD-NOT-STAGE is a claim about the FIXTURE, never about the mechanic**, and at exit 0 that claim is published as a clean bill of health. Five converted and shown red by forcing the guard; the rest are filed, not swept.

#496 IS A PREMATURE CLOSE AND IT IS FILED AS OBSERVED, NOT AS A REGRESSION. `probe_trace_list.js` exits 1 on the live tree, reproduced **3 of 3** with byte-identical counters across two different pool digests. The row closed on a PINNED sample (`44bd49403231`) and this run drew a LIVE pool while `engine/medicham2-browser.js` was being edited — so the corpus stamps do not match and the claim stops at OBSERVED. Naming that boundary is this division's job; attributing it is not.

ONE THING WENT WRONG AND IT IS RECORDED RATHER THAN TIDIED AWAY. `data/engine-release.json` was found holding an automatic `game differential mode A` cut, I judged it my own leakage and reverted it to HEAD, then put it back ~90 seconds later. It has since been superseded by ENGINE's deliberate cut `0415c53255a9` ("the absorb answers arrival one…"), so the net effect is zero. **The rule the error breaks is the one this division enforces on everyone else: a measuring agent does not write a file it does not own, and on a moving tree it cannot attribute one either.** The correct action was to report the drift and leave it.

OWED, NOT RUN. `node engine/status.js --write` — this pass was denied the game slot, so every `<!-- GENERATED -->` block in the ledgers is stamped one pass behind.

A SUBTRACTION THE GATE MAKES MUST NAME WHAT IT SUBTRACTED. 2026-08-28, CHANGELOG 5.196.0.

ROADMAP `#520` . **This is a REPORTING change and it moves no clause.** Measured rather than argued: `HEAD:engine/quarantine.js` and the working copy were compiled in ONE process against the SAME on-disk artifacts and returned identical verdicts and identical counts on all eight clauses.

THE RULE THIS ENFORCES IS ALREADY THIS DIVISION'S, AND THE GUARD WAS BREAKING IT. A filter may only ever subtract from a number a reader can still see. The DECLARED register prints every row that MAY subtract, with its ruling, whether or not it fired — and the owner's closet, sitting one line away in the same clause, printed the integer `4 shelved by the owner` and nothing else. No names, no dates, no rulings. A fifth entry could have appeared and nothing would have said so.

`SHELVED BY THE OWNER` now names each row with its carrier, its cause, its board verdict and the dated ruling; publishes `owner_shelved` / `owner_shelved_summary` / `owner_shelved_rows` ; prints at zero as well as at four; and **compares the derived rows against the artifact's own summary** rather than assuming they agree — a derived set is not a fact until something compares it to its source.

NAMING IT FOUND SOMETHING ON THE FIRST RENDER. Of the four shelved rows on release `aea838766e7f` , three are `ANNOUNCEMENT-ONLY` and `item:metronome` **is** `board_verdict: STATE` — 859/960 against 868/960, a board that genuinely parted. Will's Metronome ruling is explicitly cost-based and stands; the point is that **the shelf is not uniformly a no-board-effect shelf**, and until this run nothing said which of its rows were which.

THE COMPANION FINDING IS A REFUSAL. Bitter Malice and Night Daze were proposed as the `CLOSETED` kind's first two entries and were refused: both already carry `deferred = ILLUSION_SHELF` , derived from `GD.CLOSET_SPECIES` off the ABILITY rather than a name list, and `classifyMechanics` skips a `deferred` row **before** it asks `declaredMatch` — so a declaration could not have been reached even if it matched. `game_differential.js` additionally drops all 43 Illusion-carrying teams from the pool, so the whole-game clause holds zero zoroark causes. Writing the rows would have registered a permanent exemption that fires on nothing.

TWO METHOD NOTES THIS DIVISION SHOULD KEEP.

1. **A SELFTEST THAT READS A LIVE ARTIFACT IS NOT A SELFTEST.** The first version of the printer assertion drove `mechanicsClause()` off disk and went RED mid-session for a reason that had nothing to do with the code: another division cut a release, `data/engine-release.json` moved `aea838766e7f` -> `b035aa665740`, and the clause correctly took its MEASURED-AGAINST-A-DIFFERENT-ENGINE early return. A green/red signal another agent can flip is noise, and noise is precisely how a red test becomes "one of the two known failures". `mechanicsClause` gained the `inject` door the file already uses twice.
2. **THE HAND-WALKED DERIVATION WAS WRONG AND THE VALIDATOR WAS RIGHT.** Walking prevo/`baseSpecies` chains reported Zoroark-Hisui as a Night Daze learner. `TeamValidator` over the 347 legal species of the format refuses it — **Bitter Malice -> Zoroark-Hisui, Night Daze -> Zoroark, one legal learner each.** When a legality question has an authority, ask the authority; a chain walk is a reimplementation of one, and this repo names its own casualty for that.

THE WHOLE-GAME AND BOARD-MATERIAL BASELINES WERE RESET ON PURPOSE ON 2026-08-27. DO NOT SUBTRACT ACROSS IT.

ENGINE fixed the `middle` arm's die (ROADMAP **#489**, CHANGELOG 5.176.0): bare FNV-1a has no diffusion after its last round and the last field of every address is `nth`, so an indexed draw only TRANSLATED the previous value — max circular shift **0.0351571** against ~0.5, and **1.75** distinct damage buckets from a ten-hit address instead of 7.56.

	before (`01be9daf14ee` **, pins** `2efbc9ed1946` **)**	**after (** `f9d6be635d34` **, pins** `f646b0163bc0` **)**
whole-game, counted	3 of 961 (8 raw, less 5 declared)	**14** of 961 (19 raw, less 5 declared)
board-material	1 of 961	**12** of 961

THE RISE IS THE INSTRUMENT SEEING WHAT IT WAS BLIND TO, NOT A REGRESSION, and it was predicted before the run. Attribution was checked rather than assumed: the two frozen releases differ in exactly one SOURCES file and its only executable difference is the three finaliser lines.

MEASURE'S PART IS THE REFUSAL, AND IT IS ALREADY WIRED. `DICE_MODEL` now rides in `PIN_DIGEST` (`engine/game_differential.js`), which `engine/arms_comparable.js` compares through `mode`. The gate prints, unprompted:

> *DIRECTION OF TRAVEL WITHHELD — the baseline was stamped under* `A/middle/pins:2efbc9ed1946/...` *and this run is* `A/middle/pins:f646b0163bc0/...`*. One pin is one corner: those are two instruments, and subtracting one rate from the other invents a trend.*

Without that, a pre-fix run and a post-fix run would have been tabled together as comparable and 3 -> 14 would have entered the record as an engine regression. **A number that quietly spans a reset is worse than no number.**

RE-STAMP DELIBERATELY OR NOT AT ALL. `node engine/quarantine.js --stamp-whole-game` moves the baseline to this pin. It has NOT been run here: whether `f646b0163bc0` is the pin to hold going forward is a MEASURE judgement, not an ENGINE one, and the withheld direction-of-travel line is the correct state until somebody makes that call.

Why the refit lives here, not in ENGINE or SEARCH

The refit is the expensive event on the one expensive edge, and it invalidates seven artifacts: counterplay, winrate-backtest, opponent-calibration, weight-multiplicity, then the mag / mew / scoreboard bundles. `provenance.js` derives that set rather than carrying a typed list of it.

The division that owns *knowing when a number stopped being true* is the one that should be pulling that trigger.

A restamp is only valid if the feature FUNCTION is unchanged. Damage table moved → refit. Not a restamp. There is no version of this where the shortcut is fine.

Open — in priority order

THE CLOSET IS A GATE EXEMPTION NOW, AND IT SHIPS EMPTY — 2026-08-27

Will's ruling, 2026-08-26: *"no if i put things into the closet it should not be gated — like illusion"*, and 2026-08-27: *"things in the closet shouldnt block a gate if we know why they fail and choose to accept it."* Implemented as a third `DECLARED_KINDS` entry, `CLOSETED`, in `engine/quarantine.js`. ROADMAP `#508`. Full account: `docs/_reports/2026-08-27-closet-gating.md`.

THE GATE DID NOT MOVE AND IT MUST NOT BE REPORTED AS THOUGH IT HAD. 5 of 8 clauses PASS before and after, measured on the same artifacts (release `f3d423e19e88`, 961 games, read at 23:47 EDT); whole-game `1 of 961` either way. A gate that improves because the rules changed is not an engine improvement, and this one did not even improve.

THE ONE ROW THIS KIND WAS BUILT FOR WAS NOT WRITTEN. Will closeted Tailwind's expiry order on 2026-08-24; `#493` FIXED it on 2026-08-27. The current differential holds six causes and none is a `-sideend` / `tailwind` pair, so a declaration would have registered a permanent exemption for a divergence that no longer occurs. `#355` is closed with that evidence and the refusal is recorded as a comment where the row would have gone.

WHAT THE DOOR ASKS, AND WHY IT IS A SCHEMA AND NOT A SENTENCE. Nine fields, refused at `declaredMatch` if any is missing: the owner, the date, his own quoted words and the register row; the instrument, release, date and finding of the measurement behind the no-board-effect claim; and what would make the entry wrong. This is the deliberate inverse of `NOT A DEFECT`, which is a regex over a register cell — *"a sentence somebody typed as a note"* is the failure mode this division was sent to avoid reproducing.

AND IT IS RE-CHECKED. Four declarations in this project have been refuted — speed ties, Tailwind's coin, Moody, and a fainted body in an active slot — and the die fix of 2026-08-27 voided every measurement taken before it. `closetEvidenceStale` compares the entry's evidence release against the release of the artifact it is excusing and prints `EVIDENCE NOT RE-CHECKED` naming both. It still subtracts; the owner ruled. An unstamped run reports nothing and never reports it fresh.

THE RECEIPT ON THE OTHER DOOR OVERSTATED ITSELF 8x, AND THAT IS FIXED AS A DISPLAY BUG. `□DEFECT□` matches inside the phrase `NOT A DEFECT`, so the open-defect clause reported every row carrying the phrase as excused. Measured over 432 register rows (205 open): **8 carry it, exactly 1 (`#252`) would have counted without it.** No verdict moves — the open set, the red set and the clause pass/fail are byte-identical. The nine rows named in `docs/_reports/2026-08-27-open-defect-clause.md` are now **eight**; `#344` was refuted and closed the same night, and the list was re-derived rather than inherited.

OWED. `#336` (the sleep draw ADDRESS, never checked) is still excused by the phrase and is the one of the eight worth re-opening: it claims nothing and carries a `WHAT WOULD DECIDE IT`, but *"the distribution is right and whether the draw is addressed identically has never been checked"* is the exact shape of the four refuted

declarations. It is not suppressing anything today — the receipt now proves that — so it does not hold the gate.

THERE IS NOW A SPEED BASELINE, AND THE BOX IS PART OF IT — 2026-08-27

Full account: `docs/_reports/2026-08-27-speed-baseline.md`. Will: *"i dont care about the old one i just want an honest baseline of how fast medicham can play a game out"* — so no prior figure is quoted or compared against; a number taken on a different engine is a different measurement.

One complete game: wall p50 7.4–9.7 ms, p90 10.6–14.1 ms, p99 14.4–19.1 ms, CPU 11.5–14.3 ms. Do not quote a max from it — in all twelve repeat runs the slowest game WAS the cold first game. n = 2,831 games in the headline run, all played to natural completion by a side wipe, **zero** hitting the 200-turn safety cap. Turns p50 7, p90 10, max 21. Release `6a845424c450` (HEAD, 0 of 26 files moved), `--team-store data/team-pool-frozen`. Composable primitive: **~1.1 ms per turn**; the turn loop is 98.7% of a game and turn count explains 60% of the variance in its length.

The cap does not bind. Only 4.4% of games reach turn 12 at all, so the whole-game differential's 12-turn cap truncates 1 game in 23 and is a no-op on the rest.

Nobody is searching in that number. It is `battleTurn(S, rng)` with no action arrays — medicham2's own greedy policy on both sides, the same call `battle()` and `previewLeaf` make. A MILTANK-driven game will run longer and cost proportionally more. Scale by your own turn count, never reuse 8 ms.

AND THE HEADLINE IS A RANGE FOR A MEASURED REASON, WHICH IS THIS DIVISION'S PROBLEM AND NOT AN ASIDE. The within-run noise floor (LESSONS §9, one arm split by interleave) is **0.006–0.152 ms**. On that floor almost anything is significant. It is the wrong floor: **twelve byte-identical repeats drifted 8.04 → 9.74 ms p50, monotone, 140× that floor**, and an idle gap took it back to 7.82 before it began climbing again. `cpu/wall` held flat at 1.40–1.45 throughout and free RAM did not move, so it is neither contention nor memory — it is a laptop shedding boost clock.

So: on this box, any A/B whose arms run at different times, on a difference under ~20%, is measuring the thermal state of the laptop. Interleave the arms or do not run the test. That binds MEASURE's own future work first.

Also measured and worth carrying: **CPU per game is 1.4× wall** (V8 GC/compile threads), so six concurrent processes want ~11 of 16 cores at ~500 MB RSS each — the documented six-process cap is tighter than it looks. And `tools\lownode.cmd` (BelowNormal) cost **0.069 ms**, below the noise floor: free, exactly as its header claims.

THE FIXTURE-LEGALITY GATE WAS RED AT FIVE AND THREE OF THE FIVE WERE THE GATE — 2026-08-26

Full account: `docs/_reports/2026-08-26-fixture-legality.md`. ROADMAP #266. CHANGELOG 5.151.0.

5 FAILED → **ALL GREEN**, baseline **22 → 15 verdicts / 23 → 15 pairs**, no allowance added. **Two real illegal entities** (`Incineroar can't learn Knock Off.`, one commit old; `Milotic can't learn Calm Mind.`, a ternary branch that can never be taken and typed the name anyway) and **eight stale allowances** that ENGINE had already repaired in `24fe4c5c` — proved a repair rather than a scanner that stopped looking, since `24fe4c5c^` declares all eight, HEAD declares none, and the file is still the sweep's largest contributor at 204 declarations.

The other three failures were the ruler. Helper detection judged a top-level `const` on a flat 500-character window that ran off the end of the declaration, so `const ok = (cond, label, extra)` inherited the `const stage = rows => rows.map(...)` beneath it and **every assertion in two files was scanned as a set declaration** — `'medicham='` read as the species Medicham. Fixed at the window; measured at **875 → 872 declarations, diff exactly the three phantom rows**, strays 5 → 0.

AND THE CHECK DOES NOT CATCH THE CLASS, WITH A COUNT. Three shapes walk past it. The largest is a positional row whose moves array is not in slot 2 — `stage(rows)` in thirteen files writes moves LAST — hiding **124 distinct sets, 21 of them illegal, 15 verdict sentences not on the baseline**. It was measured and **deliberately not armed**: those repairs change what their scenarios measure and belong to the divisions that own them, which is the same order matcher (C) followed. Second is `engine/validate_damage.js`'s golden master (two species and a move, all scalars, no array) — the file where a human, not this gate, found Choice Band, Choice Specs and an Amoonguss on 2026-08-25; audited by hand at **36 rows, 0 problems**, clean and still unseen. Third is rule 1's normalisation, which lets a punctuated fragment "name itself".

No game number moved and none could — no engine byte, no census, no differential, no release. Neither edited fixture was re-run; this batch may not play a game, so that is a PREDICTION and is recorded as OWED in the report.

CLOSED. ONE DECLARED LIST, ONE READER — THE MECHANICS CLAUSE COUNTED A ROW THE WHOLE-GAME CLAUSE HAD ALREADY DECLARED — 2026-08-26

Full account: `docs/_reports/2026-08-26-declared-list.md`. ROADMAP #464. CHANGELOG 5.147.0.

`DECLARED_DIVERGENCE` in `engine/quarantine.js` was consulted at exactly one site, inside `wholeGameClause`. That clause declared the Supreme Overlord `fallenundefined` line AUTHORITY-WRONG and subtracted its 5 games; `tests/test-mechanics.js` carried a live probe asserting we refuse the line deliberately; and `classifyMechanics` counted it as a defect on the same run, filtering on `!r.diverged || r.deferred` and never looking at the list.

It was never two artifacts that needed reconciling — it was one grammar with one reader. Both artifacts write `<cls> :: |lineA <> |lineB` from the same comparator, so the matching rule needed no loosening and was not loosened. `declaredMatch(cause, ev, threw)` is now the one door; the whole-game clause's inline loop was deleted rather than duplicated.

MEASURED, HEAD beside the working copy in one process against MD5-frozen artifacts (release `667278050dcf`): mechanics clause **9 → 8 of 16**, exactly one row leaving — `ability:supremeoverlord`, 112 teams in 13,116 open-sheet games, the most-played of the sixteen, so the reach filter would never have removed it. Whole-game clause **unmoved at 10 of 961** with a byte-identical `why` string; board-material **unmoved at 2 of 961**; census **unmoved at 750 live / 753 probed / 3 missing**; shelf list identical; `declared + counted + shelved + unknown + cleared = 16 = rowsSeen = summary`. **No engine byte touched.**

Nine selftest assertions added (**100 → 109 passing**), pushing a SYNTHETIC declaration rather than asserting anything about Supreme Overlord, because a gate built from an instance catches that instance and not the class. Shown red first: breaking the mechanics door (`const dec = null`) turns 4 red and leaves the whole-game one green.

What still walks past the door is named in the code's own header — `differentialClause` (numbers, no cause string), the three roster stages (our two engines, `DEFERRED-BY-OWNER`), `coverageClause` and `openDefectClause` (no cause string), and `orderProbeClause`, which DOES carry a cause and does not read the list. That last one is inert today (0 pairs probed, no ordering declaration) and is the nearest thing to the next instance of this bug. `SHOWDOWN-ONLY` is likewise a verdict no clause reads; those eight ability rows are handled correctly BY ACCIDENT. Both are recorded, neither is fixed here.

CLOSED. THE SWITCH INDEX WAS NOT THE BUG — 2026-08-25

Full account: `docs/_reports/2026-08-25-switch-index-instrument.md`.

ENGINE handed over the last two board-material "a chosen switch the authority performs and medicham2 does not" games with the harness named as the suspect: `engine/game_differential.js` sends `switch N` against a `side.pokemon` array Showdown **reorders** (`sim/battle-actions.ts:118-132`), so a cached index would not name the same body twice. Right first suspect — in this repo the ruler has been the culprit five times in two days — and **refuted**.

The instrument resolves that index off the LIVE array, by species, immediately before `battle.choose`. It now says so with its own counter rather than by being read: release `2ecd3bdc274b` , 961 games, **63,258 switch indices sent, 43,125 of them against an already-permuted party, 0 MISADDRESSED.** The counter has been shown RED — `MEDI_SWITCH_BY_INITIAL_INDEX=1` restores the cached-index bug and produces 4,932 misaddressed, 1,796 choices refused by Showdown and 901 of 961 games thrown. **Zero of the 23 first divergences on this release are a switch-addressing artefact**, so every board-material figure taken on this instrument stands.

The real mechanism is **ENGINE's and is one defect for both games**: medicham2 evaluates `preventsSwitch` at switch-EXECUTION time while Showdown evaluates it at CHOICE time and never re-asks, so a Gengar-Mega arriving on an earlier switch in the same turn retro-cancels a switch this engine had already been told to make. `tests/probe_trap_timing.js` isolates it in five arms with the authority's own refusal as the positive control. Predicted effect of the fix, stated before it is taken: 23 → 21 raw, gate 18 → 16 of 961, board-material 10 causes → 8, honest range 21–22 / 16–17 / 8–9.

A second instrument defect was found on the way and fixed: `freshBodies` dropped `_switchKey` , so it was `undefined` on every body this instrument has ever played and the medicham switch lookup has always fallen through to the mutable display name. CLAUDE.md already names that cause (Morpeko) and it was still live. Predicted before the run — misses stay 3, diverged stays 23 — and measured exactly so. A dead safety net looks exactly like a working one.

CLOSED. THE PLANTED-STATE PROOF WAS FAILING ON THE FIXTURE, NOT THE COMPARATOR — 2026-08-25

Full account: `docs/_reports/2026-08-25-planted-state-proof.md` .

`planted_state_proof_ok` has been **false on every committed artifact back to 24 August**, and `game_differential.js` exits 1 on it — so every board-material figure carried an unexamined caveat about whether the state comparator can detect anything at all.

It can. Thirteen of forty-two plants failed on the artifact's proof pair, and all thirteen were aimed at bodies that could not carry them: at the plant boundary side B is two corpses and all four benched bodies on both sides are dead. `board_state.js` holds the post-faint group — `item` , `status_counter` , `boosts` , `ability` , `vol` , `stall` — on a body both engines call dead, so seven volatile plants written into a fixed `S.actB[n]` moved no compared leaf and six bench plants correctly refused to plant at all. **The control is what settles it:** the same seven mutate functions handed a body that is standing are caught, localised, at the same boundary.

The pass/fail was a function of `--games` . `diff_swarm` picks by a deterministic stride, so the SECOND team of the proof pair moves with the sample size; the identical code at `--games 45` passes all 42 plants and at `--games 1200` fails thirteen.

Three fixture fixes in `engine/game_differential.js` , none of them a weakening: the volatile plants find a standing body and return the slot they used (a tighter localisation than four of them asserted before); `applied` now means the board MOVED where the comparator looks, asked of `BS.compare` rather than of the callback's return value; and a plant that cannot land at the last agreeing boundary walks back through the earlier ones — **on** `applied` **only**, never on `caught` , because retrying a plant that landed and was missed would hide the one thing this proof exists to expose. 42 of 42 CAUGHT+LOCALISED afterwards on the same pair, and `tests/test-state-differential.js` passes with `42/42 applied` on all six pairs.

The published board-material figures stand. `ROADMAP #314`'s consequence line — that DIFFERENT-END-STATE is a lower bound wherever quoted, including the 83 in CHANGELOG 5.54.0 — rested on those thirteen being indeterminate. They are the fixture, so that caveat is withdrawn. The committed artifact still carries the false flag until the next `--state` run regenerates it.

CLOSED. THE BROWSER RULEBOOK DRIFTED FROM THE NODE RULEBOOK, TWICE, AND NOTHING COMPARED THEM — 2026-08-25

Full account: `docs/_reports/2026-08-25-tags-drift.md`.

`data/tags.json` is what the node engine reads; `data/abra-tags.js` is the browser copy of it, frozen into every engine release. A `tag_dex` run at 2026-08-24 23:37 rewrote the first and left the second at its 22:59 content, **and both were committed that way** — so HEAD carried two engines disagreeing about three moves (Bug Bite and Pluck stealing any item instead of only a berry, Thunder Wave ignoring the type chart). Both params have live consumers in `medicham2-browser.js`. An earlier instance of the same drift ran for **two days**. Neither was caught by a check; the second surfaced because a release cut happened to look.

No new gate. `build/build_tags_js.js` gained a `--check`, copying the pattern `build/build_guru_js.js` has had since 2026-08-04, and `engine/artifact_audit.js` — already a gate, already the file for *"a derived artifact is not a fact until something compares it to its source"* — gained **check G**, which derives the pairs from the `GENERATED by <builder> from <source>` headers under `data/` and runs each builder's own `--check`. Nothing is re-implemented and nothing is typed.

It was **RED on the real drift** and green after `data/abra-tags.js` was regenerated — that regeneration is the only artifact this pass changed, and it changed the file to agree with a `tags.json` it should always have agreed with.

`build/build_browser_data.js` got the same `--check` and is now compared too: `data/move-effects.js` (frozen in every release, carries move priority) and `data/mega-formes.js` were derived from the Champions dex with **nothing standing between them and it** but an mtime test. Shown red on a deliberate break in a scratch copy. **Six bundles remain uncovered and check G prints them by name on every run** — the list is derived, so it is never quoted from here.

oooooooooooooooooo. MOODY IS DECLARED INCOMPARABLE; SPEED TIES WERE REFUSED, BECAUSE THIS HARNESS ALREADY SHARES THE TIE DIE — 2026-08-23

Full account: `docs/_reports/2026-08-23-declared-moody-ties.md`. Will ruled on both by name (*"yeah some things we can just quarantine as a known failure with a quoted reason (speed ties, moody, etc)"*, then *"yes we can have two things, impossible to compare, and too difficult and irrelevant to add at the moment"*). One of the two landed.

MOODY LANDED. `data/abilities.ts:2691-2716` picks the stat with `this.sample(stats)` twice, over the five main stats only, and `medicham2-browser.js:25831` implements the same rule over `['at','df','sa','sd','sp']` with a real draw — so the RULE is not in dispute and only WHICH STAT the die names is. The middle arm addresses a draw by `[seed, turn, category, move, target]` plus an occurrence index, and a residual `sample()` has no named category on either side, so both engines take it off the generic `any` stream at indices each fills with its own unrelated draws. 8 of 961 games. The row states that this is **the instrument's addressing, not a law**: give the residual pick a named stream on both sides the way ROADMAP #290 did for `tie`, and the row must be DELETED rather than kept.

SPEED TIES WERE REFUSED, AND THAT IS THE FINDING. The derivation handed over — `sim/battle.ts:429` speedSort ends in `prng.shuffle`, so a tie is a coin flip — is true of the GAME and false of this HARNESS, and only the harness's number reaches the clause. Showdown's shuffle is a **no-op** in every shipped arm (`sdShuffleReverses` false), medicham2's tied-group key has had **its own stream since 2026-**

08-20 (`RNG_STREAMS` includes `tie`) which the middle arm neutralises with `o.tie = () => 0`, and 3.74.0 fixed the tie at the root — which is why the two `tie-second` arms were RETIRED for "breaking a correct one". **The sixth die the declaration would have claimed does not exist is already shared.** So the 3 causes whose own probe reads `speed_tied: true, speed_gap: 0, same_priority: true` are a REAL turn-order disagreement, and declaring them would have subtracted a live defect under a heading reading "nothing to fix" — the `medicham2-browser.js:17440` failure with a better-sounding reason. They stay UNDECLARED and are proposed as a roadmap row.

THE TWO KINDS PRINT APART AND ARE NEVER SUMMED. `INCOMPARABLE` ("the authority makes a random draw we have no shared address for — no defect") and `AUTHORITY-WRONG` ("matching it would make us less correct") get separate headings and separate counts, plus `declared_by_kind` on the clause result. The clause moved `declared 5 -> 13`, `undeclared 43 -> 35` on 48 raw over 961 games; `diverged` and `declared` both still print, so the allowance is legible rather than absorbed. **It is still RED and nothing here could have opened it.**

THE THIRD KIND — DEFERRED — IS DELIBERATELY NOT BUILT, AND THE HOLE IS NAILED SHUT. A DEFERRED row asserts the opposite of the other two: that there IS a defect. `DECLARED_KINDS` is a whitelist, so a row typed with any other kind is NOT subtracted, counts as UNDECLARED and is NAMED on the run. The selftest pushes a `kind: 'DEFERRED'` row with `match: () => true` and asserts `declared=0, undeclared=1, ok=false`. The design for it, if it is ever wanted, is in the report.

THE NEGATIVE CASES ARE THE PROOF, NOT THE POSITIVE ONE. Nine mutations of the Moody cause — each a real defect wearing the same shape — were injected through the shipping clause and every one fell through to UNDECLARED: accuracy in the pool, evasion in the pool, `+3` instead of `+2`, `-2` instead of `-1`, our line unattributed, one of two occurrences unattributed, the authority attributing the boost elsewhere, the boost following that slot clicking a move, and a different slot on each side. Speed ties are asserted undeclared at gap 0 **and** gap 40, so re-adding the row goes red. Selftest: 108 passed, 0 failed.

ooooooooooooooooo. THE IDENTITY GATE IS A RUNTIME TRIPWIRE NOW, AND IT FOUND ON ITS FIRST RUN A DEFECT THE COUNT HAD BEEN GREEN ON FOR THREE WEEKS — 2026-08-23

Full account: `docs/_reports/2026-08-23-identity-gate-permanent.md`. Will: *"you do what you think is best just make it a permanent solution."*

What `tests/test-effective-identity.js` **asserts now.** Every mon a `Board` switches in gets a recording accessor on `ability`, `baseStats`, `weighthg` and `weightkg`. Every active on the test board holds a mega stone whose forme ability differs from every ability its base forme can have — swept out of the dex and the damage table, nothing named — so on that board **the declared ability is wrong for all four actives** and a raw read is a defect by construction. The board is then driven through the live decision path and every read is recorded with its stack. **Exactly one call site may see the raw field:** `board.js effective()` **.** That is Fowler's SELF ENCAPSULATE FIELD, which this file's header has cited since 2026-08-02, stated as an executable assertion instead of as a count.

IT FOUND A REAL ONE IMMEDIATELY. `engine/position_features.js:249` reads `f.mon.ability` off a live stone-holder to build the defender list for `priorityRefusedAbove()`. `engine/board.js:3520` is the same function one file over and **it resolves** — `effAbility(f.mon, dex)`. One fact, two implementations, which is the failure CLAUDE.md names as *FEATURES ARE PER-MODEL, FACTS ARE GLOBAL*. **The retired count ratchet was GREEN on that file and had been since 2026-08-02**, because `position_features.js` sits exactly at its per-file number of 5 and a count only ever asks whether the number went up. Exposure is zero today and the gate **re-derives that on every run** rather than asserting it: the value is consumed only by the

priority bar, and no legal mega in this format gains or loses a `blocksMove` ability. If one ever does, the gate goes red on that entry by name. MEASURE does not own `engine/position_features.js`, so it is declared with the guard and proposed as a roadmap row rather than edited.

SHOWN RED ON SIX PLANTED SHAPES AND ON ONE REAL BREAK. `ABRA_EI_PLANT=all` compiles six stale reads through `vm` — so they exist only while the knob is set and no scanner can ever count them as debt — and the gate named all six and exited 1: `mon.ability`, `const {ability} = mon`, `({ability}) => ...`, `{...mon}`, `Object.assign({}, mon)` and `mon[k]` with a computed key. **The last three were conceded by the old header as undetectable by any text scan.** Separately, `engine/board.js:3520` was edited to read raw, the gate named `engine/board.js:3520` with the offending source line, and the file was restored from a byte copy taken first (SHA-1 verified identical, `90ae57fe3bcaf886a29f8affc273bf437dd9d2af`) — not `git checkout`.

WHAT IT PROVABLY CANNOT DO, and this is stated in the gate's own header rather than here. Its coverage is EXECUTION coverage. It drives `board.js candidates()+featuresFor()` and `foeActionDistribution()` for all four actives, `position_features.js positionFeatures()` both sides, and `rollout_leaf.js rolloutWinProb()` both sides — the MAG feature path and the leaf. **The drive list is printed on every run** so a reader can check whether their consumer is in it. It does not cover `magnemite.js`'s live loop, the fitters, or the differential harnesses. It is also blind to a value copied out of the sheet before a Board existed and then treated as live: that is the old scan's `Object.assign` hole **moved rather than closed**, and it is written down as such.

THE COUNT RATCHET IS RETIRED, NOT RESTAMPED. `data/effective-identity-baseline.json` keeps every number it ever asserted under `last_count_baseline` (generated 2026-08-11, count 1198, 80 files) with the reason beside it; `--update` and `--propose` now refuse and exit 2. **Nothing was adopted.** The scan still runs as a printed inventory that asserts nothing, and the 32 walked-file notes are kept because each is somebody's line-by-line account of a file and that is the expensive part. One narrow static assertion survives — `baseSpecies(...).baseStats` at zero — because it is the one text shape a whole-repository walk named as dangerous and it reaches files the tripwire never executes. It is a supplement and the file says so.

The 18 behavioural assertions are untouched, including the three that build a body, mega-evolve it inside a real turn, and assert the effective ability is what gets read. The run is 24 passed, 0 failed, exit 0.

oooooooooooooooo. THREE OF THE FIVE FAILING GATE CLAUSES WERE STALENESS, NOT A BROKEN SIMULATOR — FULL REFRESH 2026-08-23 ON RELEASE `0faabe2a3f1b`

Full account: `docs/_reports/2026-08-23-gate-refresh.md`. The gate went **6 of 8 clauses failing to 3 of 8** without a single line of engine code being touched, because the three roster clauses had been measured against release `c36782953dee` and were correctly withheld as *an answer about other bytes*. Re-run on one fresh release: **items 0 DIFFER / 0 DID-NOT-FIRE, abilities 0 / 0, moves 0 / 0.** Named, because a row number is not a finding — the previously red entities *Big Root*, *Greninjite*, *Dragon Cheer*, *Fake Out*, *Matcha Gotcha*, *Psych Up* and *Transform* all now match.

THIS IS THE CASE THE PASS / FAIL-BECAUSE-BROKEN / FAIL-BECAUSE-UNMEASURED SPLIT EXISTS FOR. Five clauses read red this morning and three of them carried no information about the engine at all. A gate that cannot tell those apart trains people to read *closed* as *broken*, which is the same normalisation failure as "one of the two known failures".

AND ONE CLAUSE WAS BLIND FOR A THIRD REASON THAT LOOKED LIKE THE OTHER TWO. *"THE REACH FILTER CANNOT BE APPLIED"* was neither staleness nor breakage: `engine/all_mechanics_fire.js` defaults to `--kind moves`, so the published `data/all-mechanics-fire.json` held `rows.moves` and nothing else, and `quarantine.js:718` refuses to filter unless all three populations have

per-entity rows — falling back to counting every divergence unfiltered. Re-run `--kind all` and the clause answers for the first time: **29 of 36 diverging mechanics played and uncleared** (moves 22, abilities 12, items 2; 7 below the reach shelf), worst by reach *Cursed Body* 2,177 teams, *Toxic Debris* 1,840, *Disable* 1,799 clicks, *Regenerator* 1,596, *Poltergeist* 1,383, *Mental Herb* 967. **22 → 29 is a RE-BASELINE, not a delta** — different population and a filter that could not previously run.

The remaining two failures are real. Whole-game: 961 games on the `middle` arm, 73 parted less 5 declared = **68 undeclared (7.1%)**, of which the end-state comparison says **21 DIFFERENT-END-STATE and 52 wording-only**; direction of travel stays WITHHELD because the stored baseline is stamped under `top-tie-first` and this run is `middle` — one pin is one corner, and `--stamp-whole-game` was deliberately not run. Register: five open rows name a RED instrument (#218, #224, #241, #258, #273).

BLOCKED, NOT OWED: `engine/wire_ladder.js` **CANNOT RUN AT ALL.** It exits 4 and writes nothing — correctly. All **fifteen** arms pin releases frozen before `engine/medicham2-browser.js` exported `natureL50`, `rngStreams`, `spreadL50`; that is all fourteen distinct ids, and `compat` reports 168 of 351 releases predate an export. LESSONS §12 at ladder scale — `data/wire-ladder.json` stays UNSAFE, the release-ladder figure stays WITHHELD, and the unit of work is re-pinning the arms, never a re-run.

And leaf calibration — this division's one number — is still QUARANTINED and still withheld. A reliability curve measured through an engine that parts from the authority on 21 of 961 games is a claim about the wrong engine, and publishing it with a caveat is the bug. It becomes re-runnable when the gate opens, not true.

ooooooooooooooo. THE MUTATION ARTIFACT WAS SIXTEEN DAYS STALE, AND WHAT IT WAS HOLDING WAS NOT WHAT IT SAID — RE-RUN 2026-08-22 ON RELEASE `6fb9ebd3b704`

`data/mutation-coverage.json` had not moved since 2026-08-06 and was quoted as *"163 class-A rows, tag never read"*. Re-run against a fresh cut of the current tree. **The count is 148 and the count is the least useful thing in the file.**

It could not run at first, twice, and both refusals were correct. The triage calibration hand-decided `taunt / forbidsStatusMoves` as class A against release `032b4a2979dd`; ENGINE has wired it since, so the rule returned D and the sweep refused. The planted `speedMult` stub anchored on `s*=+_sm.mult`, which is now `_mods.push(+_sm.mult)`, and `plant()` threw rather than planting nothing. A hand answer is a claim about a BUILD; neither constant recorded which. Both repaired, `decidedAgainst` added, and the A branch moved to a synthetic that no fix can close — ROADMAP #326.

The two counts are not comparable and the per-key story is. The tag corpus grew 182 -> 292, so the battery scope moved `620f24df16ff -> c5c9acf2cc9d` and the ratchet correctly declined to compare them. Of the 163 old class-A operators: **10 NOW-LIVE** (Taunt, Knock Off's `failsIfNone`, Substitute, Poltergeist, Pollen Puff, Shed Tail — closed by ordinary engine work), **28 reclassified** to B/C/D, **11** became UNREACHED-BY-THIS-BATTERY, **3** downgraded, **6** no longer emitted, and **105 still class A**. 43 of the 148 are rows the old battery never had.

CLASS A IS NOT A DEFECT LIST, AND SAYING SO IS THE FINDING. 31 of its 36 carrier x tag rows carry an ARMED census probe that PROVES the mechanic works. I hand-read the other five: `move:leechseed / immunityGate` is graded A with *"Nothing in the simulator implements this fact"* and the Grass immunity is implemented at `medicham2-browser.js:18616` through a SIBLING tag the classifier cannot see. ROADMAP #323. Three real gaps survive — Trick vs Sticky Hold (**0 corpus uses**), `punishesMinimize` (Minimize is **32 of 198,840 sheet entries**), item Metronome (**27**) — total reach under 0.3%, filed LOW as ROADMAP #327 so they are not re-discovered rather than so they are fixed.

THE VEIN IS CLASS B AND C: 119 VALUES IN `data/tags.json` THAT NOTHING READS. The engine consumes the tag's membership and substitutes its own number. Protect's `shieldsUser` / `stallCounterChecks` / `targetClass` at **67.75%** of sheet entries, Fake Out at 12.44%, **Life Orb's `costsPerAttack` at 10.69%** — the row rediscovered the expensive way on 2026-08-21 after sitting in this file for sixteen days — Sucker Punch 7.11%, Flare Blitz and Wave Crash `recoil.readFrom`, Knock Off's `mult = 1.5`, the drain `num` / `den` on Matcha Gotcha and Giga Drain. None is a behavioural defect today; that is the point. It is *A DERIVED ARTIFACT IS NOT A FACT UNTIL SOMETHING COMPARES IT TO ITS SOURCE* at 119 live sites with no comparator. **The unit of work is one comparator, not 119 fixes** — ROADMAP #324.

And what is NOT a defect, said out loud because the counts grew fastest here. 98 UNREACHED-BY-THIS-BATTERY (was 32), 44 UNSTAGEABLE tags (was 25), 9 NO-CARRIER. The fixture did not keep up with a tag corpus that grew 60%. A COULD-NOT-STAGE verdict is a claim about the fixture, never about the mechanic, and none of these may be reported as breakage.

Pinned: engine release `6fb9ebd3b704` (26 files). The battery is NOT census-steered — `censusCrossRef()` reads `data/mechanics-census.json` only to ORDER class-A rows, so no census pin is needed for the sample; the census was digest `80e648f34d56` and was not regenerated. ENGINE cut `136a9894af62` while this ran, which is exactly what a release is for.

oooooooooooo. TWO OF THE THREE ROWS HOLDING THE THIRD GATE CLAUSE SHUT ARE FIXED AND CLOSED; THE THIRD IS RED FOR A NAMED REASON — 2026-08-19

`engine/board.js` (`stampSky`, `jointFeaturesFor`), `engine/magnemite.js` (`_candsFor`), `engine/seed_source_audit.js`, `engine/gate_weather_guard.js`, `engine/gate_seed_source_audit.js`, and twenty-four catch blocks across eighteen files. ROADMAP **#286 CLOSED**, **#287 CLOSED**, **#258 still open**. The clause `no open, known engine defect` went from naming **six** rows to naming **four** (#218, #241, #258, #290).

#286 — A GUARD THAT HAD NEVER BOUND, ON A FIELD NOTHING WROTE. `weatherSetupHelpsPartner` read `norm(A.__weather || '') !== w`, and `__weather` occurred exactly once in the live tree — that read. So the comparison was `'' !== w` and the "only if the weather is not already up" clause had never bound in the history of the file. `board.stampSky(cands, board)` is the assignment, and **both** candidate producers call it: `board.candidates` for the fitter, the fixtures and the rollouts, and `magnemite._candsFor`, which builds its menu from the live REQUEST and does not route through the first. Stamping one of the two would have been the fitting/playing mismatch this project has already paid for twice. The value is `board.weather`, the expiry-aware accessor #276 built.

THE SECOND DEFECT THE ROW RECORDED IS ALSO FIXED, AND THE EXISTING ARM COULD NOT SEE IT. The loop runs `[[A,B],[B,A]]` and asked `A.__weather` on both passes. Reverting that left `gate_weather_guard.js` at **exit 0** — because both candidates of a real pair come off the same board and carry the same stamp, so the wrong-half read agrees by accident. A fourth pass was added: with only the SETTER's field populated, the feature must answer the same in either argument position. That is exact rather than sampled, so it cannot false-positive — the bar the row's rejected static scan failed. Both halves were shown RED on a deliberate break before being trusted.

REACH, MEASURED WEIGHT-FREE, BECAUSE THE WEIGHTS ARE QUARANTINED. Over **2,000 corpus games, 21,757 joint decision points and 1,211,091 pair evaluations**, through `joint_rows.build` with a ZERO ranker and K above the longest candidate list so that no pair is dropped by a quarantined number, and with both arms taken from ONE process so ENGINE editing the simulator underneath could not separate them: the feature fired **344 times before the repair and 219 after — 125**

suppressed, 36.3% of every firing of this column was a redundant weather setup — on 64 of 21,757 decision points (0.29%). At 300 games the same quantity reads 49.2% of 59 fires, which is what a sample that small is worth. **THE ARGMAX-FLIP RATE IS WITHHELD RATHER THAN UNMEASURED**: each flipped pair's score moves by exactly the fitted weight for this column, and publishing a rank change would be quoting the quarantined vector.

AND THE LIVE HALF IS PROVEN, NOT ASSUMED. A 4-game self-play through the joint path reports `skyStamped=6779`, `jointSkyUnstamped=0`, `skyStampNotAnArray=0`. `tests/test-wiring.js` caught the first attempt outright — `_dropDeadVolatiles` returns `{cands, user, doubles}`, not an array — which is precisely the class of defect that file exists for.

A SEARCH-OWNED FILE WAS EDITED UNDER A MEASURE BRIEF. `engine/board.js` and `engine/magnemite.js`, on the router's instruction. It is on the record here rather than implied.

IT MOVES A FITTED JOINT COLUMN, AND NO REFIT WAS RUN. `weatherSetupHelpsPartner` means a different quantity after this than before it, so it joins the set the refit owes. The refit was ALREADY owed and is gated behind MEDICHAM rather than behind compute, so this adds a column to a debt that exists; it does not create a new one. Nothing was fitted, and nothing downstream of the weights was re-quoted.

#287 — AN AUDIT THAT SUBSTRING-MATCHED A HAND-TYPED LIST. `STUB_HAS_NO` is gone. `seed_source_audit.js` derives the class by calling `board.unmodelledBasePower(dex, board)` — asking each callback to produce a number and recording the ones that cannot — over a union of an empty board, its own staged board and every canonical fixture board, with the board count recorded and a fixture that will not build REPORTED rather than swallowed. Nothing reads a callback's spelling any more. The derived class is **four: avalanche, beatup, payback, ragefist**; the row's earlier seven, including Triple Axel, is exactly the figure a derivation corrects with no edit, because #283's `mv.hit = 1` made Triple Axel answerable after that list was typed.

THE ARTIFACT'S FALSE CLAIM IS GONE AND ITS STATE IS DERIVED. `openAndNotFixed` keeps its AUTHORED caveat text and reads OPEN/CLOSED from `docs/ROADMAP.md` through `quarantine.roadmapRowIsClosed`, imported and never copied. #244 reads CLOSED, so the caveat is DROPPED with its reason recorded rather than annotated; an unreadable register WITHHOLDS every caveat instead of publishing on an unchecked premise.

THE ROW'S REASON FOR NOT REGENERATING WAS MEASURED AND IS FALSE. It said regenerating today would bake ENGINE's in-flight `data/tags.json` into the artifact. With `engine/tags.js` and `data/tags.json` both made unreadable by a preload, the regenerated artifact is BYTE-IDENTICAL apart from its timestamp. The file's header already declared it does not read tags; that declaration is now checked rather than trusted, which is the difference between a claim and a measurement.

AND THE GATE ITSELF HAD THE LESSON. Arm 2 read `.row` off a single object. When the repair turned `openAndNotFixed` into `{checkedAgainst, rows, dropped}` the arm found ZERO assertions and went GREEN — a check that stops looking and calls the silence a pass, inside the gate written to catch exactly that. It was found by breaking the artifact deliberately and noticing the arm did not shout. It now accepts three shapes and REFUSES anything else with exit 2.

#258 — RE-MEASURED ON THE DAY, AND IT IS STILL RED. Both figures the row carried were stale. Today: **827 catch blocks, 291 silent (35%), 102 manufacturing; 201 baselined, 1 fixed, 91 NEW, 48 of those manufacturing.** All **24 of the new-manufacturing blocks reachable this session are fixed**, taking NEW 91 -> 67 and new-manufacturing **48 -> 24**. Each repair keeps the conservative value and adds the reason — a counter, a named `console.error`, or both — because inventing a different answer is a second defect rather than a fix. The most consequential were `mod_audit.js` x3 (three

reads whose EMPTY answer read as *"Champions changes nothing"* inside the tool whose whole job is that question), `champions_sim.js` (a `checkCanLearn` exception became a LEGALITY CLAIM about this format) and `quarantine.js` (a declared-divergence matcher that threw silently moved its cause into UNDECLARED and INFLATED the rate this file publishes).

THE 24 THAT REMAIN ARE NAMED, AND EVERY ONE IS IN A FILE ANOTHER DIVISION HELD THIS SESSION: `engine/game_differential.js` (6), `engine/tag_dex.js` (2), `engine/medicham2-browser.js` (1) and fifteen across `tests/`. **This is not a filed failure.** The gate is RED, the reason is stated, the reason is ownership, and the next holder of those files closes it. NOT re-baselined.

000000000000. THE HEADLINE WITHHOLDS, THE FIGURE WAS RE-TAKEN, AND FIVE ROWS THAT ASSERTED BREAKAGE NOW HAVE GATES — 2026-08-18

`engine/quarantine.js` (`--whole-game`, `clauseExit`), `engine/gate_fail_and_silent.js`, `engine/gate_weather_guard.js`, `engine/gate_seed_source_audit.js`, `engine/gate_offfield_target.js`. ROADMAP #298 / #241 / #286 / #287 / #218 / #224, and #299 / #300 / #301 filed.

#298 — THE MOST-QUOTED ENGINE NUMBER IS WITHHELD ON A RELEASE MISMATCH, AND CLAUDE.md HAD ALREADY DECIDED THAT. `wholeGameClause` was the only one of four clauses that compared no releases at all; `mechanicsClause`, `decisionImpact` and `orderProbeClause` have all refused an artifact cut against other bytes since they were built. It now refuses too, and on a mismatch **nothing measured comes back** — no rate, no `diverged`, no `games`, no class composition, and none of those numbers inside the verdict string either, because a withheld figure still sitting in the prose is the same bug with a different word in front of it. What comes back is which release it wanted, which it got, and the command that repairs it. It refuses a MISMATCH and not an ABSENCE: an unstamped artifact still answers, exactly as `orderProbeClause` allows one. The exit mapping is now `clauseExit`, one implementation shared by `--whole-game` and `--order-probe`, so a withheld verdict cannot exit 0 through one command and 2 through the other. `node engine/quarantine.js --selftest` is **87 cases, 0 failing**, four of them new and two of them asserting the red case twice over.

AND THEN IT WAS RE-MEASURED RATHER THAN LEFT WITHHELD. `engine/game_differential.js` on the tree's own release: **696 of 995 = 69.9%** (`data/game-differential.json`, release `978ca8fe72c9`, arm A/middle, 700 raw less 4 declared, 0 cleared on decision impact, planted-divergence proof caught, 0 threw). `node engine/quarantine.js --whole-game` prints it and exits 1. **READ THAT DENOMINATOR AS WHAT IT IS: every game played, including the ones where the instrument itself desynced.** It is not the rate among games the comparison was valid for, and #299 below is that finding.

IT IS NOT A BEFORE/AFTER AGAINST THE `690 of 1230` THIS WAS FILED ABOUT, AND THE PAIRED ARM IS WHY RATHER THAN A HEDGE. The driver steers off a census that is regenerated and draws pairs from a team pool read live from the store, so two runs a day apart do not play the same games — the same request for 1,230 games returned 995. `--release 6875c8ace00e` replays the OLD engine bytes under TODAY's steering, which is the comparison that isolates the engine, and the two arms are the same run to the game: identical primary-arm counts, identical class composition, identical `event missing from medicham2` block. **The engine moved by one game in one tie arm and by nothing at all in the primary one.** Every difference between 56.5% and 69.9% is the sample.

#299 — FILED, AND IT IS THE FINDING THIS RE-RUN ACTUALLY BOUGHT. The driver's own rule is that a game whose per-category draw counts diverge is VOID, not a divergence. Its stdout says so in three numbers; the artifact persists two of them. By the run's own arithmetic **72 of the 700 diverged games are themselves void**, and on the games where the instrument is valid the engines disagree about

628 of 645 = 97.4% rather than 69.9%. The honest figure is worse, not better, and neither one can be published today because the void count lives only in a console line nothing parses. The clause is green only at zero either way; what is wrong is that the published ratio is over two populations that are not the same one.

THE FIVE UNGATED ROWS, EACH WITH ITS INSTRUMENT AND ITS EXIT CODE. Every gate refuses a stale artifact with exit 2 rather than passing, and every one was shown red on its own selftest before being trusted.

row	instrument	exit	what it measured
#241(3)	`node engine/gate_fail_and_silent.js`	**1**	30 causes / 51 games, green only at zero
#286	`node engine/gate_weather_guard.js`	**1**	2 of 2 staged pairs still fire with the weather already up
#287	`node engine/gate_seed_source_audit.js`	**1**	both directions plus the false claim about a closed row
#218	`node engine/quarantine.js --whole-game`	**1**	the gating half, runnable at last
#224	`node engine/gate_offfield_target.js`	**0**	the defect is ABSENT on a current artifact

#241 — THE PIN WAS SEEDED AT 3 AND RE-SEEDED AT 30 THE SAME DAY, AND THAT IS THE LESSON RATHER THAN A CORRECTION. The gate exited 3 and said *this got WORSE*. It had not: the paired `--release` arm shows the class byte-identical across both releases under one sample. The population moved, not the engine. So the pin now carries the census digest and the team-pool digest out of the run's own steering block, and **a REGRESSION verdict is WITHHELD whenever they differ** — still red, still LIVE, but no movement attributed to a lever nobody measured. A pin without its sample stamp is a coincidence, and one integer is exactly as good at hiding that as the bare `25` in #295 was.

#286 — THE FUNCTIONAL ARM, AND THE STATIC SCAN STAYS REJECTED. The gate finds the pairs the shipping `jointFeaturesFor` itself scores as a weather synergy with no weather up, then re-asks the same boards with `board.setWeather` through the Board's own door and the candidates rebuilt through `B.candidates`. It names no move, no type and no weather — it reads them off the candidate it was handed. A third pass populates the guard field BY HAND and requires the feature to drop, which is not a pass condition and not a fix: it is the proof the gate is not stuck red, because a check no repair can clear is decoration.

#224 — CLOSED ON A MEASUREMENT, AND GATED ANYWAY. It had closed on the right kind of claim and then nothing re-took it for six days while the simulator was rewritten nightly. A measurement taken once and quoted afterwards is prose about a number. The gate reads the engine's own `traceBodyOffField` counter as well as the artifact text, and **does not count a stale artifact**: `data/divergence-turns.json` is stamped five releases back and is named and excluded rather than contributing an old zero.

TWO MORE ENGINE FINDINGS, FILED AND NOT TOUCHED. #300: the catch that reports a failed Showdown wrap assigns a `let` declared five hundred lines below it, so a bad `SHOWDOWN_PATH` kills the run with a temporal-dead-zone error naming neither the path nor the require — the failure REPORT is the part that does not work. #301: `switch lookups that MISSED: medicham 22, showdown 0`, on a line the run itself marks MUST READ 0, identical on both releases, caught today only by a human reading stdout.

NOT TOUCHED, DELIBERATELY: #220 (with ENGINE), and #273, whose `tests/probe_red_demo.js` exits 1 and reads PREMATURE CLOSE in `register_reality.js` — `tests/` is ENGINE's. `register_reality.js` exits 1 today for that row and that row alone.

000000000000. THE REACH SHELF IS ONE ANCHOR AND A RATE, AND THE REGISTER WIRE HAD NEVER CARRIED A ROW — 2026-08-18

`engine/quarantine.js` (`--reach`, `--order-probe`, `--selftest`), `engine/register_reality.js`, ROADMAP #295 / #296 / #297 / #298.

#295 — THE SHELF WAS TWO THRESHOLDS WEARING ONE NUMBER, AND FIXING IT MOVED NOTHING. The literal 25 was compared against CLICKS in a 64,846-game population and against TEAMS in a 13,116-game one, so one integer meant **0.0020% of clicks** or **0.095% of teams** — a move cleared the bar at roughly **48x lower relative usage** than an ability. Nobody decided that. The anchor is unchanged and now carries its unit in its name (`REACH_SHELF_CLICKS = 25` , from `tests/roster.js:1517` 's `USAGE_SHELF_BELOW` , applied at :1522 as `clicks >= USAGE_SHELF_BELOW` — a clicks threshold, ruled on as a clicks threshold; Will, 2026-08-18: *"leave it at 25"*). The common unit is **occurrences per stored game**, which both instruments already express: `clicks / click-counts.store_games` (64,846) and `teams / sheet-usage.sheet_games` (13,116). The teams threshold is derived at the anchor's rate — 25 x 13,116 / 64,846 = **5.06 teams**, counting from 6 — and the comparison is integer cross-multiplication, so the move shelf is preserved bit-for-bit at `n < 25` and no float boundary can move a row.

	before	after
counted (played, uncleared)	**34**	**34**
shelved by reach	**15**	**15**
rows that moved		**none, in either direction**

IT REPRODUCES AND THAT IS STILL A NEW THRESHOLD. No diverging ability sits between 6 and 24 teams — the lowest counted one is `angerpoint` at 25 — and every shelved row is a move decided by the unchanged anchor. So the equality is a property of today's population, not of the change: an ability diverging at 6–24 teams used to be shelved and now COUNTS, which is the direction the defect demanded. **Anchoring the other way does not reproduce**: take the ability figure as the anchor and the move shelf becomes 124 clicks, shelving twelve currently-counted moves from `move:ficklebeam` (112) down to `move:attract` (30). Same fix, opposite result — which is why the anchor is named rather than assumed. Measured on `data/all-mechanics-fire.json` release `488fd1bf3f7c` .

THE DENOMINATOR NOW TRAVELS WITH EVERY COUNT. `reachOf` carries `denom` / `denomLabel` , the clause prints `2177 teams/13,116 open-sheet games` , and `--reach` prints the population, the rate per 10k games and the applicable shelf on every single row. That was the actual defect: a bare integer beside a name cannot carry its unit, and one header line is not a fix. Two further literals were untangled — `coverageClause` reads the CLICKS anchor because its loop walks `click-counts.json.moves` and nothing else, and the decision-impact sample floor became `DECISION_POINTS_FLOOR` , the same 25 in a THIRD unit.

#296 — THE GATE READ THREE KEY NAMES THE VERDICT ARTIFACT HAS NEVER WRITTEN. The open-defect clause read `rr.rows` , `v.command` and `v.exit` ; `engine/register_reality.js` writes `results` , `cmd` and `green` . **Zero rows had crossed that wire since the day it was built.** Every open row fell into the DEBT bucket, `withRed` was structurally empty, and the clause printed *"no open row names an instrument that is RED"* for the reason that should have made it loudest — while #258's instrument was exiting 1. That is `merge_mega_into_engine.js` to the letter: matched on nothing, reported success. The fix is a declared shape (`REGISTER_REALITY` + `registerRealityRows`) that the WRITER writes through, not a tolerant reader that would hide the next rename; an artifact carrying no recognised array now returns NULL and says which keys it found, because *no rows* and *I cannot see the rows* are the two answers this bug confused. **Consequence, stated plainly: the clause now FAILS.**

#297 — THE CLOSED-DETECTOR LET PROSE OUTRANK AN `open` STATUS CELL. `roadmapRowIsClosed` honoured a cell saying `closed` and ignored a cell saying `open` , so a row whose narrative contains `CLOSED 2026-08-11` about a part that IS closed read as a closed ROW. Measured over all 217 register rows before changing it: **exactly five verdicts move, all closed → open, three of them asserting breakage — #218, #220, #224.** #220 is the expensive one: its `-fail` vs `-singleturn ... protect` family

is **238 games of the 695** in `data/game-differential.json`, the largest single family in the whole-game differential, sitting in a row the gate could not see and `open_work.js` was not printing. The open-defect population went **5 → 8**.

#298 — FILED, NOT FIXED. `wholeGameClause` publishes its rate with no release check while `mechanicsClause`, `decisionImpact` and the new `orderProbeClause` all refuse an artifact cut against other bytes. Today's `690 of 1230` was measured on release `6875c8ace00e`; the tree moved twice while this was written. Making the headline clause refuse withholds the project's most-quoted engine figure and is a judgement with an owner.

ROADMAP #290 — THE ORDER PROBE IS A GATE NOW, AND `ordering 229` IS NOT ONE POPULATION — 2026-08-18

`node engine/quarantine.js --order-probe` is the instrument #290's `INSTRUMENT OWED` specified: it FAILS while any `order_probe` row carries `speed_tied: false` AND `same_priority: true`. Exit 1 = the defect is present, exit 2 = the clause CANNOT ANSWER (no artifact, no probe, or a probe cut against other bytes), 0 = clean. Both non-zero codes are RED to `register_reality.js` deliberately: a `VERIFIED BY` exiting 0 on a missing artifact would close a live defect.

ON THE LAST ANSWERABLE ARTIFACT (release `6875c8ace00e`, 1,230 games, arm A/middle): 26 move-vs-move pairs probed, 15 tied, 11 CARRYING THE CONJUNCTION across 11 distinct games. Speed gaps of 10, 20, 30, 32, 42, 56, 64, 76, 76, 147 and 213, every one at identical priority with every die pinned on both sides. Nine are Tailwind, three are Protect.

THE TWO NUMBERS ARE NOT THE SAME POPULATION AND THEY ARE ONE APART BY ARITHMETIC. They are two different groupings of one run: `classes[].cls === 'ordering'` is **228 games**, while the gate's composition `ordering` is `divergence_shape.shapeOf(cause)` summed across ALL classes, which is **229**. Cross-tabbed:

	games
shape ORDERING **and** class `ordering`	**223**
shape ORDERING inside class `event missing from medicham2`	**6**
class `ordering` that shapes as EMISSION	**5**

223 + 6 = 229 and 223 + 5 = 228. **The counts differ by 1 while the sets differ by 11 games.** Reading them as one number was about to attribute a whole class to the wrong cause.

WHAT IS ESTABLISHED IS 11 GAMES, NOT 228. The probe covers move-vs-move pairs only, so 11 is a FLOOR on the ordering class and never a count of it. Extrapolating 11/26 across 228 gives ~96 and is an extrapolation wearing a measurement's clothes; it is not published. The evidence is also one release stale — the differential ran at 04:09Z on `6875c8ace00e`, release `488fd1bf3f7c` (#294, the per-target accuracy roll) was cut at 05:05Z — so the gate exits 2 today and the count is WITHHELD rather than annotated.

THE FIVE UNMEASURED ROWS, WORKED — 2026-08-18

row	outcome	instrument	exit
#258	**CONFIRMED** — open, red instrument	`node tests/test-no-silent-failure.js`	**1**
#290	**CONFIRMED** — gate built, defect demonstrated on the last answerable artifact	`node engine/quarantine.js --order-probe`	**2** (cannot answer: artifact one release stale)
#241	**CONFIRMED, re-measured** — part (3) is **3 causes / 3 games** on the current artifact, not the stale upper bound of 8	INSTRUMENT OWED (a ratchet, which the re-run now makes possible)	—
#286	**CONFIRMED by measurement** — `__weather` has 1 read, 0 assignments, 265	INSTRUMENT OWED (the functional arm; the static scan was built, measured at 7 false positives of 8, and	—

row	outcome	instrument	exit
	frozen copies	rejected)	
#287	**CONFIRMED by measurement** — both directions reproduce, and the artifact still asserts closed #244 is open	INSTRUMENT OWED (compare against `board.unmodelledBasePower`, which exists at `board.js:3239`)	—

A RED TEST FOUND IN PASSING AND NOT FILED AS A STATUS: `register_reality.js` reports #273 as a **PREMATURE CLOSE** — the row is closed and `node tests/probe_red_demo.js` exits 1. `tests/` is ENGINE's and was live while this ran, so it is reported here by name rather than touched.

0000000000. THE MEDICHAM GATE'S FINISH LINE IS DECISION-EQUIVALENCE NOW, AND THE FILTER IS DERIVED FROM THE STORE — 2026-08-18

`engine/quarantine.js` (`--reach`, `--selftest`), `data/click-counts.json`, `data/sheet-usage.json`, and a contract for a `data/decision-impact.json` that does not exist yet.

THE BAR IS WILL'S. *"medicham needs to be good enough that when miltank uses it, it wouldnt affect any decisions"*, with the worked example *"so like if trick or treat isnt working, that doesnt matter because no one uses gorgeist."* Two clauses were asking for something else. The mechanics clause counted every diverging entity whether or not anybody plays it; the whole-game clause demanded literal zero over a unit nothing was allowed to filter. Neither is a weaker bar now — they are a different question, and the answer to it can actually be delivered.

FILTER 1 — REACH, at the threshold the project already had. A diverging mechanic below **25 observations in the store** is shelved: still staged, still played, still printed with its usage, and it stops counting. 25 is `tests/roster.js`'s `USAGE_SHELF_BELOW` and the coverage clause's shelf, not a new number; `coverageClause` used to carry its own literal 25 and now reads the same constant. The unit is whatever the store can observe for that kind — **clicks** for moves (`engine/click_counts.js`, 64,846 games, 1,259,717 clicks), **teams** for abilities and items (`engine/sheet_usage.js`, 26,232 teams from 13,116 open-sheet games). Both denominators print on every run because 25 teams and 25 clicks are not the same exposure and must not be read as if they were.

`tests/roster.js`**'s header says no honest store-derived ability usage exists. That was true on 2026-08-10 and** `data/sheet-usage.json` **was generated on 2026-08-11** — *"The first honest store-derived ability usage this project has had."* The shelf reaches abilities and items because the instrument now exists, not because the rule was relaxed.

UNKNOWN IS NOT ZERO, and that half is what makes the filter honest. An entity the usage instrument STRUCTURALLY CANNOT SEE is not below the shelf: it counts, and it is named in its own column. `data/sheet-usage.json` declares that set itself — a team sheet reveals the PRE-MEGA ability, so an ability that lives only on a mega forme can never appear in it. Absence from an instrument that covers the whole class (every move click in every stored game) is an observed zero and is a different thing. If a usage artifact is missing outright, every row of that kind is UNKNOWN and nothing is shelved.

THE FILTER HAD TO NOT USE `tags.json`**, AND THIS ALMOST WENT WRONG.** The nine entities handed to me as unplayed carried `tags.json.uses` figures. Against `engine/click_counts.js`: bittermalice reads **0 there and 519 real clicks**, attract 2 against 30, fairylock 1 against 11. A reach filter run off `tags.json` would have shelved a 519-click move as unused — ROADMAP #70 landing on a live decision for the second time.

FILTER 2 — DECISION IMPACT, wired as a contract and refusing everything today. For a mechanic people do play, the question is whether fixing it moves the argmax, and that is a measurement, not something a clause may guess. `engine/argmax_paired.js` (ROADMAP #278) is the instrument. The clause reads `data/decision-impact.json` and clears a row only when **all** of: the artifact exists; `null_demonstrated:`

`true` (its identical-dice control reported exactly 0 flips); its `engine_release` equals the tree's; and the row reports `flips: 0` over `paired >= 25` decision points and names the arm the fix landed in. The 95% upper bound from the row's own n (rule of three) prints beside every cleared row — **0 flips in 25 is an 11% bound, a floor and not a zero.** The same contract serves the whole-game clause through `cause:` rows, which is the only filter that clause gets: a game is not an entity, 690 of its divergences are `ordering` and `field` classes naming no mechanic, and thresholding those on usage would be a fabrication.

NOTHING IS CLEARED TODAY. There is no `data/decision-impact.json`, and the clause says so in the same sentence as the count — an exemption mechanism silent about being empty makes "0 were cleared" and "we did not check" the same line.

THE `drag: a different body` DECLARATION IS WITHDRAWN. It was declared the same morning as "not a rule, a bench index", and the mechanism was right: `sim/battle.ts` `getRandomSwitchable` samples `side.pokemon` order, the authority takes one index under the pinned die and this engine takes another. The CONSEQUENCE was wrong for this bar. A different body arriving on the field is a different position, and every decision after it is taken against a board the authority does not have — the largest decision impact a divergence can have, not the smallest. It was never a claim that the authority is wrong, which is the only thing `DECLARED_DIVERGENCE` is for. **Supreme Overlord's `fallenundefined` stays**: an unguarded `onEnd` emitting a literal broken string on a `[silent]` line is a typo, and reproducing a typo is not correctness.

AND THE RECEIPT FOR KEEPING ALL THREE BARS NARROW. `medicham2-browser.js:17440` carried a declared divergence whose stated reason was COST — *"a far larger change than this wire is buying"*. It hid the largest real defect in the engine: Showdown rolls spread accuracy per target, this engine rolled once per move, so Rock Slide (18,122 clicks) and Heat Wave (11,121) could never hit one body and miss the other. At 90 accuracy the exactly-one case is 18% of outcomes and did not exist here at all. **That one clears a reach filter without breaking stride.** Reach may never be the only filter, and no row may reach any of the three axes by a cost argument.

MEASURED, on `data/all-mechanics-fire.json` generated 2026-08-18T04:06Z (release `6875c8ace00e`; the tree is `29ddfe81594a`, so the clause refuses these rows as stale and `--reach` prints them anyway — a listing is not a verdict):

diverging mechanics, closet excluded	**49** (moves 35, abilities 14, items 0)
below the reach shelf	**15**, all moves, 0–22 clicks
no usage figure	**0** — every diverging row has a store-derived number
cleared on decision impact	**0** — no run exists
counted: played and uncleared	**34**

THE WORKED EXAMPLE DOES NOT CLEAR THE SHELF AND THAT IS REPORTED, NOT PAPERED OVER. Trick-or-Treat is **70 real clicks**, not the 10 `tags.json` reports, so at a shelf of 25 it would hold the gate. Gourgeist-Super is **13 of 26,232 teams (0.05%)**, comfortably below. The move outlives the body that brings it. Raising the shelf to cover the example needs a number ≥ 71 and that is a decision with an owner; picking one here to fit one anecdote is not.

Shown RED on a deliberate break before being trusted, all three: unknown-reads-as-zero, `null_demonstrated` ignored, and the shelf boundary moved to `<=` (an ability sits at exactly 25 teams today, so that off-by-one would have excused a live row). `node engine/quarantine.js --selftest` is 56 cases, 0 failing.

000000000000. ROADMAP #258 — THE SILENT-CATCH RATCHET: MEASURE'S OWN FILES ARE CLEAR, THE FLOOR FELL 216 → 201, AND 75 REMAIN IN OTHER DIVISIONS' FILES — 2026-08-17

`tests/test-no-silent-failure.js`, `data/silent-catch-baseline.json`.

THE GATE IS STILL RED AND I AM NOT FILING IT. The row is CONFIRMED against its own instrument by `engine/register_reality.js`, which is the honest state: the row asserts breakage and the gate exits 1. What changed is that the remaining population is now entirely outside this division.

MEASURE TOOK EVERYTHING IT OWNS. Fifteen silent catch blocks in `provenance.js` (8), `sprt.js` (4), `mew.js` (1), `stamp.js` (1) and the ratchet itself (1) now speak, and `--update` locked the fixes in: **baseline 216 → 201, ten keys removed, nothing laundered.** `node tests/test-no-silent-failure.js --in <file>...` lists what is left in any named file, so a division can work its own without grepping a repo-wide list — the second-scanner shape CLAUDE.md names.

ONE OF THEM WAS HIDING A LIVE DEFECT IN THE STALENESS CHECKER, WHICH IS THE ROW'S WHOLE ARGUMENT. `provenance.js`'s corpus classifier follows one level of `require('./x')` to decide whether an artifact's game count should be judged against the LADDER or the OPEN-SHEET ceiling. It resolved every such token against `engine/`, so for any generator living in `tests/` it resolved nothing at all — the read threw, the catch returned `''`, and the `games.(ots|bo3).jsonl` test ran against the empty string and returned the same `'ladder'` it returns for a generator that genuinely reads the ladder. **A capability absent, reporting success**, inside the tool whose job is catching that. The moment the catch was made to speak it named six files. Fixed by resolving against the requiring file's own directory and by reading the comment-stripped source (`engine_release.js` documents its API with `require('./x.js')`, which was being resolved as a dependency). **Verdicts before and after are byte-identical on today's tree** — the fix removes a hole rather than moving a number.

THE RATCHET COULD NOT SEE ITS OWN BLIND SPOT. A source file it failed to read was skipped in silence, so `files scanned 333` and a clean bill of health for 333 files were indistinguishable from a clean bill for 332 plus one nobody looked at. It now counts, names and FAILS on it. Shown red on a deliberate break before being trusted.

WHAT IS LEFT, RANKED, AND IT IS A HANDOVER RATHER THAN A LIST TO WORK THROUGH. Re-measure on the day — `node tests/test-no-silent-failure.js --all` — never this paragraph; the population moved twice while this was being written as SEARCH added files. As of 2026-08-17 it is **75 above the floor, 41 of them manufacturing a value**, and the manufacturing ones go first because each hands a made-up answer to whatever reads it. By file, manufacturing-first:

owner	file	mfg	the shape
ENGINE	`tests/roster.js`	5	`buildableSpecies` returns **false** when `mcKey` throws, so *unbuildable* and *not in this format* are one answer — the `buildMon("Scizor")` shape; and a precondition that THROWS is reported as `COULD-NOT-STAGE`, a claim about the fixture
ENGINE	`engine/mod_audit.js`	3	`return null` / `return []` — an empty override set reads as *Champions changed nothing*
ENGINE	`engine/tag_dex.js`	2	`return _ptShape` / `return null` on the tag derivation
ENGINE	`engine/game_differential.js`	2	`base = null`, `rt = null`
ENGINE	`engine/mega_census.js`	2	`return null` — the comment already says null must never read as *nothing is a mega*
ENGINE	`engine/champions_sim.js`	1	`ok = false`
ENGINE	`engine/medicham2-browser.js`	1	`rows = null`
ENGINE	`engine/merge_mega_into_engine.js`	1	`priors = null` — this is the builder whose 67 writes once all missed

owner	file	mfg	the shape
ENGINE	`engine/conformance.js`, `engine/scenario_catalogue.js`, `engine/fixture_legality.js`, `tests/staged_board.js`, `tests/probe_volatile_leaves.js`, `tests/test-switch-carry.js`, `tests/test-forme-assert.js`	1 each	fixture and probe paths; `test-forme-assert.js:113` returns **false** from a `buildMon` throw
SEARCH	`engine/million_run.js`	2	`return false`, `k = null`
SEARCH	`engine/rollout_switch_census.js`, `engine/rollout_switch_probe.js`, `engine/click_counts.js`	1 each	`return null` from a census reader
MEASURE-adjacent, unclaimed	`engine/leaf_engine_contrast.js` (2), `engine/diff_swarm.js` (2), `engine/million_targets.js` (2)	6	these sit on the leaf/target path and no division ledger names them; routed on request rather than taken unasked
WEB	`tests/test-web-quarantine-loaders.js`, `tests/test-web-quarantine.js`	1 each	`w.ABRA_STATUS = null`, `age = '(mtime unreadable)'`
OPS/infra	`tests/test-lownode.js`, `tests/test-farm-ram-guard.js`, `tests/test-roadmap-register.js`, `tests/test-unmodelled-clicks.js`	1 each	**three of these four are false positives of the detector**: `sawFailure = true; code = e.status`, `threw = true` and `log = null` all record the failure and are asserted on the next line. The detector spots a recorder by NAME (`/fail\w*\s*\(/`), which `noteReadFailure` and `sawFailure` do not match. Not widened here: a regex guarding 801 catch blocks is not the place to buy four rows

0000000000000. ROADMAP #241 / #276 / #283 — THE THREE ROWS NOTHING DECIDES, NOW SAYING SO — 2026-08-17

`engine/register_reality.js`, `data/register-reality.json`.

Every one of the three is OPEN, asserts breakage, and had no instrument named — so the MEDICHAM gate was being held shut by three sentences. Each had a plausible-looking candidate gate that decides something else, and pointing a `VERIFIED BY` at any of them would have made a live defect read CONFIRMED-and-green, which is worse than the prose it replaced:

- **#241** — `engine/game_differential.js` MEASURES the missing `-fail` emissions and deliberately exits **0** on them (*"a divergence is a FINDING"*, its own header), and `tests/test-game-differential.js` fails only when the INSTRUMENT is wrong. Nothing decides the row by exit code.
- **#276** — `tests/test-seed-clock.js` is GREEN and decides **#270**, the SEED's clock. #276 is the BOARD's own `weather` field, which no gate reads.
- **#283** — `tests/test-rollout-fallen.js` is GREEN and decides **#244/#245/#246**, the SEED's roster. #283 is `board.movePower`'s stub, which no gate reads.

So `register_reality.js` gained a second marker, `INSTRUMENT OWED:`, which records that NOTHING decides a row and names what would have to exist. It is counted and printed **separately from the verified figure**, because a declared debt is not a measurement.

0000000000000. ROADMAP #285 — THE DOCS-CURRENCY CENSUS WAS EXEMPTING FIGURES ON THE STRENGTH OF ITS OWN COMPLAINT ABOUT THEM — 2026-08-15

`engine/docs_scan.js`, `tests/test-docs-current.js`, `data/docs-currency-baseline.json`.

THE GATE IS RED AND I AM NOT FILING IT. `docs/ABRA-whitepaper.md: 12 untraceable figures, was 11`. The whitepaper has not been edited since 2026-08-10; the regression came from underneath it. Three instrument defects were found chasing it, all three fixed and shown red; **the figure itself is not resolved** and #285 says exactly what the next actor needs.

1. **SELF-REFERENCE — a gate that could satisfy itself.** `allArtifactNumbers` unions every `data/*.json`, and the baseline IS one. It records the census's own findings as strings, and the artifact walk reads numbers out of strings, so **writing a figure down as an offender made that figure traceable on the next run.** Demonstrated by building it worse: recording the untraceable SET dropped the whitepaper from 12 to 5 in one run. The loop had been live through `citation_mismatches` since that list was created. **And the same loop arrives through the REGISTER**, caught in the act within the hour: `data/open-work.json` is a copy of ROADMAP.md's prose, so any figure quoted in a register row becomes traceable. The row filed about this defect quoted the offending value while describing it, and the clause went green with the real regression still underneath. **Writing down "this number has no source" must never become that number's source.** It was masking **nine figures across three documents**; closing it takes `docs/MEDICHAM-SPRINT-NOTES.md` from 25 to 32 and restores `docs/ROLE-FAMILY.md`. Those are a DISCOVERY, not a regression, and the counts were **not** raised to match — that split is ROADMAP #188.
2. `--update` **WAS #258's LAUNDERING BUTTON, IN A SECOND FILE.** It turned a failing clause green *and* adopted every new offender into the floor, and the write gate was `if (F === 0 || UPDATE)` — so a red run could record its own regression. Monotone now, and proved: `--update` leaves the artifact byte-identical and exits 1.
3. **IT RATCHETS A COUNT IT CANNOT EXPLAIN.** The baseline said `11` and not *which* eleven, so the twelfth is not recoverable from the tree. A count also launders by SWAP — fix one, add one, count unchanged, gate green.

ONE HAND EDIT TO A RATCHET, AND THE ONLY REASON THAT IS EVER ALLOWED: the self-referential run had written the counts DOWNWARD (whitepaper 5), which is stricter than the truth and would have left the gate permanently red against a floor no correct run could meet. They are restored to the values the file held at `2026-08-15T03:11:13.624Z`, with the reason written into the artifact beside them.

oooooooooooo. ROADMAP #266 — THE FIXTURE-LEGALITY RATCHET COULD BE LAUNDERED, IT WAS UNDER-COUNTING, AND ONE ENTRY WAS A DIFFERENT DISEASE — 2026-08-14

`tests/test-fixture-legality.js`, `engine/fixture_legality.js`, `engine/champions_sim.js` (the shared oracle), `data/fixture-legality-baseline.json`, and the fixtures repaired in `tests/test-protocol-trace.js` and `tests/test-click-censoring.js`.

THE COUNT, AND WHY THE ROW'S 41 WAS NOT IT. Read from `data/fixture-legality-baseline.json` and `engine/fixture_legality.js`, never typed:

verdicts open at the start of this pass	32 (the row's 41, minus the ten repaired earlier the same day)
illegal DECLARATIONS behind those 32	**34** — `pairOrigin.count`
verdicts now	**22** — `count`
declarations now	**23** — `pairCount`
repaired in this pass	**10 verdicts / 11 declarations**
UNREACHABLE — no legal carrier anywhere in the regulation	**1**

THE VALIDATOR STOPS AT THE FIRST PROBLEM PER POKÉMON, SO THE RULER WAS SHORT. A Snorlax declared with Swords Dance, Iron Defense, U-turn and Roar — four moves it cannot learn — produces the single sentence *"Snorlax can't learn Swords Dance."* Two declarations were hidden that way, and the cost is worse than a low count: repairing the first illegal move in a set makes the second appear as a **NEW** verdict, so a repair reads as a regression. Every declared move now also goes through `TeamValidator#checkCanLearn` and the baseline ratchets those pairs beside the sentences.

THE CLASSES ARE NOT ONE PROBLEM. Of the 34: **0** named a species this format does not contain, **1** named an entity with no legal carrier at all, and **33** were a legal body holding something a legal body somewhere else can hold — re-aimable, which is what the repairs did. The one that matters most is the smallest: **Spore has ZERO legal carriers in Champions Reg M-B**, derived twice (whole-roster `checkCanLearn`, and an independent raw learnset walk through the prevo and base-forme chains; both answer 0), and `tests/staged_status_counters.js` stages its two sleep-counter scenarios on it. That is the shape that manufactured four phantom engine defects in one session — a probe fails, it reads as an ENGINE DEFECT, and the engine was correct not to model a thing the format cannot produce. **The repair is not a rename**: every sleep move that does have a carrier here is sub-100 accuracy (Hypnosis 60, Sleep Powder 75, Sing 55) and both scenarios run on the `top-tie-first` arm, where every sub-100 move MISSES by construction. Milotic can legally learn Hypnosis, so the mechanic is reachable on the bottom arm and nowhere else. **Held, not repaired** — that file belongs to the owner of the differential and moving a scenario between pin arms changes what it measures.

THE LAUNDERING BUTTON, THE SAME SHAPE AS #258's `--update`. "Repair a verdict" and "excuse a new one" were the same edit: a line out, or a line in. The label `PRE-EXISTING` is a **claim about time** and nothing checked it. The 41 sentences that existed when the check was added are now written down as a closed `origin` set, and a PRE-EXISTING entry that is not one of them fails by name. A genuinely new illegal set can still be declared — as DELIBERATE, with a reason, which is a visible and arguable act — and an UNREACHABLE entity may never be called DELIBERATE, because there is no body to stage it on. **Shown red on the full laundering attempt**: the offender planted in a fixture AND added to both `verdicts` and `pairs` AND marked `unreachable`, and the gate still failed by name and exited 1.

A POPULATION HOLE, MEASURED AND CLOSED. The sweep found a declared set two ways — a helper call, or an object literal keyed `species:` — and a set written as a **positional row** `['species', [moves], ability, item]` was neither. Twelve such rows drive 200 games in `tests/test-protocol-trace.js` and **five of the twelve were illegal, including a Toxapex holding an item that does not exist in Gen 9**, while the gate reported the tree clean. Repo-wide that shape occurs in exactly one file (measured), so the rows were repaired first and the matcher turned on after: the population grew 627 → 639 declarations and the verdict count did not move at all.

WHAT IS STILL OPEN: 22 verdicts / 23 declarations, all in files this division does not own. `tests/staged_board.js` (16) and `tests/staged_status_counters.js` (7) are the differential's staged scenarios, filed to their owner; `tests/test-switch-carry.js` (2) needs a filler POLICY rather than a rename; and `tests/test-click-censoring.js` (1) is Farigiraf/Encore, where the repair is a re-cast rather than a swap — **derived: no legal body in this regulation learns Encore, Roar and Follow Me together**, and that fixture's one p1a body plays all three roles.

THE ORACLE IS SHARED, BECAUSE FACTS ARE GLOBAL. `champions_sim.legalRoster / abilityCarriers / moveCarriers / canLearn / unreachable` — one derived answer to "can anything in this format carry this", in the module that already owns `checkLegal`. `tests/probe_red_demo.js` still carries its own `abilityUnreachable` with its own dex walk; that file is ENGINE's and adopting the shared one is theirs to do. Two implementations of "is this legal here" will diverge silently, because both keep working.

00000000000. ROADMAP #258 — THE SILENT-CATCH RATCHET HAD A LAUNDERING BUTTON, WHICH IS WHY IT SAT RED FOR A WEEK — 2026-08-14

`tests/test-no-silent-failure.js` (reworked), `engine/provenance.js`, `engine/quarantine.js`, `engine/status.js`, `engine/open_work.js`, `data/silent-catch-baseline.json` (lowered, never raised).

THE NUMBERS, MEASURED RATHER THAN QUOTED. The register row said 78 new and 101 manufacturing; an earlier note said 81 and 101. Both were stale and neither was today's:

catch blocks	787
silent — say nothing at all	**296** (38%)
…of which MANUFACTURE a value	**97**
…of which only skip or continue	199
baselined floor	216 (was 220)
NEW since the baseline	**80** — 41 manufacturing, 39 skipping

THE TWO POPULATIONS ARE NOT ONE PROBLEM AND THE REPORT NOW SAYS SO. A `catch { continue; }` over a torn JSONL line is correct silence. A catch that RETURNS A PLAUSIBLE VALUE is this project's named failure mode in its purest form — a capability that could not prove it ran, reporting success. The new list is grouped by file, manufacturing first, instead of 25 flat names and "… and 57 more".

THE INSTRUMENT'S OWN DEFECT WAS THE REASON THE DEBT COMPOUNDED. `--update` rewrote the floor to the CURRENT silent set, so locking in a fix and laundering every new offender were THE SAME COMMAND. The row says as much — *"NOT RE-BASELINED, because* `--update` *would launder a week of it into the floor"* — and the consequence was a gate red for a week with three separate agent reports correctly observing they had not caused it. The sum of those correct observations is a check nobody acts on, which is "known failure" wearing the word *pre-existing*.

`--update` is now MONOTONE: `min(baseline, current)` per key. It removes what was fixed and cannot add what is new. The only door into the floor is `--accept <file> "reason"`, one file at a time, with the reason written into the artifact beside the keys, refused outright if either argument is missing. **Demonstrated:** two `--update` passes took the floor 220 → 216 and the gate stayed RED at 80 NEW, where the old behaviour would have written 303 and reported zero.

FIXED HERE — seven, all MANUFACTURE, all in files this division owns:

file	what the silence cost
`provenance.js` ×3	`declaredWriter`, `keysOf` and `mtime` each turned *corrupt / permission-denied / mid-write* into the same answer as *absent*, inside the tool whose whole job is saying whether an artifact can be believed. They call `logUnreadable` now and the count prints every run, **including at zero**
`quarantine.js` ×2	an unreadable artifact was reported to the gate as `NO ARTIFACT — run the instrument`, which is a wrong diagnosis handed to the clause that decides whether MEDICHAM is correct; and the open-defect clause failed loudly without ever saying WHY it could not read the register
`status.js` ×1	`metaPathFor` throwing silently disabled the sidecar clause, so a figure whose stamp is rotten would print as clean — the one thing that block exists to prevent
`open_work.js` ×1	an unreadable instrument dropped out of the measured half of the report that exists to print what is open

AND THE FIRST DRAFT OF THE `quarantine.js` FIX WAS ITSELF THE LESSON. It reported on `.js` bundles that are handed to a JSON reader on purpose, and printed six lines of noise on a clean run. Narrowed to `.json` paths: a ratchet that flags code for doing what it asked is how a ratchet gets ignored, and that is the fourth correction of exactly this shape in this repository.

A second detector blind spot is now STATED in the file rather than worked around: `blank()` erases template literals, so a reason travelling in `${e.message}` is invisible while `' (' + e.message + ')'` is seen. Fixing that would mean changing the brace scanner every other check in the file depends on; the call sites were changed instead, and the limit is written down. It costs a false POSITIVE, never a false negative.

STILL OPEN, AND NOT THIS DIVISION'S TO EDIT. The remaining 80 sit in `tests/roster.js` (5 manufacturing), `engine/mod_audit.js` (3), `engine/tag_dex.js`, `engine/game_differential.js`, `engine/million_run.js`, `engine/diff_swarm.js`, `engine/mega_census.js`, `engine/leaf_engine_contrast.js`, `engine/board.js`, `engine/medicham2-browser.js` and others owned by ENGINE and SEARCH. **The population moved by eight blocks during one hour of measuring it**, because both divisions were writing at the time — which is worth knowing before anyone quotes a single figure off this gate as though it were a constant. `node tests/test-no-silent-failure.js` prints the current per-file list; do not type one.

0000000000. ROADMAP #257 — A VERIFICATION RUN CAN NO LONGER REPUBLISH A SMALLER MEASUREMENT — 2026-08-14

`engine/publish_guard.js` (new), `tests/test-publish-guard.js` (new), `data/published-samples.json` (new), `tests/test-engine-diff.js` (four write sites).

FIRST, THE FIGURE, BECAUSE THAT IS THE HALF THAT MATTERS: IT IS TRUE. `data/engine-diff.json` carries `requested 6000, compared 6000, agreed 6000, disagreed 0`, generated 2026-08-14T06:36Z. The white paper, the deck, the technical docs and the site cite 6,000 and are backed by a real run of that size; `tests/test-docs-current.js` is green on all 21 clauses. Nothing had to be restated and nothing is withheld. **One caveat that is not this row's to close:** `engine/medicham2-browser.js` moved at 03:19 local, 43 minutes AFTER that run, so 0-of-6000 is a photograph of the 02:36 build. That is provenance's question and `status.js` prints it.

SECOND, THE DEFECT, WHICH IS THE `--write` AND NOT THE 150. The stated mitigation on the row — *"a verification run passes `--out` or omits `--write`"* — was measured to be a no-op: the file has neither flag and wrote `data/engine-diff.json` unconditionally on every run. A rule you can only follow by remembering it is worth what the ban list of four was worth. So it is a mechanism:

- **A publish below the published sample is REFUSED.** The run still completes and its output goes to `data/verification/<name>.n<sample>.json`, so a verification run loses nothing except the ability to republish. `process.exitCode = 3`, because a run that did not publish what its own output describes must not read as a pass — that is exactly how the original quick check looked successful.
- `--republish-smaller` **is the deliberate door**, and it stamps `sample_shrunk {from, to, at}` INTO the artifact, so the next reader sees the decision in the file rather than in a closed terminal.
- **A high-water mark per artifact** lives in `data/published-samples.json`, so the refusal survives a hand edit, a merge, or a writer that never called the guard at all.
- **The three conformance sections now amend the path `publish()` actually wrote.** They used to re-read `data/engine-diff.json` BY NAME — so a `--plant` run, which goes out of its way to write beside the artifact instead of over it, stamped three sections onto the published one anyway.

SHOWN RED ON THREE DELIBERATE BREAKS BEFORE BEING TRUSTED: the refusal disabled (clause A fails on 5 assertions), the artifact shrunk behind the guard (clause C names `data/engine-diff.json` and prints `ON DISK compared=150, RECORDED 6000`), and a bare write restored in the caller (clause B names the file and line). And on the real defect: `tests/test-engine-diff.js --n 150` now exits 3, leaves the 6,000-comparison artifact untouched, and writes `data/verification/engine-diff.n150.json`.

The guard is deliberately general — `publish({file, artifact, sampleKey, argv})` and `amend(path, sampleKey, fn)` — and only `tests/test-engine-diff.js` uses it today. **27 other top-level artifacts in `data/` carry a numeric sample field and are still unprotected** (counted, not listed — the shape is

`compared` / `requested` / `n` / `games` / `pairs` / `decisions`); `node engine/publish_guard.js record <artifact> <key>` is how one joins, and each belongs to the division that writes it.

oooooooooo. ROADMAP #242, FIRST HALF — THE RESIDUAL ORDER TABLE WAS A SUBSET PRESENTED AS A POPULATION: 42 ROWS AGAINST 90 — 2026-08-14

`engine/residual_order.js` (rewritten), `tests/test-residual-order-population.js` (new), `data/residual-order.json` (regenerated, 42 → 90 rows). `engine/medicham2-browser.js`, `tests/test-mechanics.js`, `engine/board.js`, `engine/rollout_leaf.js` were **not touched** — the placement half of #242 is ENGINE's and reads this table rather than re-deriving it.

THE NUMBER. The table published 42 walk participants. The authority walks 90. Measured against `Battle#fieldEvent` in `gen9championsvgc2026regmb`, filtered to the regulation:

walk participants	**90** (was 42)
that expire from inside the walk	**58**
…of which own no residual handler at all	**48**
…of which announce a protocol line	**28**
…silent but still order-bearing	**30**
that declare no order and sort at the `4294967296` sentinel	**29**

WHY IT WAS 42. `fieldEvent` sets `getKey = 'duration'` for the Residual event, and every collector admits an effect on `callback !== undefined || state[getKey]`. This file enumerated on the first clause only, so a `if (!hook) return;` guard threw away every effect that is in the walk purely because it has a live duration — Tailwind, Trick Room, the screens, Safeguard, Protect, the guards, the two-turn moves.

AND THE DIAGNOSIS IN THE ROADMAP ROW WAS WRONG IN THE WAY THAT MATTERS, INCLUDING MINE. The row says these effects have no order of their own. **They do, and it was in the format the whole time.** `tailwind.condition` declares `onSideResidualOrder: 26, onSideResidualSubOrder: 5` and simply declares no `onSideResidual`; `resolvePriority` reads `effect[callbackName + 'Order']` whether or not the matching callback exists. The order was never missing — the guard threw the effect away before anything read it. That is a better outcome than the row assumed: 28 of the 58 expiries have an exact published position, not a heuristic one.

PROVED BY RUNNING IT, NOT BY READING IT. One staged walk on the official simulator, four bodies, every family live at once:

```
|-weather|none                                  sandstorm expiry     order 1
|-heal| … [from] Grassy Terrain                  terrain heal         order 5
|-heal| … [from] item: Leftovers                 Leftovers            order 5
|-damage| … [from] psn                           poison               order 9
|-end|p1a: …|move: Taunt                         Taunt expiry         order 15
|-sideend|p1: A|move: Light Screen               screen expiry        order 26 sub 2
|-sideend|p1: A|move: Tailwind                   Tailwind expiry      order 26 sub 5
|-fieldend|move: Trick Room                      Trick Room expiry    order 27 sub 1
```

Twelve seeds, identical sequence. It is not a coin flip: Light Screen precedes Tailwind because their declared subOrders differ, not because a tie shuffled.

FOUR PUBLISHED VALUES WERE WRONG, AND ALL FOUR WERE THE SAME MISTAKE — A RULE COPIED INSTEAD OF CALLED. The old file owned a literal transcription of `resolvePriority`'s subOrder defaults:

- `psn`, `tox`, `brn` were published at subOrder **2**. Their effectType is `Status`, which is not in the authority's map at all — the real value is **0**.
- `wish` was published at subOrder **2**. It is a slot condition, so the authority gives it **3**. The old file *had* a clause for that and its own de-duplication defeated it: the row was already emitted by an earlier loop, so the slot-condition patch never reached it.
- the map's flat `Condition: 2` is not what the authority does at all. `resolvePriority` refines by **where the effect is attached** — `state.target instanceof Side` → 4, `isSlotCondition` → 3, `instanceof Field` → 5 — which no effectType can express.

None of the four changed a grouping medicham2 currently consumes, so nothing downstream moved. They were wrong in the published artifact regardless, and they are the argument for the fix: **this file no longer owns a copy of the sort rule. It calls `Battle.prototype.resolvePriority` and `Battle.prototype.comparePriority` on the real objects.**

THE GATE FOUND AN ERROR IN MY OWN REPLACEMENT WITHIN A MINUTE OF EXISTING, which is the strongest thing I can say for it. I shaped the `state` argument by hand — `{ target: Object.create(Field.prototype) }` — and published Fairy Lock at subOrder 5. Live, it resolves to 2, because `field.addPseudoWeather` **writes no** `target` **at all**: the field-condition refinement branch is unreachable in this Showdown build. The generator now stages a throwaway battle and keeps the real state object each attach method produces. A derivation that models the authority instead of asking it is wrong exactly where nobody thought to model.

THE GATE. `tests/test-residual-order-population.js`, auto-discovered by `tests/run-all.js`, 14 checks. It stages one battle, calls the authority's OWN `findSideEventHandlers` / `findFieldEventHandlers` / `findPokemonEventHandlers` with the same `getKey: 'duration'` that `fieldEvent` passes, and asserts every handler that comes back has a row whose `(order, subOrder)` equals what `resolvePriority` just produced live. **Shown RED first**: reverting the enumeration to handler-only makes it name 41 live participants with no row, plus 31 duration-carrying conditions from an independent lower bound.

AND THE FIXTURE HAD TO BE MADE INDEPENDENT, BECAUSE THE FIRST DRAFT STAGED FROM THE TABLE IT WAS AUDITING. With `ROWS.filter(site === 'side')` as the stage list, a table that had dropped every side condition also staged no side condition, and the membership clause reported one missing row instead of forty-one. The ids now come from the moves. This is the same defect as the row itself, one level up, and it is only visible because the break was run.

WHY `tests/test-residual-order-observed.js` **COULD NOT HAVE CAUGHT ANY OF THIS.** It stages a turn and checks the chips come out in the table's order — but it only ever looks at effects the table already contains, so a table missing a whole class of participants agrees with it perfectly. It is green today and was green throughout. **A check that watches a copy of the population cannot report that the population is short**, which is the sixth instance of that shape this week.

THREE MORE FINDINGS THAT ARE NOT ABOUT ORDER.

1. **Lucky Chant and Mist are** `isNonstandard: 'Past'`**.** The roadmap row's hand-derived list of 19 announcers names both. They are not in this regulation and cannot appear in a Champions turn. The list also omits the four weathers, `partiallytrapped`, Syrup Bomb, Encore, Perish Song and Uproar. The measured count is **28**, and Heal Block belongs on it — the *move* is `Past`, but the volatile is reachable in the regulation through **Psychic Noise**.
2. **Grassy Terrain is in the walk TWICE, eleven orders apart.** Its heal arrives through the per-active field collection at `onResidualOrder` **5** carrying the healed body's speed; its expiry arrives through the field collection at `onFieldResidualOrder` **27** with no speed at all. One row cannot say both, which is why a row is now an `(effect, attachment site)` pair rather than an effect. The 42-row table said only the first.

3. **A duplicate published key would have silently unplaced a medicham2 step**, and the first run of the rewrite produced one: both Grassy Terrain participants wanted `field:grassyterrain`, and medicham2 builds a `Map` from these rows, so the last would have won and its `terrain` step would have moved from order 5 to order 27 **while still reporting itself placed**. The generator now refuses to publish a duplicate and the gate asserts it. All 42 legacy keys still resolve; only `condition:grassyterrain` is gone, and it was a duplicate of `field:grassyterrain` that nothing read.

WHAT ENGINE NEEDS FROM THIS TABLE FOR THE SECOND HALF — a read, not a re-derivation:

- `rows[].site` is the load-bearing field. It decides the callback name, hence which `*Order` is read, hence where the effect lands. `ns:id` is the stable published key.
- `route` — `handler`, `duration`, or `handler+duration`. **A `handler+duration` row decrements first and, on the turn it reaches zero, runs `end` and SKIPS its own residual callback** (`continue` in `battle.ts`). Ten rows: the four weathers, Encore, Perish Song, Uproar, Syrup Bomb, `partiallytrapped`, `lockedmove`.
- `announces` + `announceLine` — 28 rows emit protocol at their sorted position. **30 more expire silently and are still order-bearing**, because the `duration--` spends a position in the same walk. An engine that skips the silent 30 will place the loud 28 correctly and still diverge.
- `order: null` means no declared order: `resolvePriority` stores `false` and `comparePriority` substitutes **4294967296**, so those 29 sort LAST — after Trick Room — by holder speed then subOrder.
- `speedFrom` — a side or field condition's expiry has **no speed** (its holder is a Side/Field, which has no `getStat`), so it sorts as 0 and lands behind every body-held effect at the same order.
- The blank-player-name defect on `|-sideend|p2: |move: Tailwind` is untouched here. The authority emits `|-sideend|p2: B|move: Tailwind`; that is a protocol-rendering bug in medicham2, not an ordering one.

NOT DONE, AND IT IS THE HALF THAT MOVES A GAME. medicham2 still places these expiries wrongly — 21 games of `data/game-differential.json` regroup to `ordering :: |-damage|<slot>|H/H|[from]sandstorm <> |-sideend|<side>|tailwind`, the largest single shape on the board. That is ENGINE's, it is live in that file, and the table it needs now exists.

0000000000. ROADMAP #266 — TEN OF THE FORTY-ONE ARE REPAIRED, AND ONE OF THEM WAS NOT A LEGALITY DEFECT AT ALL — 2026-08-14

`tests/test-volatile-duration.js`, `tests/test-perish-song.js`, `tests/test-protocol-trace.js`, `tests/test-game-diff.js`, `tests/test-speed-tie.js`, `tests/test-nature-differential.js`, `tests/probe_volatile_leaves.js`, `data/fixture-legality-baseline.json`. `engine/medicham2-browser.js`, `tests/test-mechanics.js`, `engine/board.js`, `engine/rollout_leaf.js`, `engine/game_differential.js`, `tests/staged_board.js` and `tests/staged_status_counters.js` were NOT touched.

THE NUMBER: 41 distinct verdicts → 32, 55 rejected sets → 45, and 1 stray literal → 0. `tests/test-fixture-legality.js` is ALL GREEN on the shrunk baseline, including its stale-allowance clause — every removal came off because the set became legal, and every one is named with its cost in the `repaired` block of `data/fixture-legality-baseline.json`. Population unchanged at 627 declarations and 227 distinct sets, so nothing came off because the scanner stopped looking.

verdict removed	what replaced it	what it cost
literal `"dampro"` (the stray)	`damprock`	nothing — the body is benched all scenario
Toxapex's item Black Sludge does not exist in Gen 9	Leftovers	nothing — benched body, and EXISTENCE is never deliberate
Clefable can't learn Perish Song	**Azumarill** sings, in BOTH files that share the singer	Clefable's Unaware leaves the field; nothing in either scenario boosts, so it could never have fired

verdict removed	what replaced it	what it cost
Weavile can't learn Encore	**Alakazam** is the user	**Pressure. Named below.**
Snorlax can't learn Pound	Round / Terrain Pulse (duration), Round (perish, never clicked)	both Special and 100-accurate, so they are paid into SpD rather than Def
Snorlax can't learn Agility	**Recycle** — this repo's own derived no-op	strictly quieter: Agility was moving the foe's Speed two stages a turn
Corviknight can't learn Pound	Body Press	nothing — the anchor only ever clicks Protect
Archaludon can't learn Body Press	Iron Defense	nothing — the slot Protects for six turns
Whimsicott can't learn Stealth Rock	the hazard and the screen **swapped bodies**	the screen guards the special side now, and the rocks lose Prankster's +1
Ninetales can't learn Dazzling Gleam	Psyshock	the click is single-target instead of spread

THE WORST ENTRY ON THE LIST PRODUCED NO VALIDATOR VERDICT AT ALL. `tests/test-protocol-trace.js:131` gave Politoed the item `'dampro'`. `dex.items.get('dampro')` returns a row that does not exist whose `.name` is the empty string — **and an empty item is a LEGAL item** — so `checkLegal` was asked about a Politoed holding nothing and answered honestly. The fixture built a Politoed holding nothing and every check went green. That is *"a capability was absent and everything reported success"*, inside a test, and it is exactly why `fixture_legality.js` reports stray literals in their own section rather than folding them into the illegal-set list: nobody is being accused of an illegal pairing, the string simply means nothing. The intended item is spelled correctly sixteen lines further down the same file.

A REPLACEMENT BODY IS DERIVED FROM THE FORMAT, NEVER RECALLED, AND THE DERIVATION IS THE ANSWER TO "WHY THAT ONE". Exactly **seven** legal bodies in this regulation learn Taunt, Encore and Disable together (Alakazam, Salazzle, Gardevoir, Gallade, Banette, Chimecho, Sableye) and exactly **six** learn Perish Song and Protect together (Gengar, Altaria, Absol, Politoed, Primarina, Azumarill). Alakazam is the fastest of the seven at 120 base Speed — the nearest thing to Weavile's 125 — and equally frail, so "the user must survive three turns of being hit" stays a real constraint instead of being quietly removed. Azumarill is the slowest and bulkiest of the six and the only one whose ability neither sets weather nor copies anything: Politoed's Drizzle would put rain on every board, Altaria's Cloud Nine would take it off, Absol brings Pressure into a PP-compared board.

AND THE COST OF THAT ONE IS A CAPABILITY THIS FILE NO LONGER STAGES, SAID OUT LOUD RATHER THAN ABSORBED: NONE OF THE SEVEN CAN HAVE PRESSURE. Weavile's Pressure made Snorlax pay double PP for every click into the user's slot, and PP is a compared board leaf. `tests/test-volatile-duration.js` still stages PP — both sides spend it every turn — but it no longer stages the DOUBLED rate. That rate is staged by `tests/staged_board.js`, whose Corviknight/Pressure bodies are actually targeted, so the coverage is not lost from the repo; it is lost from this file. Same shape as the Weather Ball case (#265): no legal body can carry the combination, so the honest move is to name what went with it.

THE FIXTURE AUDIT IN BOTH RED GATES IS NOW GREEN, AND `tests/test-perish-song.js` **IS GREEN OUTRIGHT.** Its two scenarios — four bodies dying at once, and the pivot that escapes the count — pass against the official engine with an Azumarill singing.

`tests/test-volatile-duration.js` **PASSES ITS AUDIT AND STILL PARTS FROM SHOWDOWN ON HP, AND THAT IS NOT THIS REPAIR. IT WAS MEASURED RATHER THAN ASSUMED.** The audit calls `process.exit(1)` before a single game runs, so nobody had seen this file's engine comparison since the learnset clause landed on 2026-08-12. The **pre-repair fixture was replayed body-for-body** — Weavile / Snorlax / Clefable, Pound and Shadow Punch, the same three-turn scripts — and it parts in all four scenarios by the same magnitude and in the same direction (`weavile sd=72 us=76` , `snorlax sd=199 us=204`). The legality repair moved the numbers; it did not create the disagreement.

WHAT THAT DISAGREEMENT IS, IS NOT MINE TO SAY TONIGHT, AND SAYING SO IS THE POINT. Isolated one click at a time it reads like a DAMAGE ROLL: medicham2 answers the same number every time while Showdown's varies with the scenario id, which is the signature of one engine on a pinned corner and the other on a die. `tests/test-speed-tie.js` shows the same shape on bodies this pass never touched — its `no-tie-CONTROL` case parts on `-damage` 75 vs 69 with Weavile and Volcarona. But `engine/medicham2-browser.js` was written at **02:26 tonight, while these runs were going**, and `engine/game_differential.js` carries in-flight middle-arm RNG work belonging to another agent. **A measurement is a photograph and nothing in frame may move.** So the absolute verdict is WITHHELD rather than annotated; what is published is the paired claim, which is valid because both arms were measured minutes apart against the same bytes: **pre-repair and post-repair part identically.**

WHAT IS DELIBERATELY NOT REPAIRED, AND WHY EACH ONE IS A SEPARATE PASS. 32 verdicts remain, and the reason is written per entry in the baseline rather than summarised here. Three blocks: `tests/staged_board.js` (15) and `tests/staged_status_counters.js` (7) are the staged-board library, which ROADMAP #266 routes to `@engine` and whose illegal clicks are load-bearing FILLER BOOSTS — a body clicking Iron Defense at +2 Def while a damage arm reads it is not a rename, it is a different board; `tests/test-protocol-trace.js` keeps 10, every one of which moves an event-coverage board (Clefable's Trick Room, Slowking's Perish Song and Haze, Incineroar's Knock Off and Politoed's Scald are all CLICKED to reach a protocol line, and the file's own verdict cannot be re-measured while ENGINE is live in the simulator); and `tests/test-switch-carry.js` keeps 2 because repairing only the two the static sweep can SEE would leave the identical defect in the Milotic and Vaporeon fillers it cannot see — those are declared through identifiers, so they are outside the population, and the fix is a filler policy rather than two edits.

ONE SITE CAME OFF WITHOUT A VERDICT COMING OFF, AND IT IS COUNTED AS A SITE AND NOT AS A REPAIR. `tests/test-nature-differential.js:152` gave Incineroar Knock Off, which it cannot learn here; it is now Darkest Lariat, its legal Dark physical click. *"Incineroar can't learn Knock Off"* is still produced by five other sites in two files, so the verdict stays on the baseline — a ratchet keyed on the sentence only shrinks when the last site producing it is gone.

000000000. ROADMAP #265 / #266 — THE FIXTURES WERE DECLARING TEAMS THE GAME WOULD REFUSE, AND THE RULER FOR THAT DID NOT EXIST — 2026-08-13

`engine/fixture_legality.js` (new), `tests/test-fixture-legality.js` (new), `data/fixture-legality-baseline.json` (new), `engine/feature_fixture.js` (six repairs + a body digest), `tests/test-feature-semantics.js` (three new clauses). `engine/medicham2-browser.js`, `tests/test-mechanics.js`, `engine/game_differential.js` and its artifacts were not touched.

THE HEADLINE IS A POPULATION, NOT A VERDICT. 334 `.js` files, 627 set declarations across 26 files, 227 distinct sets, 55 REJECTED by `TeamValidator`, producing 41 distinct verdicts. Six of those were in MEASURE's own `engine/feature_fixture.js` and are repaired here; the other 41-minus- those are baselined, named, and owed as #266.

THE VERDICT IS THE AUTHORITY'S AND NOTHING HERE RE-IMPLEMENTS IT. Will, mid-task: *"i thought we were using showdowns team validator as the ultimate legality test."* The brief this pass started from said to check the species with `isNonstandard`, then the ability against the species' list, then the item, then each move with `checkCanLearn` — **four hand-written rules standing in for a table Showdown maintains**, which is FACTS ARE GLOBAL broken inside the file written to enforce a rule. A piecemeal check only finds the rules somebody thought to write. `validateTeam` carries the 66-point SP budget, the 32-per-stat cap, the level, the item clause and every ban the Champions mod adds next. So `fixture_legality.js` calls `champions_sim.checkLegal` — one shared validator instance, five padding slots that are themselves proved clean — and prints the validator's own sentences rather than a summary of them. **It is also less code.**

THE SCANNER IS DERIVED AND IT WAS WRONG FOUR TIMES BEFORE IT WAS RIGHT, WHICH IS WHY EACH RULE IS WRITTEN DOWN WHERE IT LIVES. A hand-listed set of fixture files is the shape this repository has been wrong about most often, so the sweep walks the tree and finds sets by shape:

the rule	what it was reporting before it existed
a literal is an entity only if it normalises to that entity's OWN id	`dex.species.get('p2')` answers **Porygon2**; `p2` in this tree is a SIDE
moves come from ARRAY literals only	`hit('vaporeon','icebeam')` clicks the move AT that body — **three false accusations**
helpers are taken at TOP LEVEL only	`test-mechanics.js` redeclares `run` / `hit` / `at` in dozens of probe callbacks; a file-global name set paired an ability stamped on the ATTACKER with the DEFENDER's name
a set is declared by an object literal **or by MUTATION**	requiring a `species` key hid **64 real sets** in `test-protocol-trace.js`

507 CONSTRUCTION SITES CARRY NO LITERAL SET AND ARE OUTSIDE THE POPULATION, WHICH IS CORRECT AND IS PRINTED RATHER THAN IMPLIED. A fixture that builds its bodies from the format — `tests/roster.js` walks the regulation, `engine/all_mechanics_fire.js` reads the tag dex — cannot type a name that does not exist. A fixture that builds through `buildMon(key)` and assigns the click LATER, which is most of `tests/test-mechanics.js`, has no set to validate at the point the body is made; the sweep says so instead of guessing, and that is the honest limit of a static sweep.

THE GATE IS A RATCHET, SHAPED LIKE `data/fixture-learnset-baseline.json` AND A SUPERSET OF IT. Keyed on the **validator's own sentence**, not on file:line, so the same illegal declaration moved to a new scenario is the same defect. Six clauses, and two of them exist because a green gate is worth nothing on its own: a **population floor** (a scanner that finds nothing passes everything below it — it must find ≥150 distinct sets, against 227 today), and a **self-check that the ratchet still discriminates**, run on every invocation rather than demonstrated once in prose. The baseline **may only shrink**: a baselined verdict that stops being produced FAILS, because a repair that leaves its allowance behind is a hiding place for the next illegal set. Every entry carries a `kind` (`DELIBERATE` or `PRE-EXISTING`) and a written reason, and an entry with neither fails.

SHOWN RED THREE WAYS BEFORE BEING TRUSTED, and the first is the real historical instance: re-planting Rocky Helmet on Venusaur reports `[EXISTENCE] Venusaur's item Rocky Helmet does not exist in Gen 9.` naming `engine/feature_fixture.js:87`; two invented baseline entries fire the stale-allowance clause by name; one carrying a bad `kind` fires the reason clause. The tree was restored byte-for-byte from a copy taken before the plant.

THE SIX REPAIRS ARE ALL IN MEASURE'S OWN FIXTURE AND COVERAGE WAS MEASURED ACROSS EVERY ONE. Rocky Helmet → Miracle Seed (4 sites), Electric Seed → Mental Herb, Throat Spray → White Herb, Grimmsnarl's Thunder Wave → Taunt, Hippowdon's Weather Ball → Rock Slide, and a new board.

	before	after
boards	10	**12**
candidates / pairs	352 / 1,407	384 / 1,534
columns that never fire	0	**0**
columns that LOST a firing	—	`statusBites` **5 → 4, and nothing else**

TWO OF THOSE REPAIRS WERE NOT RENAMES AND THE COST IS RECORDED RATHER THAN ROUNDED AWAY. *(a)* **No body in this regulation can carry Weather Ball in sand.** Derived over the format, the move has **80 legal carriers and not one is a Rock or Ground body or sets sand**,

so `sand-is-up` had been claiming the ROCK type-flip path through a move Hippowdon cannot learn. Repairing it IN PLACE was tried first and measured: swapping in Heliolisk costs that board its Earthquake and its Yawn and thinned `allyHit` **4 → 3,** `abilityBlock` **3 → 2,** `deadStatus` **2 → 1,** `statusBites` **5 → 2** — the first two being exactly the pair this file was built to protect — while **every column stayed non-zero, so the coverage check would have waved it through**. The flip moved to its own board, `sand-weather-ball`, which is what this file's own record of two bad edits says to do. *(b)* **Grimmsnarl has no legal primary-status move at all**, only secondary-chance ones, so Taunt cannot carry `deadStatus`; the click moved to Torkoal's Will-O-Wisp into an already-paralysed Incineroar. That restores `deadStatus` and leaves `statusBites` one short, because the only body that could repay the last one is the Clefable whose fourth move is the single 4x hit anywhere in the fixture. Trading `eff4`'s only firing for one of five is a worse board, so it is not made. **4/384 against 5/352, declared.**

AND THE FINDING THAT OUTLIVES THE FIX IS THE GUARD'S OWN, FOR THE EIGHTH TIME. `feature_fixture.verify()` decided *"is this the same fixture?"* from `ROUND` and the list of scenario **LABELS**. A label is not a board. Edit a species, an item, an ability, a nature, a move or a pre-set state inside a scenario that keeps its name and the identity check passes — after which every moved column is announced as *"these features changed MEANING since the weights were fitted"*, which is **FALSE**, and which accuses `board.js` of a change it did not make. That is the refit/restamp distinction this whole file turns on, collapsed. It now carries a **body digest** covering species, item, ability, nature, moves, bench and pre-set state, reported as *"the fixture's BOARDS changed while its scenario labels stayed the same"* and explicitly **NOT** as evidence about `board.js`. A stamp written before the digest existed is treated as an OLDER STAMP and not as staleness, matching the convention the `table` block already set.

WHAT THIS DOES NOT SAY. It says nothing about whether a fixture's board is a good test — only that the team could be brought. It cannot see a set assembled at runtime. And the 41 baselined verdicts are **not repaired**: they are named, costed per file, and routed as **#266**. Nothing was re-baselined into silence, and the gate is what enforces that.

REPORTED, NOT FIXED, NOT SILENCED. `tests/test-mechanics.js` was run WITHOUT being edited: **567 live / 0 missing, exit 0** — but ENGINE was working in the same window and took the census 565 → 567 under #264, so that number is ENGINE's and is quoted here only as evidence that nothing in this pass moved it. `tests/test-protocol-trace.js:131` gives Politoed the item `'dampro'`, which names nothing in this format: the fixture builds a Politoed holding **nothing** while its source says otherwise, and reports success. That is *a capability was absent and everything reported success*, and it is filed rather than repaired because that file has twelve other verdicts and they move as one batch.

00000000. ROADMAP #254 — THE HAZARD SIDE. THE FIT DOES NOT MOVE, THE LIVE BOT DOES, AND THE GUARD COULD NOT SEE EITHER — 2026-08-13

`engine/board.js` (`sideFor` + three call sites), six sweep sites, `engine/feature_fixture.js` (one new board), `tests/test-hazard-side.js` (new), `tests/test-feature-semantics.js` (one new clause). `engine/medicham2-browser.js` was not touched.

THE HEADLINE IS THE REFIT VERDICT, AND IT IS NO. `board.js` recorded every side condition on the MOVER's side. Seven of the eleven legal side-condition moves in this regulation are `target: allySide` and were right by accident; the four `foeSide` ones — Stealth Rock, Spikes, Sticky Web, Toxic Spikes — landed on the side they can never be on. That is a feature the FIT reads, so the question this division has to answer is whether the weights now describe a different quantity.

THE WHOLE FIT CORPUS, REPLAYED THROUGH `FP.decisionsFor` UNDER BOTH BOARDS: 9,226 games, 237,052 decisions, 1,731,851 candidate vectors, 0 errored, identical decision key set, and 0 decisions, 0 games and 0 feature vectors move. The pre-#254 board was compiled in

memory and injected into `require.cache` under `board.js`'s own resolved path; the tree was never reverted, so both arms ran against the same everything-else.

THAT ZERO HAS TWO POSSIBLE MEANINGS AND THEY ARE OPPOSITE, SO BOTH WERE MEASURED. Thin exposure would make the zero uninformative. Exposure is small but not nil: **24 hazard clicks in the fit corpus** (Stealth Rock 19, Toxic Spikes 2, Sticky Web 2, Spikes 1) against **9,911** `allySide` **clicks**, and **258 of 237,052 decisions (0.109%) carry a hazard as a legal candidate**. The zero is therefore not "nothing to move".

IT IS AN ISOMORPHISM, AND IT WAS CONSTRUCTED RATHER THAN ARGUED. `noteMove` wrote to the mover's side and both readers read the mover's side, so the old board is the new one with the two sides RELABELLED — every read that was derived offline lands on the same set. Same probe, all eleven moves, both boards:

how the condition arrived	pre-#254	fixed		
OFFLINE — `noteMove(..., worked=true)`, which is every corpus replay	`deadSide` p1=1, p2=0	p1=1, p2=0 — **identical**		
LIVE — `\	-sidestart\	`, which is engine/magnemite.js:647`	**p1=0, p2=1**	p1=1, p2=0

SO THE DEFECT WAS ONLY EVER IN PLAY, AND IT IS THE WORSE HALF. The live path takes the side straight off the protocol and calls `noteMove(..., worked=false)`, so the offline derivation never runs and nothing relabels the read. On the old board the LAYER read `deadSide=0` after laying Stealth Rock — it would re-lay it every turn believing it was not up — and the VICTIM read `deadSide=1` and refused to lay its own. Both exactly backwards. **MAG was fitted in a world where** `deadSide` **was effectively right and played in one where it was inverted**, which is *fitting environment and playing environment must match* broken in a new place — the same shape as the sheet-visible fit and the broken-mega champion — and it is closed here in the one direction that costs no refit.

A REFIT IS NOT OWED BY THIS ROW. The weights describe the same quantity on the corpus they were fitted on, to the vector. A refit remains OWED for the reasons `status.js` already prints (`medicham2-browser.js`, `board.js`, `engine-data.js` and `abra-tags.js` all moved after the fit) and nothing here changes that; no fit was run and no weight file was restamped.

ONE CONSEQUENCE TO NAME RATHER THAN DISCOVER LATER. Adding a fixture board takes the scenario count 10 → 11, so `feature_fixture.js --check` now answers *"the fixture itself changed … Old hashes cannot be compared"* for every stamped file, which SUPERSEDES the per-feature message it printed before. That branch is correct and it is also less informative, and it is a restamp-or-refit decision that belongs to whoever next moves the weights — which is Will, who is reworking them.

THE FINDING THAT OUTLIVES THE FIX IS THE GUARD'S, AND IT IS THE REASON THIS ROW CAME HERE. `tests/test-feature-semantics.js` is the guard against a feature changing MEANING under an unchanged NAME. Run under both boards with the fixture as it stood, **0 of 76 hashed columns moved** — `engine/feature_fixture.js` contained **zero hazard clicks** and the only side conditions any of its ten boards pre-set were `reflect` and `tailwind`, both `allySide`, which is exactly the half `sideFor` leaves alone. The guard would have certified this silently. R7 for the seventh time, and the new part generalises: the earlier misses were a SPECIES the boards did not stand on and a FIELD STATE they never entered; this one is a MOVE CLASS nobody clicks.

A NEW BOARD, `hazards-already-up`**, AND IT IS DELIBERATELY ONE-SIDED.** `stealthrock` and `spikes` up on p2, `stickyweb` and `toxicspikes` up on p1, so the flip is exercised in BOTH directions on one board — setting a hazard on both sides makes every reader answer 1 before and after and moves nothing.

`deadSide` now moves `b984c210828d` → `d1be4f95d589` and no other column does. Every set is learnset-checked against `Dex.forFormat`, and the board is added rather than an existing one edited, per that file's own record of two attempts that were inert and one that regressed coverage.

AND ONLY `deadSide` IS OBSERVABLE, WHICH CORRECTS THE ROADMAP ROW RATHER THAN REPEATING IT. The row said a write-only fix would credit `setupTurns` every turn. Measured: none of the four hazards carries a `condition.duration` (Reflect 5, Tailwind 4, the hazards none), so `dur()` returns 0 and `setupTurns` is 0 for them under every variant. The `alreadyUp` site is routed through the helper for the one-fact-one-function rule and binds today only on the seven `allySide` moves, where `sideFor` is the identity.

NINE CALL SITES, NOT THREE. The write, the two reads, and six verbatim copies of the same mover's-side derivation that ENGINE's sweep found outside `board.js` — `fit_policy.js:793` (the fit itself), `joint_rows.js`, `branch_recall.js`, `corpus_shift.js`, `feature_coverage.js`, `redirect_audit.js`. Fixing only `board.js` would have left the fit deciding the fact for itself. `tests/test-hazard-side.js` holds all six to reading `B.sideFor`, and reverting one of them fails that clause **by file name**.

SHOWN RED THREE WAYS BEFORE BEING BELIEVED, none of them by editing the tree. `tests/test-hazard-side.js` was **5 passed / 3 failed at HEAD** and is 11/0 after; under the injected pre-#254 board the semantics guard's new clause names all six wrong candidates in both directions; and reverting one sweep site fails by name. `tests/test-mechanics.js` is **565 live / 0 missing**, unmoved — the whole-game differential exercises `medicham2`, not the feature board, so it was not expected to move and did not.

REPORTED, NOT FIXED, NOT SILENCED. `engine/selftest.js` is RED (exit 1) on *"every raw reader of the ladder store declares why — 9 file(s)"*: it is a MISSING DECLARATION, not a corrupt read — the gate demands each raw reader carry a `RAW-STORE-OK`-style reason and nine do not. Pre-existing and untouched here. `tests/test-fragility.js` is RED on two Storm Drain assertions and is **identical under both boards**, so it is not this change. And `engine/feature_fixture.js` gives Venusaur a **Rocky Helmet**, which this format banned on 2026-08-04 — left alone deliberately, because changing a fixture body moves every stored hash.

ooooooo. GROWING A `need` LIST SILENTLY RETIRED OLD MEASUREMENTS. IT NOW COSTS A NAMED ARTIFACT — 2026-08-12

`tests/test-artifact-rerunnable.js` (new) + the `provides` recorder in `engine/engine_release.js`. Nothing else was touched.

Will: *"should i be concerned we suddenly cant run old things" → "you need to fix it and make it so it doesnt happen again i dont know the solution"*.

A release freezes the ENGINE and not the READER. Every symbol a caller adds to its `need` list retroactively strands every release cut before that symbol existed: the snapshot still verifies, still holds its bytes, and simply stops being openable. ROADMAP #222 split the RNG streams and `rngStreams` appeared in 30 snapshots; `spreadL50` stranded two more. **Both changes were right. The bug is that paying their cost was invisible.**

THE HEADLINE, AND IT CONTRADICTS THE FIRST READING OF THE SAME FACT. 41 artifacts name a release across 21 releases: 29 RE-RUNNABLE, 1 STRANDED, 11 UNKNOWN-PRODUCER, 0 retired. The first scan reported that essentially nothing could be re-run. That scan unioned every `need` list in `engine/` into one 24-symbol set and held every artifact to it — so `game_differential.js`'s artifacts were failed for lacking `hitChance`, which only `million_run.js` has ever asked for. **An artifact is judged against the caller that PRODUCED it**, read from its own `by` field, and the requirement table comes from `engine_release.js`'s `callerNeeds()` rather than a second scanner. The one genuine stranding is `data/nature-arms.json`, whose release predates `rngStreams` and `spreadL50` that `game_differential.js` now needs.

AND "168 OF 200 RELEASES ARE UNOPENABLE" IS ONE CALLER'S NUMBER WEARING THE STORE'S NAME. Asked per caller over all 201 releases on disk, against `engine/medicham2-browser.js`:

caller	releases that can serve it	predate an export	pruned / absent / unloadable
`engine/game_differential.js`	**28 of 201**	168	5
`engine/million_run.js`	97 of 201	99	5
`engine/replay_differential.js`	140 of 201	56	5
`engine/speed_vs_pokeenv.js`	**196 of 201**	0	5
the UNION, which is what `census()` asks	28 of 201	168	5

The union is the strictest caller and nothing else, so `data/release-census.json`'s `runnable` is a lower bound on every caller but one. It is not wrong — its own comment says it asks the hardest question any live caller asks — but it must not be read as "the store is 86% dead". **Filed: `census()` should report the per-caller column beside the union.**

UNKNOWN-PRODUCER IS A BAND, NOT A PASS. Eleven artifacts either record no `by` or name a producer with no `REL.require(file,{need})` site — a `.py` script, a driver that shells out, or a caller that reaches a snapshot through `REL.read` / `REL.path`. Their requirement cannot be READ, so they are not accused; they are still held to the one requirement that needs no guess, that the release opens at all. `data/all-mechanics-fire.json` — the 93 move divergences — is in that band, and the cheapest way to move it out is for its generator to stamp a `by`.

THE PARSER WAS DELETED RATHER THAN PATCHED, AND ITS FAILURE IS THE SAME ONE THREE TIMES. The check originally hand-rolled a `module.exports` reader and reported that a release cut MINUTES earlier lacked `fails` and `hitChance`. `medicham2-browser.js` interleaves block comments with the keys inside that literal, so a comma split glues each comment onto the key after it: **11 of 78 exports lost, and 4 more invented out of prose — one of them the word `deliberately`.** That is `provenance.js`'s `writesNear` hole and `callerNeeds`'s stripped-comment rule, for the third time. `engine_release.js` already answers the question by LOADING the frozen module (`surface()`), at 18ms, so the verdicts use the loader and this file contains no parser.

THE RECORDED `provides` FIELD IS CORRECTED AND IS NOW AUDITED RATHER THAN TRUSTED. It is kept because it is the only thing that survives a prune — once the bodies are gone `surface()` can answer nothing. It is stamped `provides_by` so a legacy list is distinguishable from a current one, and the check compares every recorded list against the loader on every run. The one manifest written by the old recorder is reported, not rewritten: a release record is immutable and is not edited to make a check green.

SHOWN RED BY REPRODUCING THE REAL EVENT. Adding `rngStreams` to `engine/replay_differential.js`'s `need` list — exactly what #222 did to the differential — turned its five artifacts STRANDED and named all five as NEW against the ratchet, while every artifact of every other caller stayed put and `replay-differential-freezes.json` (no `by`) correctly stayed UNKNOWN-PRODUCER rather than being accused. The caller was restored byte-identically (sha256 `a21d2158…`) and the check returned green.

The ratchet is `data/artifact-rerunnable-baseline.json`; it may only fall, and a new stranding fails by name.

Filed, not fixed: `callerNeeds()` reads `REL.require` sites only, so the file requirements of `tests/roster.js` and `tests/mutation_harness.js` — which reach a snapshot through `REL.read` and `REL.path` — are invisible to it and their ten artifacts are judged on openability alone. That under-reports and never over-reports.

`callerNeeds()` also hard-codes an `engine/` prefix on the caller name, so a `tests/` scan is mislabelled and the label is repaired at the call site. Both belong in `engine_release.js`, in a pass that is not running beside two other divisions.

000000. THE END-STATE COUNT BECAME A SEVERITY LADDER, AND THE WORST BAND IS NINE PERCENT — 2026-08-12

`engine/end_state_severity.js` (new) + `engine/game_differential.js` end-state reporting + `tests/test-end-state-severity.js` (new). Nothing in `medicham2-browser.js` or `tests/test-mechanics.js` was touched; the run reads the frozen release `6155acc0fb26`, the same one the standing 31.9% / 36.6% bar and the published end-state table use.

WHY. `DIFFERENT-END-STATE` was a COUNT. A game in which a healthy body is killed by a move it cannot be hit by — after which a replacement comes in and every later line is a different game — weighed **exactly the same as a three-HP rounding residue**. On the same day, five defects were verified against Showdown's own source, staged, shown red, fixed, and the whole-game count moved by **+1 and +3**; all five were narration. Will then read twenty-five battles by hand and found three wrong OUTCOMES. **No instrument here could have separated those two kinds of finding, because a count has no order on it.**

THE HEADLINE — 2,300 games per arm, frozen team pool, release `6155acc0fb26`, turn cap 12, 0 threw, planted proof green, `data/game-differential-endstate.json`. Bands are over the `DIFFERENT-END-STATE` games, which is the parted ones PLUS the games whose narration never parted and that still ended apart (31 top, 18 bottom — a class the protocol instrument is structurally blind to).

band	what it means	top-tie-first	bottom-tie-first
1	**DIFFERENT WINNER**	**0**	**0**
2	**DIFFERENT BODIES ALIVE** — dead in one engine, standing in the other	**25 (9.0%)**	**27 (9.1%)**
3	HP differing by more than a typical hit	47 (16.9%)	59 (19.9%)
4	different species / typing / ability on a LIVE body	86 (30.9%)	95 (32.0%)
5	a status, item, hazard, screen, weather, PP or volatile	51 (18.3%)	47 (15.8%)
6	HP under one hit, or a stat stage — plausibly rounding	69 (24.8%)	69 (23.2%)
	banded	**278**	**297**

SO ROUGHLY ONE DIVERGED GAME IN ELEVEN ENDS WITH A DIFFERENT SET OF POKEMON ALIVE, and about three in ten with a body whose identity we have wrong while it is still standing. Neither was visible before; both were inside one number.

THE BAND 3 THRESHOLD IS MEASURED AND THE RULER IS THE AUTHORITY'S, NOT OURS. Every DIRECT `-damage` event Showdown narrated, as a fraction of the struck body's own maximum HP; the median is the threshold. **2,092 hits, median 25.9% of a health bar, IQR 14.1–47.3** (top); **1,808 hits, 28.8%, IQR 17.5–53.1** (bottom). Cut per arm and never pooled — the arms sit at opposite corners of the damage roll.

TWO THINGS THE FIRST VERSION OF THAT RULER GOT WRONG, BOTH FOUND BY READING ITS OWN OUTPUT. *(a)* Residual chip — sandstorm, burn, Leech Seed, hazards, Life Orb — was counted as a hit and put the median at **12.7% with quartiles 6.2 and 34.2**, the middle of a bimodal mixture, describing neither half. Showdown's own line says which is which (`[from] …`), so the filter is read off the protocol rather than guessed from a size cutoff, which would be circular. **4,838 and 4,947 residual events are excluded and counted.** *(b)* `0 fnt` carries **no denominator**, so a parse requiring `\d+/\d+` discarded **every killing blow** — 13 direct hits narrated in one measured game and 8 reaching the ruler, the five missing being the knockouts. A ruler built only from hits too small to kill anybody. It now resolves against the body's carried maximum; hits went 1,612 → 2,092 and 1,321 → 1,808.

THE SHAPE PRIOR HOLDS AS A RATE AND IS WRONG AS A HEADLINE. The prior was that ORDERING-shaped divergences land harmless and RULE-shaped ones land severe. Share of each shape's games reaching band 2:

shape	top	bottom
RULE	7/47 = **14.9%**	11/68 = **16.2%**
EMISSION	16/137 = 11.7%	15/126 = 11.9%
ORDERING	1/14 = 7.1%	1/21 = 4.8%
UNPARSED	0/48	0/57
protocol never parted	0/31	0/18

RULE is the most dangerous shape per game and ORDERING the least, so the prior is right. **But EMISSION supplies 16 of the 25 and 15 of the 27 band-2 games** — the majority of the worst outcomes — because it is by far the largest bucket. "Fix the rule-shaped ones first" is right per game and wrong per evening. **And ORDERING is not zero: one game in each arm has an ordering-shaped first divergence and a different set of bodies alive at the end.** That is the more interesting half and it is not dismissed here.

THE WORKLIST, RANKED BY BAND FIRST AND BY CORPUS USAGE SECOND (teams containing the body in `data/meta-usage.json`, read LIVE — it is not in the frozen release, and it is **generated 2026-08-04**, so the ranking rests on an eight-day-old corpus model):

- `sinistcha` **is the largest single body in band 2 — 6 games (top) and 7 (bottom), 2,668 corpus teams**, and the same shape every time: `0/146` **for us against** `146/146` **for the authority.** A body at FULL health in Showdown and dead here. Filed to `@engine`; not diagnosed here.
- `pair-redirect-priority` **supplies 14 of 25 and 13 of 27 band-2 games**, `pair-protect-bust` 8 and 7. That is a fact about the steered sample as much as about the engine and both readings are open.

BAND 1 IS ZERO AND THAT IS NOT A CLEAN BILL — THE HARNESS CANNOT CURRENTLY MEASURE IT. A battle that does not resolve cannot have a different winner, and **2,275 of 2,300 games stop at the 12-turn cap**. Raised to 19 (`data/game-differential-endstate-turn19.json`, 983 games, proof green) it is still **938 of 983 at the cap, band 1 = 0 in both arms**.

AND ABOVE TWENTY TURNS THE COMPARISON COLLAPSES, FOR A REASON THAT WAS THE HARNESS. `battleOver` is `S.turn >= (S.maxTurns || 20) || …` and this driver had never set `maxTurns`. Measured at `--turns 40`, 983 games: **943 ENDED-APART, 937 of them "ONLY medicham2 ended the battle"** — 96% of a run produced by one hard-coded default, reading exactly like a catastrophic engine disagreement. (`data/game-differential-endstate-turn40.json` is that run, kept as the receipt; it was taken under the pre-fix driver and its band-3 figures are not to be quoted.) The driver now sets `S.maxTurns = max(MAXTURNS + 1, 20)`, so **every 12-turn run is byte-identical to before** and only the already-broken configurations move.

A 30-TURN RUN THEN REFUSED TO PUBLISH, CORRECTLY. With the horizon lifted, one of the state comparator's own planted defects — *PP spent on a slot NOTHING has touched* — can no longer be staged, because in a 30-turn game every slot has touched every move. The run printed `THE STATE COMPARATOR FAILED ITS OWN PROOF` and declared its state numbers worthless. That is the instrument working. **So the honest position is that DIFFERENT-WINNER has no denominator in this harness yet**, and closing it needs a plant that survives a long game, not a bigger sample.

SHOWN RED FIRST, AND SHOWN NOT FIRING. `tests/test-end-state-severity.js`: a planted death reaches the top rung and is localised; **a planted three-HP residue must NOT** — a ladder that answers "severe" to everything passes the first half alone; a one-sided forme change must land on the identity rung and not read as a death, because the party is keyed by species and a rename is indistinguishable from a corpse in the diff list; all six rungs reachable; a board carrying a death, a boost and a burn bands as the death; identity

on a body dead in BOTH engines does not reach the identity rung; `severity()` REFUSES to run without a measured threshold; the `|split|` duplicate is stepped over; and the whole path through the real driver, a planted faint surviving to the last board against the same pair and seed run clean as a control.

THE ONE PLACE THE BRIEF WAS THE WRONG SHAPE, AND IT IS RECORDED RATHER THAN QUIETLY OBEYED. The brief ranked *a different set alive* above *a different winner*. First-match-wins over that order makes the winner rung **structurally empty** — a different winner is always also a different set alive — and a band that can never fire is not a band. They are swapped, and the containment is printed on every run. A fifth rung was also added: folding a burn, or a layer of Spikes, into "plausibly rounding" would repeat in miniature the exact weighting bug the ladder exists to fix.

AND A RED THAT WAS OPEN ON ARRIVAL IS CLOSED, WITH THE ATTRIBUTION CORRECTED. `docs/ENGINE.md` recorded `tests/test-end-state.js` PART 3 failing and attributed it to staleness in the pass that wrote it, having restored both `medicham2-browser.js` and `game_differential.js` to their HEAD bytes and reproduced it. That control was sound and it held the wrong variable still. **Same frozen release, one flag apart: live team pool → 2 FAILURES;** `--team-store data/team-pool-frozen` **→ ALL GREEN.** `diff_swarm.js` reads the pool LIVE from a file OPS appends to, so `pairsFor` returns a different first pair as the store grows, and PART 3's item plant — legitimately undoable by a Knock Off or a berry — eventually lands in a battle that undoes it. The test now pins the pool by default and a caller can still override it.

WHAT THIS DOES NOT SAY. A band is exactly as strong as what `board_state.js` compares; its `NOT_COMPARED` list is published with the artifact. `SAME-END-STATE` remains a claim about where the two engines ARRIVED and not about the turns in between. ENDED-APART (4 and 6) is a THIRD answer and is counted beside the ladder, never inside it.

00000. TURN 1 IS THE PRIMARY READING NOW, AND THE CONTAMINATION THEORY IS HALF RIGHT — 2026-08-10

Will: *"lets only do turn 1's so we know nothing from the previous turn is messing up. then we can begin to look into later turns"*. `--turn1-only` refused to write an artifact, correctly, because one of the three planted defects is **structurally impossible on turn 1** — every body starts at full HP, so an `hp` event written at the top of turn 1 puts 100 over 100 and there is nothing left to detect.

THE REFUSAL WAS NOT WEAKENED. THE GATE IS STATED OVER CLASSES OF DEFECT INSTEAD OF ARM NAMES. An arm declares its class; an arm the mode cannot carry declares itself INAPPLICABLE with the reason; and the run refuses unless every class has an arm that RAN and was CAUGHT. Dropping an arm because the mode cannot carry it is the same bypass as never running it, one level up from the `--selftest` flag this file already refuses to hide behind.

A FOURTH PLANT COVERS THE PRE-TURN-BOARD CLASS ON TURN 1: A SPECIES SWAP. A lead body is replaced by one whose FASTEST legal Champions spread — taken over every weather — is slower than the SLOWEST legal spread of the body the record shows it outspeeding, so the recorded resolution order becomes something no legal spread can produce. It is planted at **every suitable site up to 12 and every one must be caught**; planting once and stopping at the first success is how a detector that works on one board in twenty passes a gate.

plant	class	all turns	turn 1 only
a damage figure cut to a quarter	outcome	runs	runs (restricted to turn 1)
a freeze the game cannot produce	unreachable	runs	runs (restricted to turn 1)
a wrong pre-turn HP under a recorded KO	pre-turn board	runs	**INAPPLICABLE — declared, not skipped**
a species swap on the pre-turn board	pre-turn board	runs	runs

TWO CANDIDATE PLANTS WERE MEASURED AND REJECTED FIRST, AND THE FIRST ONE IS A FINDING. A species swap into a **type immunity** is invisible to this instrument: `dmgRange` returns an EMPTY roll array for an immune matchup and `damageVerdict` turns that into `unresolved — dmgRange returned no rolls` rather than a divergence. So "the record shows damage on a body our engine says cannot be touched at all" is REFUSED, not accused. **FILED, not fixed** — it would move the headline on the same day the mode changed. A swap into a bulkier body was rejected for a different reason: the attainable interval is ~60 points of max HP wide, so it fires or does not fire depending on the sample, and a red proof that is a coin flip is not a proof.

SHOWN RED TWICE, DELIBERATELY, BEFORE BEING BELIEVED. Sabotaging the order comparator's disjointness test produced `8 of 8 PLACED PLANTS WENT UNNOTICED`; marking the species arm inapplicable produced `A WHOLE CLASS OF DEFECT HAS NO ARM THAT RAN AND WAS CAUGHT: preturn`. Both refused to write.

THE HEADLINE: 1,249 of 19,715 turn-1 units diverge = 6.34%, release `30b21c5a335a`, 20,000 games read and 285 skipped (1.43%), 0 exceptions. bot-v-human 6.71%, human-v-human 5.62%.

THE MOVE FROM THE PUBLISHED 5.36% IS ENTIRELY SAMPLE, AND THAT IS MEASURED RATHER THAN ASSUMED. On the same first 3,000 games the rebuilt instrument reproduces **158/2,947 = 5.36% exactly**, at the old release `70794711fe6d` AND at the new one — so the instrument moved nothing and the engine moved nothing on this arm. Games 3,001–20,000 run **6.51%**, and the gap survives inside both population strata rather than being the rising bot share alone.

TURN 1 AGAINST LATER TURNS, SAME 20,000 GAMES, SAME RELEASE, ONE RUN OF EACH MODE. `--turn1-only` reproduces the full run's turn-1 arm to the unit (1,249/19,715 both ways), so the mode changes the denominator and not the measurement.

	turn 1	**turns >= 2**
turns compared	19,715	106,558
diverged	1,249 (**6.34%**)	8,185 (**7.68%**)

Per 1,000 turns, by family — and the split between "everything that fed this was known" and "something was never revealed" is the whole point:

family	**turn 1**	**later**	**later / turn 1**
damage, all inputs known	17.40	25.08	1.44
damage, attacker's item never revealed	34.90	37.53	1.08
turn order	5.02	8.39	1.67
status, source ability known or no source	**1.07**	**2.97**	**2.79**
status, source ability never revealed	8.17	7.56	0.93
weather	0.00	0.17	—

THE STATUS BUCKET: THE RAW COUNT BARELY MOVES AND THE CLEAN ARM COLLAPSES. 182 status divergences on turn 1 (9.23 per 1,000) against 1,123 later (10.54 per 1,000) — a 12% reduction, which does NOT support "the 188 status divergences are largely false" as a blanket claim. **88% of the turn-1 status rows (161 of 182) are rows where the body that clicked into the status had an ability the record never revealed**, and that split did not exist until this pass: the damage family has carried `[the attacker's item was never revealed]` since it was built and the status family was quoting two different claims as one number.

Split, the answer is sharp. **SLEEP IS THE CARRIED-OVER-STATE CONTAMINATION AND IT IS ISOLATED**: 130 cause-known sleep divergences on later turns (1.22 per 1,000) against **ONE** on turn 1 (0.05) — a Yawn or a sleep counter from an earlier turn, exactly as suspected, and it cannot survive a turn-1 arm.

FREEZE IS THE OPPOSITE and is the one clean status candidate for ENGINE: 10 of the 14 turn-1 freezes have a known cause and every witness is a Blizzard or an Ice Punch.

THE LARGEST SINGLE TURN-1 STATUS WITNESS IS NOT AN ENGINE DEFECT: `sneasler Fake Out -> psn` **, 57 times in 20,000 turn-1s.** That is POISON TOUCH, and `data/engine-data.js` carries Sneasler's modal observed ability, which is UNBURDEN — in a closed-sheet game, 98.3% of this store, the body is built with it and cannot poison on contact. **A general lesson rather than a Sneasler one:** the modal-ability prior is doing the same damage to the status family that the unrevealed item does to the damage family.

AND THE SPREAD-MOVE INGEST DEFECT IS WORSE ON TURN 1, NOT BETTER. The unresolved rate is **39.56% of 41,762 damage units** against 35.30% over all turns, because **10,304 of the 16,519 unresolved rows (62%) are the spread-move conflation** — turn 1 is where both foes are most likely to be alive, so it is exactly where a spread move hits two bodies and the store collapses them into one row. Turn-1-only removes the invisible-carried-state class and removes none of this one. `sinistcha Matcha Gotcha -> brn` is the top turn-1 burn witness, which is that defect showing up in the status family too.

THE JOINT 66-POINT BUDGET IS NOW ENFORCED AND IT BUYS NOTHING — 0 of 28,982 corners clamped. Every corner this instrument evaluates pushes at most TWO stats of one body (defensive stat + HP pool = 2 x 32 = 64 <= 66), so the constraint never binds and no interval moves by a point. The clamp is live rather than decorative and starts counting if the cap ever changes. **The over-width that is real is a different one:** every EVENT is evaluated at its own corner, so one body can be the max-offence spread while it attacks and the max-bulk spread while it defends, in the same turn, on the same 66 points. Tying a body to one spread across a game is a joint solve, and it errs toward ACCUSING the engine — not done here, and not on the day the mode changed.

Filed, not fixed: the immune-matchup hole above; `readGames` holds the whole sample in memory, which is what caps the sample rather than time (20,000 games is 35s in turn-1 mode); and the status family still has no split for a self-inflicted orb or a residual, which land in `[source ability KNOWN or no preceding move]` beside the genuine candidates.

0000. ROADMAP #68 — THE ENGINE NOW GETS MEASURED AGAINST GAMES THAT ACTUALLY HAPPENED — 2026-08-10

`engine/replay_differential.js` → `data/replay-differential.json` and `data/replay-differential-freezes.json`. New file; nothing existing was edited.

The authority is `data/games.ladder.jsonl` **, not Showdown.** `tests/test-engine-diff.js` is one damage calculation per row with no turn loop, and at its default n=150 it reported zero disagreements for weeks while n=20,000 finds 19. `engine/game_differential.js` has a turn loop and plays SYNTHETIC games against the library. Neither asks whether our engine describes the game people actually play.

THE HEADLINE, 3,000 games, release `70794711fe6d` **.** 2,947 replayed, **53 skipped (1.77%)**, all for the same reason — the row has no turns. **18,421 turns compared, 1,008 diverged = 5.47%.** On turn 1, where no invisible state can exist yet, **5.36% of 2,947**. **0 exceptions.** Bot games are included and split rather than filtered: bot-v-human 5.95%, human-v-human 5.06%.

THE BOARD IS REBUILT FROM THE RECORD AT THE START OF EVERY TURN, so each turn is an independent unit and one 6-turn game is ~6 tests rather than one fragile chain. The price is enumerated in `cannot_see` and the turn-1 arm is the one with no unmeasured confounder in it.

ROLL RECOVERY DOES NOT WORK ON THIS CORPUS, AND THE REASON IS MEASURED. Will's proposal — roll all 16 and identify which one was played — is the right shape, and it turns the damage figure from an input into a test. It cannot run here: **Champions team sheets do not declare SP** (884 of

52,089 stored games carry a sheet; every one has `evs: null`), and the record states damage as an integer percent of an unknown maximum. The **median attainable damage interval is 60.1 points of max HP**, so one roll step is **3.76 points** — the legal-spread envelope is several times wider than the entire 16-roll band. `matched` fires **2 times in 37,177**. The test is therefore inverted into the one the record supports: all 16 rolls, at the observed crit state, at both corners of the legal SP envelope, and is the observed value inside. **719 no-match, 13,359 ambiguous, 13,533 unresolved (36.4%), 9,564 KO-clamped.** The 36% is printed at the top because a rate that size decides whether the headline means anything.

THREE SOURCES OF RANDOMNESS ARE RECOVERED FROM THE RECORD RATHER THAN SIMULATED — a miss is `miss:1`, a crit is `crit:1` and its ABSENCE is knowledge because Showdown always announces one, and a secondary is an `x` or `b` event. **The resolution order is the event order (ROADMAP #43), and nothing in this repository had ever read it**: 3,081 turns where our speed calculation is forced and agrees, **159 where no legal Champions spread can produce the order the record shows**, 9,312 refused as inside the envelope, 5,016 refused for a Prankster/Gale Wings/Triage carrier.

EVERY DIVERGING TURN IS A READABLE FROZEN BOARD, not a counter — Will diagnoses by reading boards. `data/replay-differential-freezes.json` carries the board before, the reconstructed clicks with a per-click confidence, the board after from the log against what our engine could reach, and the raw events. **Four instrument bugs were found by reading its own output**, each of which had been sitting at or near the top of the mechanic table: a mega not applied before the turn it happens in, a Weather Ball priced in clear skies because the sun was set three events earlier in the same turn, a KO clamp read off a reconstructed HP instead of the record's own figure (56 of 158 divergences), and weather with no expiry doubling Swift Swim and Chlorophyll speeds for the rest of the game.

THE RED PROOF RUNS ON EVERY PUBLISHED RUN AND THE ARTIFACT REFUSES TO EXIST WITHOUT IT. Three planted defects in a real stored game — a damage figure cut to a quarter, an impossible freeze, a corrupted board HP that puts a recorded KO out of reach. A `--selftest` flag somebody has to remember is the same failure one level up. It has been **shown red twice for real**, both times correctly.

FILED TO ENGINE, from the mechanic table. These are candidates and not verdicts; where the attacker's item was never revealed the row says so IN ITS KEY and must not be quoted with the others. Largest with everything known: `x0.5-ish` 72, `x0.25-ish` 56, `x2-ish` 32, `x4-plus` 17, and the status family — `slp` 59, `brn` 57, `psn` 31, `par` 18, `frz` 15 — where the record applied a status no pin of our engine can produce.

FILED TO OPS — three ingest defects this instrument had to work around. Each is a store change, not an analysis change. (1) `durable-ingest.js` records **no** `cant` **event at all**, so flinched, fully paralysed, asleep, frozen and recharging are one indistinguishable absence — Showdown emits `|cant|p2a: X|flinch` and it is dropped. (2) A **spread move's damage is conflated**: every `-damage` is attributed to the last `m` event with `dmg = max(dmg, delta)` and `tgt = tgt || <slot>`, so one row carries the maximum of two deltas against the first target's species and the last target's `tgthp`. That refuses **8,360 of 37,177** damage units — the single largest unresolved bucket. (3) **Everything before** `|turn|1` **is dropped**, including a lead's entry weather, so a Drought lead's sun is invisible.

NO GROUND TRUTH FOR THE CHOICES EXISTS. The 22 stored games with `willhoop` as a player carry the same extracted schema as every other row — no record of what was clicked. So the reconstruction cannot be validated against known answers and rests on the per-click confidence in the freeze dump. The live bot logging its own choice string would make this instrument exact.

DEFERRED, DESIGNED, AND BEHIND `--rates` : the aggregate secondary-rate test. Pinning an outcome to what was observed makes a wrong PROBABILITY agree with itself — the same trap as pinning damage. The counter-test compares each move's observed secondary rate across the corpus against the chance our tag declares. It is not part of any figure above and needs a far larger corpus before an interval on a 10% secondary means anything.

000. THE ABILITIES CLAUSE WAS GREEN OVER 27% COVERAGE AND FIFTEEN UNMEASURED ROWS — ROADMAP #120, #121, #122 — 2026-08-10

Will: *"check the abilities clause for the same hole"*. `data/roster.abilities.json` read **84 FIRED-AND-BOARDS-MATCH, 217 COULD-NOT-STAGE, 15 CONTROL-NOT-QUIET** and `engine/quarantine.js` printed `clean: 84 fired and matched` and **PASSED**. Three separate faults sat inside that sentence.

THE CLAUSE NOW FAILS, AND IT FAILS ON SEVEN ATTRIBUTED DEFECTS.

THE ENGINE MOVED UNDER THIS AND THE TWO CAUSES ARE SEPARATED RATHER THAN BLENDED. A release was cut by another division at 04:21 while this was being written, so the published run reads a different simulator from the artifact it replaces. The new instrument was therefore re-run **pinned to the OLD release** `a4c7f898ad0e` , and the two engine snapshots differ on **exactly one row**: Magic Bounce, DID-NOT-FIRE on the old and FIRED-AND-BOARDS-MATCH on the new — ENGINE fixed it in between, and the old instrument could not have said so because that row was CONTROL-NOT-QUIET. Every other movement below is the instrument.

	before (old instrument, `a4c7f898ad0e`)	**instrument only (`a4c7f898ad0e`)**	**published (`bfefdb697454`)**
FIRED-AND-BOARDS-DIFFER	0	**2**	**2**
DID-NOT-FIRE	0	**6**	**5**
FIRED-AND-BOARDS-MATCH	84	76	77
CONTROL-NOT-QUIET	15	**18**	**18**
COULD-NOT-STAGE	217	214	214
clause	**PASS**	**FAIL**	**FAIL**

1. A CONTROL THAT IS ITSELF A LIVE ABILITY IS NOW VARIED, NOT CAPTIONED (#121). The 15 rows were controlled by a second real mechanic because their carrier species has no quiet alternative — and this format has **only 8 quiet abilities**, none of which shares a species with any of the 15, so "pick a quieter one" cannot reach them and never will. A quiet ability cannot be lent from another species either: `buildPair` clamps an ability to its species' own list and falls back to slot 0, so an illegal pairing silently becomes the subject arm again.

What CAN reach them is a **SECOND CONTROL**: where the carrier has a third ability, the identical scenario is played a third time against it and the two deltas are compared leaf for leaf **in both engines**. A leaf that survives both does not depend on which ability was removed, so it is not the control's. **98 rows were varied this way.** It is the noise-floor discipline (§9 of LESSONS) applied to a control arm: vary the knob that is supposed not to matter, and believe the effect only if it survives.

IT IMMEDIATELY CAUGHT EIGHT VACUOUS GREENS. Anger Point, Justified, Rivalry, Keen Eye, Shell Armor, Stalwart, Sticky Hold and Slush Rush were **FIRED-AND-BOARDS-MATCH** and the agreement was about the CONTROL's work — Intimidate's −1 Attack on three of them, Weak Armor's drop, Gooey's drop, Stamina's boost, Supersweet Syrup's evasion drop. Anger Point needs a crit and the pin lands none; Rivalry needs a gender and `buildPair` sets every body to `N` by construction. Those rows could never have fired.

AND IT CAUGHT THE INSTRUMENT'S OWN FIRST ANSWER, WHICH IS WORTH RECORDING. The first version called a row with nothing surviving both controls *inert*. Two rows come out with identical arithmetic — 28 leaves against the first control, 0 against the second — and mean opposite things. ANGER POINT really is inert. SLUSH RUSH is live: against Swift Swim (inert in snow) Beartic's kill lands before the foe's stat drop; against SNOW CLOAK the drop **misses**, because evasion in snow turns a 100-accuracy move into a guaranteed miss under this pin — the same board by a different mechanism. **Two arms cannot separate "the subject did nothing" from "the subject and this control did the same thing"**, and Beartic has exactly three abilities so there is no third to ask with. Those rows are UNATTRIBUTABLE and say so; they are not inert and they are certainly not passes.

SIX ARE DECLARED UNTESTABLE, with the pool printed rather than asserted — Aroma Veil, Flower Veil, Fluffy, Imposter, Rain Dish, Solar Power. Every legal carrier of each has exactly one alternative ability and that alternative is live. The declaration is derived every run from the format, so a regulation change retires it without anybody remembering to.

A TIE-BREAK THAT WOULD HAVE BOUGHT TWO OF THOSE SIX WAS TRIED AND TAKEN BACK OUT. Ranking "carrier has a third ability" above bulk moves Rain Dish to Pelipper and Solar Power to Heliolisk — and it moved Water Absorb to Politoed, whose highest-ranked alternative is **DRIZZLE**, and Sand Rush and Sand Force to Excadrill, where the staging comes out inert in Showdown and the rows lose their coverage entirely. **Changing the fixture to suit the control is the wrong trade.** The second control is a measurement taken on whatever carrier the rule chose.

2. THE CLAUSE STATES ITS DENOMINATOR (#120). 84 of 316 is 26.6%; excluding the 115 rows whose ability has **no legal carrier in this format** — out of scope by regulation, not untested — it is 84 of 201 = **42%**. Neither number was on the line. The artifact now carries a `scope` block written at the refusal (`cannot(why, 'no-legal-carrier')`, tagged where the refusal happens, never matched out of prose afterwards) and `quarantine.js` reads it rather than re-deriving it. The clause reads `84 TESTED of 201 IN SCOPE, of 316 total` and **names the 18 unattributable rows in the text**.

An artifact predating the block says **DENOMINATOR NOT CARRIED** and names the re-run, rather than defaulting to zero — a missing count must not read as "none", which is the shape of the bug it replaces. `data/roster.items.json` and `data/roster.moves.json` are in that state now and are their owners' to re-run.

THE ONE JUDGEMENT CALL, stated so it can be overruled: an unattributable row is REPORTED and does not hold the gate shut. Six of the eighteen are untestable in this format by construction, and a clause that can never open is not a gate. They are named in the clause text on every run instead of sitting invisible inside a green. One `&&` in `rosterStage` changes that if Will wants it.

3. THE ABILITY STAGE CAN ASK FOR A SWITCH NOW, AND IT CLOSES ZERO ROWS (#122). No ability row had ever attempted one, so trapping abilities had never been asked the only question that distinguishes them from doing nothing. `ability/traps-and-somebody-tries-to-leave` reuses the moves stage's `switchVerdict` rather than building a second probe. **Asked of the format: Arena Trap and Magnet Pull have no legal carrier at all, and Shadow Tag's only carrier is Gengar-Mega — a forme whose ability the forme change WRITES, so it cannot be swapped and its only control is suppression, which `gastroWorks()` measures as dead in this simulator (6 leaves in Showdown, 0 here).** So the honest count of trapping rows closed is **zero**, and the rows say that instead of saying nothing.

The capability proves itself rather than being asserted, because a rule whose every member is COULD-NOT-STAGE never reaches `--reds` and a staging path that has never run is assumed broken: `abilitySwitchWorks()` plays the identical fixture with a NON-trapping carrier and requires both engines to complete the ask. It is green — the untrapped Kangaskhan leaves for Milotic in Showdown and in medicham2 — and it is a printed selftest line, so it can go red.

THE SEVEN NEW REDS ARE ENGINE DEFECTS AND ARE NOT FIXED HERE — filed to `@engine`, each with the second-control receipt that makes it attributable:

ability	verdict	what parted
Electromorphosis	DID-NOT-FIRE	Charge never applies — Showdown's Bellibolt hits 22 harder after being struck; ours does not move at all
Ice Body	DID-NOT-FIRE	no hail/snow residual heal — Showdown 71 → 81 → 91 → 101, ours flat at 71
Curious Medicine	DID-NOT-FIRE	the ally's stat stages are not reset on entry (−1 Atk / −1 SpA stay)
Reckless	DID-NOT-FIRE	the recoil-move base-power boost is absent (Brave Bird 11 HP short, and the recoil with it)
Sweet Veil	DID-NOT-FIRE	the ally is not protected from sleep — Spore lands in ours and is refused in Showdown
Mirror Armor	FIRED-AND-BOARDS-DIFFER	the drop is not reflected: Showdown puts Scrafty at −1/−2 Atk and −1 SpA, ours leaves 0
Supersweet Syrup	FIRED-AND-BOARDS-DIFFER	evasion drops **twice** here and once in Showdown — an entry effect firing on both foes and again, or not once-per-battle

One process note. `data/roster.abilities.json` was rewritten (its bytes are at `.prev.json`); `data/roster.json` — the labelled convenience copy of whichever stage ran last — was also rewritten and previously mirrored the moves stage. Nothing is lost: `data/roster.moves.json` is that stage's artifact and `quarantine.js` reads the per-stage file first. `tests/roster.js --keep-shared` now exists so a division running beside another can leave that file alone.

00. THE QUARANTINE IS A MECHANISM NOW, NOT A PARAGRAPH — 2026-08-08

`engine/quarantine.js`. Will's standing call: *"all engines that take medicham's output should be regarded as out of date and we should stop referencing them until medicham is up to date and we can rerun them."* CLAUDE.md states the rule; this file executes it, and `engine/status.js` no longer prints the figures it covers.

THE BUG IT CLOSES IS THIS DIVISION'S OWN. `status.js` has printed `PRE-CHANGE — measured against a different build of: …` beside the leaf-calibration number for days, and the number went on being quoted anyway — by the sessions that printed the caption. That is the identical failure to a red gate reported for two days as "one of the two known failures". **A caption is not a quarantine. The figure is WITHHELD.**

The gate, today: 4 of 4 clauses FAIL.

clause	state
game differential	**1 of 150** comparisons disagree with Showdown — `chesnaught woodhammer -> mimikyu`, showdown 0-0, medicham 120-130
deliberate roster / items	**NO ARTIFACT** — a missing stage is a FAILING clause
deliberate roster / abilities	**2** FIRED-AND-BOARDS-DIFFER, **4** DID-NOT-FIRE
deliberate roster / moves	**NO ARTIFACT**

A MISSING STAGE FAILS. `tests/roster.js --write` writes `data/roster.json` whatever stage it ran, so the file holds only the newest one; reading it three times and calling that three stages would be "a capability was absent and everything reported success" inside the guard written to stop it. A stage counts only when an artifact's own `stage` field names it — `data/roster.<stage>.json`, `data/roster.all.json`, or `data/roster.json` when it matches.

Membership is derived from one root, and it is not a list of filenames. The PLAY LAYER is the transitive closure of *requires the simulator*, seeded with `engine/medicham2-browser.js` alone: 63 modules, board.js among them through `damageEngine()`. An artifact is quarantined if its generator is in that closure, if it reads a dump one of our own runs wrote, or if it reads a quarantined artifact. **34 of 114** artifacts are held. The transitive arm is what holds `mag.js`, `scoreboard.js`, `ladder.json` and `weight-multiplicity.json` behind `policy-weights.json`.

The strict direction is the dangerous one, and nothing that measures MEDICHAM is withheld. The census, the interaction matrix, the differential, the roster, the release ladder and everything OPS counts off the ingested store all print. Most fall out for free — they are written by `tests/`, or they drive the authority through a subprocess. Two do not and are **declared with a reason**, the `RAW-STORE-OK` convention: `game_differential.js` (MEDICHAM is its subject; it is the gate's own first clause) and `derive_protocol_events.js` (it loads the simulator only to read the event list it claims to emit, and quarantining it would have withheld the differential downstream of it). Both declarations are **checked** — an exemption naming a module no longer in the play layer fails the gate.

Shown RED before being trusted. The leaf-calibration withhold was removed by hand; `--check` exited 1 naming `data/winrate-backtest.json` and the leaked verdict sentence. And the negative was verified too: driving the real `withholder` with an open gate releases **34 of 34** and withholds none, so this can lift.

~~**What the graph still cannot see, stated rather than papered over.** `provenance.js` finds a writer only in `engine/` and `build/`, so ~50 artifacts written by `tests/` or through an unfollowed path variable have no row and are neither cleared nor withheld.~~ **CLOSED 2026-08-09 — see §00a.** The graph is 115 → 160 artifacts and the unknown set is 61 → 16. `data/rollout-r1-explore1.json` classifies on its own now and is HELD by its own derived reason, so the both-files workaround at `status.js:665` is retired-able. The instruments stayed clear and the consumers went behind the gate: **34 of 114 → 40 of 160** withheld.

Where a withheld number is still cited — measured, not remembered, and ratcheted in `data/quarantine-stamp.json` (may shrink, never grow): `docs/ENGINE.md`, `docs/MEASURE.md`, `docs/SEARCH.md` and `web/status-data.js`. Not edited from here; `web/` is WEB's and a gate its owner cannot satisfy becomes a known failure.

00a. ROADMAP #105 — FIFTY ARTIFACTS HAD NO ROW IN THE GRAPH, AND ONE OF THEM WAS THE ARM MILTANK RUNS — 2026-08-09

`engine/provenance.js` discovered an artifact's writer by looking for its literal name beside a write call in `engine/` or `build/`. Anything written by `tests/`, or through a path the scan did not follow, had **no row at all** — not `ok`, not UNSAFE, absent. Nothing could compare it to its source, which is the condition CLAUDE.md's derived-artifact rule exists one level above.

	before	after
artifacts with a writer	115	**160**
artifacts with NO writer	61	**16**
withheld by `quarantine.js`	34 of 114	**40 of 160**
UNSAFE	13	**20**

Four arms, ranked by how directly each proves a write, and `via` is now part of the graph so a consumer can tell a write call from a sentence: `write line` (83), `path variable` (52), `path template` (12), `near a write` (1), `DECLARED BY THE ARTIFACT` (10). The scan reads `tests/` as well; a computed name such as `'roster.' + STAGE + '.json'`, `` `exploitability-${TAG}.json` `` or `+ '.meta.json'` becomes a regex with one wildcard per runtime value; and where no source can say — fit_policy.js takes its path from the OUT_WEIGHTS environment variable — the artifact's own by` is accepted, labelled as the weakest evidence, and only if the named script exists.

`data/rollout-r1-explore1.json` **CLASSIFIES ON ITS OWN and is HELD**, by the derived reason *"engine/rollout_r1_artifact.js reads rollout-r1-rows.jsonl — a dump of games MEDICHAM played"*. The both-files workaround at `status.js:665` — asking the quarantine about `rollout-r1.json` so the shipped arm could not slip through on a technicality — is a workaround for a hole that is now closed. **Not edited here; reported.**

Nothing was defaulted, and the strict direction held. The instruments print (`mechanics-census` , `engine-diff` , `interaction-matrix` , `roster.*` , `tag-walk` , `wire-ladder-census.pin` — all `ok`); the consumers went behind the gate (`exploitability-mag` , `exploitability-machamp` , `scoreboard` , `policy-weights-joint-presheet` , `ab-batch-effect` , `rollout-r1-explore1`). The **16 that remain UNKNOWN are printed every run, at zero as well as at sixteen**, because an empty list has to mean "every artifact has a writer" and never "we stopped looking". Seven are `policy-weights-*.json` variants written through `OUT_WEIGHTS` and declaring no `by` ; `engine-release.json` is written through `writeJsonAtomic(S.pointer, …)` , a helper no detector follows; `regulations.json` and `quality-filter.json` are CONFIG and correctly have no generator.

Four false attributions were found and closed on the way, three of them by this pass's own change. Each is written into the file beside the rule it produced.

- **A read `open()` on a line that later contains `'w'` .** `M=json.load(open('…')); … w=np.array(M['w'])` matched `open\s*\(.*['"][wa]['"]` — so a test that only loads JOLTEON's weights outranked `engine/ditto.py` , which computes them, and flipped the artifact to not-store-derived. The mode string must now be an argument of the `open` call.
- **A write into a scratch tree is not a write into `data/` .** `tests/test-miltank-release.js` writes `path.join(EMPTY, 'engine-release.json')` as a fixture and took ownership of the release pointer. A write now has to be rooted in `data/` — including through a helper whose own definition roots it, because requiring the literal word cost six correct rows on the first attempt.
- `writesNear` **never stripped comments, and this file's own new comment was its victim** — the example of what NOT to count credited `provenance.js` with generating the release pointer. The loose arm reads code now; the tight arms still read the source, because `build/build_browser_data.js` names its two targets in a trailing comment *deliberately* and stripping them took both artifacts back to "no generator".
- `data/regulations.json` **was credited to** `engine/analyze.js` **, which only reads it.** It has no generator: it is config, and `engine/conformance.js` already says so. One row lost, and the row was wrong.

A template match is CORROBORATED, not trusted. `exploitability-${TAG}.json` also reaches `data/exploitability-holdout.json` , which `exploit.js` did not write and cannot — it has no holdout mode. A template attribution must agree on top-level key shape with something the same generator writes by name, or it is revoked and the file stays UNKNOWN with that reason printed.

THE RATCHET GREW, AND THAT IS THE ONE JUDGEMENT CALL IN THIS PASS. `mtime_only` went **91 → 128**. None of the 38 regressed — 37 of them had no row for anything to ratchet, and a file that was never visible cannot have lost a stamp. The ratchet was measuring two different things: *"a generator shipped without recording what it read"* and *"this checker's coverage changed"*. They are opposite events and only the first is a fault. So the stamp now records `graph_files` , the diff splits REGRESSION from DISCOVERY, a regression still fails, and a discovery is printed in full and appended to `discoveries` in `data/provenance-stamp.json` with its date, reason and file list — the growth is permanent and auditable rather than laundered. The first run could not split (the old stamp carries no `graph_files`) and **says so instead of guessing**: an artifact-mtime heuristic was tried and accused five instruments other divisions had regenerated that afternoon. **Shown RED before being trusted** — dropping one file from the baseline while it stays in `graph_files` reproduces `RATCHET BROKEN` .

UNSAFE 13 → 20, and the eight movements are accounted for. Seven are newly visible and were always unsafe: `nature-arms.json` (older than `tags.json`, and `game_differential.js` moved under it) and six run sidecars pinned to superseded releases. The eighth was `quarantine-stamp.json`, which stamps `engine/provenance.js` by content — editing this file invalidated it by construction, and it returned to `ok` when `node engine/quarantine.js --check` re-ran. No artifact left the UNSAFE set.

Filed, not fixed:

- `engine/conformance.js` **'s S13 still hand-rolls the question.** It decides "no generator writes it" with `allSrc.includes(file)` over source text, so `data/roster.moves.prev.json` still trips it. The answer is derived now — `node engine/provenance.js --graph --json` carries `by` and `via` — and S13 should ask for it, the way `status.js` shells out rather than reimplementing staleness.
- `engine/rollout_r1_artifact.js` **writes the literal** `data/rollout-r1.json` **whatever dump it read.** The explore=1 arm exists only because somebody renamed the output afterwards, which is why no pattern over that source can reach it and why the artifact's own `by` is the only witness. Derive `OUT` from `ROWS`, as `run_stamp.js` already derives its sidecar path.
- `readsNear` **has the same comment hole** `writesNear` **just had.** It decides `from`, and `from` decides staleness verdicts, so moving it changes what this tool SAYS rather than what it can SEE. Deliberately left; it belongs in a pass that re-derives the drift table.
- **ROADMAP #108 is easier to close but is not closed.** `status.js` printing a figure `provenance` calls UNSAFE is unchanged in kind — but every artifact `status.js` reads now HAS a row, so the two gates can finally be asked the same question about the same file. `status.js` is not edited here.

0. THE FORK IS DECIDED, AND THE ANSWER IS NO — 2026-08-07 (3.69.0)

A MORE CORRECT ENGINE DID NOT MAKE BETTER PREDICTIONS, AND NEITHER DEPTH METRIC PREDICTS LEAF ERROR. `engine/leaf_engine_contrast.js` → `data/leaf-engine-contrast.json`. Read every figure from the artifact; the rows are in `data/leaf-engine-contrast-rows.jsonl` so the curves can be re-cut without replaying 74 minutes of rollouts.

What was measured. MILTANK's live in-game leaf — `rolloutWinProb` at n=200, explore=1.0, foePolicy uniform, horizon 60, read from `miltank.js DEFAULTS` rather than retyped — on **8,883 positions**, the whole clean scorable corpus at the photograph, scored through **two frozen releases**:

	release	medicham2-browser.js	cut
BASELINE	`cf6a68fa412c`	`795be0c58cd7`	2026-08-07T02:34:46Z
TOP	`dc3c43336539`	`d10db6714fc9`	2026-08-07T17:27:09Z

The run **refuses to start** unless the two manifests differ in `engine/medicham2-browser.js` and in nothing else, and they do — same weights, same board, same damage table, same tag file, same Showdown commit `20ad99f`. Identical positions, identical per-position seeds. So the contrast is the simulator and nothing else.

1. THE TWO ENGINES ARE INDISTINGUISHABLE AT THE LEAF, AND THE NULL IS POWERED.

	TOP − BASELINE	95% CI	noise floor	detectable at 80% power
Brier	**0.0000**	[−0.0007, +0.0007]	0.000642	0.001013
log-loss	+0.0013	[−0.0007, +0.0033]	—	—

The confidence interval is **narrower than the smallest effect this n can detect**, so this is a tight null and not an underpowered one. McNemar on the 7,994 positions where both engines made a decisive call: **37 discordant for TOP, 36 for BASELINE**, z = 0.117, p = 0.91. The two leaves correlate r = 0.9881, mean |Δp| = 0.0254, max 0.175 — they are not the same function, they just score the same.

2. BOTH LEAVES ARE STILL WORSE THAN A COIN, ON A SAMPLE 6.4× THE PRIOR ONE. Brier vs coin, paired: **+0.0325 [0.0281, 0.0372]** (TOP) and **+0.0325 [0.0281, 0.0371]** (BASELINE). Positive is worse. The 2026-08-04 held-out reading of +0.0502 [0.0371, 0.0628] is reproduced in direction and sign at n = 1,778 held-out: **+0.0382 [0.0279, 0.0485]**.

3. DISCRIMINATION IS REAL ONLY IN-SAMPLE, AND IT SITS ON ITS OWN NOISE FLOOR. On the full corpus the leaf names the winner on **52.48% of 8,320 decisive calls [51.40, 53.55], p < 1e-4** — and the split-half accuracy spread for that same arm is **2.49 points** against an effect of **2.48 points**. By this division's own rule (LESSONS §9) that is not an effect. On the **held-out newest fifth it is 50.48%, p = 0.70** — no ranking at all, for both engines.

4. CALIBRATION IS THE FAILURE, AND IT IS A COMPRESSION. ECE **0.1514**, MCE 0.405. The reliability curve is monotone and almost flat: 88 points of predicted range map onto **13 points of observed range**.

leaf says	0.06	0.16	0.25	0.35	0.45	0.55	0.65	0.75	0.84	0.94
it wins	.466	.443	.494	.494	.498	.513	.528	.564	.544	.594

When it says 94% it wins 59%. A maximiser lives in that column.

5. AND THE JOINT ANSWER — NEITHER LINES NOR TURNS PREDICTS LEAF ERROR. Per-position leaf Brier against that position's first-divergence depth against the official simulator, on the same bodies the leaf rolls out. Spearman, bootstrapped over positions; **negative rho is the hypothesis**.

predictor	BASELINE engine	TOP engine
divergence depth in **LINES**	**+0.0313** [0.0118, 0.0541] p=0.003	**+0.0010** [−0.019, 0.022] p=0.92
divergence depth in **TURNS**	**+0.0287** [0.0072, 0.0514] p=0.007	**−0.0000** [−0.021, 0.023] p=1.00

MDE 0.0298 at this n. Under the engine that ships, **both are zero**. Under the baseline both are significant, **both have the WRONG SIGN** (more correct simulation → *larger* leaf error), and both sit essentially *at* the detection threshold. The sharpest form — **Δdepth against Δerror**, where every position-level confound constant across the two engines cancels — is **rho −0.0115 [−0.0307, +0.0082]** on 8,601 positions that parted in both.

6. THE TWO INSTRUMENTS THE PROJECT THOUGHT WERE DISAGREEING BEHAVE IDENTICALLY. 0.0313 against 0.0287; 0.0010 against −0.0000. **The turn metric is not degenerate on this sample** — 13 distinct values, 0 through 12, modal share 0.68 — so "turns cannot predict because it has no spread" is measured and rejected. Lines and turns are two readings of one thing, and neither reads on the leaf.

7. THE BINS ARE FLAT, AND THE BEST-FIDELITY BIN IS THE WORST. TOP engine, by line depth:

depth	0–4	5–9	10–14	15–19	20–29	30–49	50+	NEVER PARTED
n	130	1,421	2,165	1,505	1,474	1,126	788	246
mean Brier	.263	.280	.281	.286	.284	.290	.261	**.321**

The 246 positions where MEDICHAM matched the authority for the whole game have the **worst** leaf error and the only sub-chance accuracy (46.9%). At n=228 decisive that interval spans 0.5 and the claim is *no trend*, not *inverted*. The largest fidelity improvement available — **241 positions that used to part and now never part** — moved the leaf error by **+0.00213**, one standard error, in the wrong direction.

8. THE FIDELITY GAIN ITSELF IS REAL, AND IT REPLICATES THE LADDER ON AN INDEPENDENT SAMPLE. These are corpus positions, not the swarm's team pool, and the engine still improved: games that never part **13 → 246**, median first-divergence line **12 → 16**, games diverging

8,842/8,855 → 8,609/8,855. **Median completed turns: 1 → 1.** So the night's work was real. It simply does not reach the leaf.

9. AN INCIDENTAL, CONTROLLED SPEED NUMBER — the first this division has. §0a says three readings of engine throughput disagree by an order of magnitude and none is reproducible. This one is like-for-like by construction: identical positions, identical seeds, identical rollout budget, identical 6-shard layout, same machine, back to back. **One MILTANK in-game leaf call costs 1,478 s / 8,883 on the pre-WIRE-1 engine and 2,591 s / 8,883 on WIRE 10 — the shipping engine is 1.75× SLOWER per leaf call.** It is not a battles/sec or turns/sec figure and must not be quoted as one; it is the cost of the thing the search actually spends its budget on. Filed to §0a rather than published as the missing artifact.

WHAT THIS MEANS, PLAINLY. Ten WIRE rungs made the simulator measurably more correct and bought **nothing** at the leaf, on the largest sample this division has ever run, with the null tighter than the smallest detectable effect. **Engine correctness is not what limits the leaf.** The leaf's failure is calibration — an 88-point predicted range compressed onto 13 observed points — and grinding the differential further cannot touch that. The remaining candidates are the ones §1 already lists and this measurement does not settle: the leaf is scored at **turn 0**, where a game-outcome label exists and where the position genuinely carries little information; and the **playout policy** is uniformly random at explore=1.0, which is a policy question rather than a mechanics one.

WHAT WOULD FALSIFY THIS. A depth metric that is not first-divergence — the differential stops at the first parting, so a position scoring 16 lines has an unmeasured remainder. If somebody builds a *cumulative* divergence measure and it predicts leaf error where these two do not, this section is wrong and should say so.

Four things about the run itself, because the run nearly went wrong twice.

- `engine/leaf_scoring.js` **is new, and it is not a second implementation.** It holds the Brier, log-loss, interval, reliability, ECE and noise-floor definitions that lived as private functions inside `backtest_winrate.js`. `node engine/leaf_scoring.js --verify` replays `data/winrate-backtest-rows.jsonl` and reproduces **749 of 749 scalars** of the published `data/winrate-backtest.json`, exactly. The generator refuses to run if that fails. `backtest_winrate.js` **was deliberately NOT edited to import it** — it is the generator of a published artifact, it has no `--out`, and it cannot be smoke-tested without overwriting the file everybody quotes. Filed below.
- **The first full run was killed by the harness at 65 minutes** with the baseline arm finished and on disk. Resume support was added and the arm recovered. The guard written while doing so **caught a real hole**: a reuse check on COUNTS passes when a re-derived sample has the same size and different members, and the store grew 8,887 → 9,003 during the run. It checks the id set now, and a resumed run reads the photograph rather than today's store.
- **A reused arm is REPRODUCED, not trusted.** 24 positions of each reused leaf arm are re-run by the current code and must be bit-identical; they were, for both engines.
- **The depth arm has no per-position reproduction, and that is a finding rather than an exemption.** The first version of that check refused the depth arm, 16 of 24 positions disagreeing. `game_differential`'s driver is coverage-seeking and its `CLICKS` / `COV_HITS` carry across games on purpose, so a divergence depth is a function of the position **and of every game played before it**. A 24-position slice starts from an empty click history and plays different games by construction. The substitute is the **reversed-order control**, which is why it was built: same release, same positions, driver history deliberately changed, **rho 0.836 [0.825, 0.846]** on 8,855 positions. That is the ceiling on every correlation in §5, and it is high — the nulls above are the world, not the ruler.

Filed from this pass, not fixed:

- `backtest_winrate.js` **should import** `engine/leaf_scoring.js`**.** One line, in the pass that next re-runs it. Two copies of a scoring rule is how `data/guru.js` came to say 0 where its source said 6.
- `provenance.js` **classes this artifact as OPEN-SHEET and it is a LADDER artifact.** The corpus detection reads one require hop deep, and `engine/game_differential.js` *mentions* `data/games.bo3.jsonl` in a comment — so a comment one file away picks the denominator. Drift is reported as 10.7% against the open-sheet ceiling where the ladder ceiling gives ~2%. Consequence is nil today because the POWER line correctly says the missing games move a proportion by at most 0.33 points, below the 0.43-point floor — but the mechanism is the same one §5 records for `winrate-backtest.json`, recurring through comments rather than code.
- `docs/ABRA-whitepaper.md`**'s 3.68.0 block quotes** `wire-ladder` **figures that the artifact does not carry** — "1,995 games per arm" and "mean 15.0 → 24.0" against `data/wire-ladder.json`'s 1,997 and 14.78 → 33.98. Another division was mid-pass on that file while this ran. Flagged, not edited.

0b. THE HEADLINE METRIC IS NOW EXPLOITABILITY, AND THIS DIVISION DOES NOT HAVE ONE — 2026-08-06 (3.59.0)

ADR-003 is accepted and published across the docs in this pass. **Exploitability replaces win rate as the project's headline metric, and the published comparator is VGC-Bench's approximately 100%.** That lands on this division, because the instrument is `engine/exploit.js` → `data/exploitability.json`, and that artifact is the one thing in the repository that is *declared void*.

Nothing here is a new measurement. The reframe rests on a number somebody else published and on a speed benchmark run earlier tonight. This entry records what the reframe costs MEASURE.

1. The headline metric has no value. `data/exploitability.json` is `void: true` and `provenance.js --strict` exits non-zero on it — §5g of this file has the full account. The 2026-07-26 figure was fitted on 17 features against the 58 shipped, on an engine 25 wire-fixes old, before the quality filter existed. The 2026-08-04 re-run is void because `data/policy-weights.json` — the defender — was refitted at 22:15:24 UTC while it was running. **So the comparison this project now leads with is a comparison it has set up and not made.** Say that, in those words, until it is made.

2. It raises the bar on the frozen-release discipline, and the evidence is our own. An exploitability run needs a best response trained against a *frozen* agent over thousands of games. The 2026-08-04 void **was** an exploitability run. That is a demonstrated failure mode on this exact measurement, not a hypothetical, and it is why the release boundary is a precondition rather than a nicety. `engine/exploit.js` stamps no engine digest and no digest of the target vector, which is why nothing caught it at the time.

3. The comparability argument is sound and should be stated whenever the number is. Their checkpoints are Reg M-A, ours Reg M-B, and their own paper shows policies do not transfer across team sets — a head-to-head is impossible. **Exploitability is intrinsic**: it is defined against a best response trained against *you*, in *your* format. So the two numbers sit on one scale although the two agents can never meet. This is the rare case where a comparator is legitimate without a shared population, and the reason has to travel with the figure or somebody will eventually read it as a head-to-head.

4. A noise floor for exploitability does not exist and is not obvious. §6 of this file says the floor belongs to the measurement rather than to a global constant. For a hill-climb the natural split-half does not apply — the arms are not exchangeable. The 2026-08-04 run gives the shape of the problem rather than a floor: it accepted **1 of 24** steps and its step scale decayed to 0.0168, so from round ~10 it was perturbing a near-copy of MAG. **An attack that dies in 58 dimensions returns an uninformative null on a still tree too**, which means a low exploitability figure from this tool is not yet distinguishable from the tool failing. That is the first thing to fix, and it is upstream of producing any number at all.

5. What this division must NOT do with the reframe. It must not report an exploitability number computed against a moving target, and it must not report an interim one. The SPRT rule applies with full force here: 66.7% became 44% and 57.7% became 50% in this project because somebody looked early. An adversarial search that is watched while it climbs is the same failure with a different name.

0a. ENGINE SPEED IS UNMEASURED BY THIS DIVISION'S STANDARDS, AND IT DECIDED AN ARCHITECTURE

Three readings of MEDICHAM's throughput are on record and they disagree by an order of magnitude: 3,401 battles/sec (ADR-001, July), 1,606 battles/sec (ROADMAP #61) and 13,041 turns/sec (the 3.59.0 re-measurement). The published ratio against `champions_sim` moves with them: **117x, corrected to 24.9x**. All three are now stated in ADR-001, ADR-002, `docs/MODELS.md`, the white paper, the deck, `docs/SUMMARY.md` and the technical docs, with the July figures kept and annotated rather than rewritten.

Everything wrong with this is a MEASURE problem, and none of it is an ENGINE problem.

- **No artifact holds any of the three.** There is no `data/*.json` for engine speed. A figure that decided the project's largest architectural decision has never been through the machinery every win rate in this repository goes through.
- **No generator exists.** There is no script in the repository that runs the comparison, so none of the three can be reproduced, and the two that disagree cannot be adjudicated.
- **No ratchet.** Nothing fails when engine speed regresses. ROADMAP #61 already recorded that a 2x regression went unseen for that reason; the same hole let a 4.7x error in a published ratio survive two weeks.
- **The unit was doing damage.** `turns/sec` compares the two engines and `battles/sec` does not, because MEDICHAM was driven to its 60-turn cap and Showdown with `choose('default')` to a natural end — so a "battle" is not the same quantity of work on the two sides. The 7.7x battles/sec ratio measured tonight is **not** a like-for-like number and must not be quoted as one. Two of the three historical readings are in the wrong unit for the comparison they were used to justify.

The correct fix is a stamped artifact with a declared method, not a fourth reading, and it is the same fix this division applied to the four rollout gates: a generator, a sidecar recording the machine and the configuration, and a floor. Filed here rather than done — it is a new measurement, and this pass was a publication pass.

One location is left uncorrected and is flagged rather than edited: `engine/champions_sim.js` lines 10 and 26 still state the 117x in the file header. It is ENGINE's file and ENGINE is working in the tree tonight.

1. LEAF CALIBRATION — MEASURED 2026-08-04. The leaf is not calibrated, and the claim is now powered.

`data/winrate-backtest.json` was re-derived against the current engine, on the whole clean corpus instead of a 350-game subsample, and it publishes a reliability curve. The finding is worse than the verdict string it replaces, and worse in a specific and actionable way.

The old number scored a leaf no live decision calls. It measured `winProb2` — `battle()` at MEDICHAM's default 20-turn horizon with entry effects re-fired. MILTANK calls neither: its team-preview leaf is a greedy playout at `maxTurns=60` with `seeded:true`, and its in-game leaf is `rollout_leaf.rolloutWinProb` at `explore=1.0 / foePolicy=uniform / maxTurns=60`. All three are now scored on identical positions, so the difference between them is about the leaf.

Confidence carries no information. The curve is close to a horizontal line:

leaf	says 0-10%	says 90-100%	discrimination	Brier vs coin (paired)
in-game, 200 rollouts, held-out n=1,378	wins 53.8% (n=52)	wins 53.6% (n=56)	50.99% [48.3, 53.7]	**+0.0502 [0.0371, 0.0628]**
preview, 40 rollouts, full clean n=6,886	wins 45.7% (n=831)	wins 55.3% (n=933)	53.22% [52.0, 54.4]	**+0.0740 [0.0668, 0.0813]**

Positive is worse. Both leaves are decisively **worse than a coin** on Brier and on log-loss, and worse than player-Elo, paired on the games where both have an opinion. The preview leaf puts **25.6% of all its predictions into the two extreme buckets**, where it is wrong by ~40 points.

Discrimination and calibration are separate failures needing separate answers. The preview leaf does rank — 53.22% on 6,700 decisive calls, p < 1e-4 — real, but only ~1.9 points above the split-half noise floor. The in-game leaf does not rank at all: 50.99%, p = 0.47. Randomising the playout bought variance and spent the signal.

What this is not. Nothing here says the engine is broken. The legacy `winProb2` leaf reproduces the 2026-08-02 number closely on the current engine — held-out log-loss **1.0243** against the **1.0748** published, discrimination **51.94%** against **52.63%** — so the twenty-two engine commits in between did not move the headline. Do not spend this finding on a mechanics hunt.

Also fixed here: the split was cutting on **store append order**, not date, and the store carries 4,775 date inversions. A side-symmetry witness scores 400 boards from both sides and reports mean(p1+p2−1) = **−0.0099**, so no side advantage inside the engine is contaminating the result.

Still open, in order:

- the leaf is scored at **turn 0**, because that is where a game-outcome label exists. `rollout_r1.js` scores mid-game positions and is the other half of this; the two have never been read together.
- the **horizon** is the first suspect. `battleResult` falls back to bodies-then-HP whenever the playout does not finish, so a confident number can be a material count wearing a probability's clothes. That is a SEARCH change, not a MEASURE one — file it, do not fix it here.
- `data/winrate-backtest-rows.jsonl` holds the per-game predictions, so the curve can be re-cut without re-running the ~15 minutes of rollouts.

2. R4 has an artifact — CLOSED 2026-08-04

`engine/rollout_r4.js` writes `data/rollout-r4.json`, and `status.js` now prints the verdict out of that file instead of `NO ARTIFACT`. It does not re-pair anything: it shells out to `sprt.js --verify` and `paired_h2h.js` and refuses to write if they disagree, the same way `status.js` shells out to `provenance.js`.

The remembered 55.5% held. **ACCEPT H1 — MILTANK takes 55.5% of 535 DECISIVE PAIRS, decided after 522 of them, LLR 3.00 against a 2.94 bound.** The corpus is 5,248 lines, which is 2,624 games, which is 1,312 seed pairs — the store writes a log-only companion record under the same id, so a line count double-counts every game and the handoff's "5,248 games" was exactly twice the truth. The artifact records all four numbers and asserts the invariant that makes them relate.

Two things it is not. The point estimate is **stopped at a boundary**, so it is biased high and the 95% CI beside it is a fixed-n formula quoted for context, not the inference — read the verdict. And status.js classes the corpus **PRE-CHANGE**: `engine/medicham2-browser.js` moved 04:47, the games were played 04:41. Both arms shared the pre-fix rollout model, so the contrast is fair and the run stands as a measurement *of that build*; that the edge survives into HEAD is an assumption. It gets re-run at the next frozen engine release.

No A/A run exists for this comparison, so the noise floor is **not established**. The substitute in the artifact is three independent split-half cuts of this run: spreads of 0.2, 3.9 and 1.3 points against an effect of 5.5. One cut alone would have been useless — the spread of a single split-half is itself a draw with sd about 4.3 points at

this sample size.

3. R1 has an artifact, and it does not say what the docs said — CLOSED 2026-08-04

`engine/rollout_r1_artifact.js` writes `data/rollout-r1.json` from the committed row dump, and `status.js` prints its verdict. The gate previously read a file of the same name that `engine/rollout_r1_join.py` wrote for the **withdrawn** cross-language join — nothing was hidden, the join prints its own withdrawal, but the gate read it because it owned the filename. The join is now `data/rollout-r1-withdrawn-join.json` with `withdrawn: true`, and `status.js` refuses to print any artifact carrying that field.

The recomputation does not reproduce the published PASS. `docs/ROLLOUT-design.md` claimed 68.18% against material's 65.26%, +2.91 [1.79, 4.04]. From `data/rollout-r1-rows.jsonl` the same formulas give **65.72% against 65.26%, +0.46, 95% CI [-0.72, +1.63] — UNDECIDED** on 9,201 positions.

The material column matches the published figure to the digit, so it is the same sample. The rollout column reproduces §4.2.1's **greedy** calibration table bin-for-bin, so the surviving dump is the `explore=0` incumbent and the `explore=1` run that produced 68.18% left no file. That is the lesson, not the arithmetic: **the dump stamped no `N`, no `explore` and no build digest, so two runs four accuracy points apart were byte-indistinguishable.** `rollout_r1.js` now writes `data/rollout-r1-rows.meta.json` beside every dump. R2 and R3 still have the same hole.

The split-half spread of this run ranges 0.43 to 2.01 points against an effect of 0.46 — the effect is inside its own noise floor, which is an independent route to the same UNDECIDED.

Open consequence, filed to SEARCH: `--rollout-explore` defaults to `1.0` and `engine/rollout_leaf.js:147`, `engine/mag_bot.js:145` and `docs/MILTANK.md` all cite 68.18% as the reason. Re-running `EXPLORE_LIST=1 DUMP=rollout-r1-rows.jsonl node engine/rollout_r1.js` and then `node engine/rollout_r1_artifact.js` settles it, and R2 says the leaf is cheap.

> ***SEARCH RAN IT, 2026-08-04. The published figure reproduces.*** `data/rollout-r1-explore-sweep.json` *and* `data/rollout-r1-explore1.json` *: on the identical 9,201 positions, explore=1.0 judges at* ***67.971%*** *against the published 68.18%, and its lift over the same material baseline is* ***+2.706 [1.596, 3.817]*** *against the published +2.91. Paired against the greedy arm it is* ***+2.25, 95% CI [1.31, 3.19]****, monotone in explore (0 → 0.5 → 1.0 = 65.72 → 67.58 → 67.97) and it holds at the live 60-turn horizon.* ***The retraction was right about the provenance and wrong as a guide to the arm*** *— R1 is UNDECIDED on the incumbent and a PASS on the arm that ships. Three things this division should act on: 1.* ***The command in this section would have destroyed the evidence.*** `DUMP` *resolves under* `data/`*, so* `DUMP=rollout-r1-rows.jsonl` *overwrites the committed greedy dump — the only record of the incumbent arm, committed for exactly that reason. SEARCH used a new filename. Worse, the sidecar path in* `rollout_r1.js:338` *is the hardcoded literal* `data/rollout-r1-rows.meta.json` *whatever* `DUMP` *is set to, so it lands beside the wrong dump.* `rollout_r1_artifact.js` *rejects it on the name-and-row-count check, which is the check working — but the fix is to derive the sidecar path from* `DUMP`*. 2.* `status.js:229` ***still prints the greedy arm as "R1 leaf accuracy".*** *SEARCH did not overwrite* `data/rollout-r1.json`*, deliberately: it is this division's artifact, written hours earlier, and it is the only record of the incumbent. But the line now reads UNDECIDED for a configuration the bot does not run. One line — point it at* `rollout-r1-explore-sweep.json`*, or print both arms. 3.* `engine/mew.js` ***exposes no*** `--miltank-explore`**.** *So the question this settles — which playout JUDGES better — cannot be escalated to the one that matters, which playout WINS more. R4 was itself run at explore=1.0 and cannot arbitrate its own setting. Two parsed flags on* `mew.js` *and the A/B becomes runnable. One hypothesis this division filed to SEARCH is* ***measured and rejected****:* `battleResult` *scoring bodies-then-HP on unfinished playouts is real but is not the mechanism. Over 1.1M playouts, 99.5–99.8% end by an actual wipeout at every explore setting and at horizons 20 and 60; cap-hits are 0.2–0.5%. Exploration makes playouts longer (4.4 → 6.1*

> *mean turns), not truncated. Filed to ENGINE as a latent hazard. The flat reliability curve in* `data/winrate-backtest.json` *needs another explanation — and note that on human corpus positions the explore=1.0 leaf is* ***not*** *flat: its ECE is 0.104 with a monotone curve running 0.166 → 0.842, against 0.196 for greedy.*

4. R2 and R3 stamp their configuration now — and R3's published number has no control

`engine/run_stamp.js` is one implementation of the sidecar `rollout_r1.js` hand-rolled inline, so the next gate cannot grow a second format. It writes `<artifact>.meta.json` — the same convention that makes `data/rollout-r1-rows.meta.json` describe `data/rollout-r1-rows.jsonl` — carrying N, explore, every knob including the ones left at a default, sha256 content digests of every source the gate reaches, the commit, and **whether the tree was dirty**. A clean commit id over a dirty tree is a lie of exactly the kind this exists to stop.

Both published numbers reproduce as arithmetic, and neither reproduction is worth much. That is the finding, and it is a different finding from R1's.

gate	published	recomputed from committed evidence	reproduces
R2	477 boards over 200 games; 5.83 ms median at n=10	the affordability table (K=3 → 0.47 s median, 1.75 s worst; K=4 → 1.49 s / 5.53 s) reproduces to the digit from `leafCostMs`	**derived layer yes, base layer NOT CHECKABLE**
R3	72.9% over 70 decisions (19 agreed, 20 skipped)	100 × (70 − 19) / 70 = **72.857142857142854**, bit-identical to the stored float	**yes, and it is a tautology**

R3's divergence is a pure function of two fields in the same file. There are no per-decision rows, so "it reproduces" means the artifact is internally consistent — nothing more. R2 dumps no per-leaf timing at all, and a duration cannot be recomputed by anyone in principle: it is a fact about a machine under a load, and nothing records the CPU, the node version or what else was running. **R2 is the one rung that is re-run or it is nothing.**

THE R3 RESULT IS NOT INTERPRETABLE AS PUBLISHED, and this outranks the sidecar work. `rollout_r3.js` computes the only control that makes a divergence rate mean anything — the same search on a different seed disagreeing with **itself**, where the truth is 0.00 by construction — and it `console.log`s it and does not write it. Its own verdict branches on that number: `rate <= floor` prints NOT A RESULT. So `data/rollout-r3.json` cannot say which branch its own run took.

`docs/ROLLOUT-design.md` §5 does publish floors — 71.7 / 50.0 / 45.5 / 43.8% — but **for four earlier runs, none of them this one**. At N=20 that floor measured *higher* than the divergence. The committed artifact is a fifth run at N=600 on 70 decisions, and its floor was printed to a terminal and lost. `engine/status.js` and `docs/MILTANK.md` both quote its 72.9%, and `MILTANK.md` spends it on a decision: "so it does diverge, and the equilibrium version is worth building."

Read plainly: **the divergence is probably real** — the doc's floor fell from 71.7% to 43.8% as N rose from 20 to 200, and this run used N=600, so its floor should be lower still. But *probably* is an inference from a different run, and the Wilson interval on 51/70 is **[61.5%, 81.9%]**, which is wide enough that a 44%-class floor is the only thing separating a result from an artefact of the argmax. The next run writes the floor; until one does, the 72.9% is a headline with its control missing.

A second defect, found on the way: `data/rollout-r3.json`**'s own caveat is false about the run it describes.** It reads "Switch candidates are excluded and counted". Commit `b4ec80b` deleted the `if (ca.switchTo || cb.switchTo) continue;` line — switches went **on** the menu, which is what that commit was *for* — and left the string alone. It has shipped that way since 2026-08-03, and the `withSwitch` / `choseSwitch` counters that commit added were printed and never written, so its own headline ("4 of 12 when one is on the menu") lives in a commit message.

R2 timed a leaf the bot does not run. `rollout_r2.js` called `RL.rolloutWinProb` without `explore` or `maxTurns`, inheriting `engine/rollout_leaf.js:197`'s `explore = 0` and `engine/medicham2-browser.js:1079`'s `maxTurns = 20`. MILTANK's in-game leaf is **explore=1.0 at maxTurns=60** — a randomised playout at three times the horizon. That is R1's hole in cost form: two library defaults, written down nowhere, deciding the number. Both are now explicit, overridable and stamped, with defaults that preserve the old behaviour exactly so nothing re-dates the committed artifact by accident.

Also corrected in the generators, all of them visible in `status.js`:

- `games` was the `GAMES` environment **cap**, not a count. `status.js` printed "477 boards over 200 games", so an environment variable was being read as a measurement. It is now the distinct games actually traversed, with the cap beside it as `games_requested`.
- `leafCostMs` quantiles per N were computed over possibly-different board sets — a leaf returning null at one N and not another silently misaligns the columns — and only the n=10 count was recorded. `samples_per_n` now records all of them.
- R3's disagreement-gap median was computed twice, once to print and once to store. One variable now.
- `docs/ROLLOUT-design.md` §5's "roughly 200x the simulated turns per millisecond" is **155x** by the arithmetic of the two artifacts it cites (10 × 20 turns / 5.83 ms against 1 / 4.52 ms), and 155x is itself a ceiling because it assumes no playout ends early. `rollout_r2.js` now prints the division instead of a remembered figure. **The doc still says 200x** — see filed, below.

Two retrospective sidecars were written by `node engine/run_stamp.js --reconstruct`, which infers the build from the commit that carried the artifact and marks every field `reconstructed: true`. Both score HIGH: `data/rollout-cost.json` was written 25 s before `05248f2`, `data/rollout-r3.json` 159 s before `b4ec80b`. That is evidence about a commit, not a record of a run, and it says so on every line — a stamp that hashed today's sources would describe the file rather than the run, which is `data/rollout-r1.json`'s own stated reason for recording null.

Filed, not fixed:

- `data/rollout-cost.json` **should be** `data/rollout-r2.json`**.** It is the only rung whose file does not carry its gate's name. Four readers: `engine/status.js:230`, `web/build-status.js:200` and `:265`, and the generated `web/status-data.js`. Three of the four are under `web/`, which MEASURE does not own. A rename that misses a reader prints NOT DERIVED and reads as "nobody ran this", which is worse than the inconsistency. Needs WEB in the same pass.
- ~~`n` / `n_unit` **on R1 and R4.~~ DONE 2026-08-04** — see §7 below.
- ~~`engine/rollout_r1_join.py` **writes a naked** `isoformat()`**.~~ DONE 2026-08-04, and it was five files, not one** — see §8 below.
- `docs/ROLLOUT-design.md` **§5's 200x, and §R3's PASS.** Both are SEARCH's document and a SEARCH explore sweep is live. §5 should read 155x-at-most, and the R3 PASS should name which run it is quoting, because the floors in its table belong to runs that are not the committed artifact.
- `docs/MILTANK.md:70` **spends R3's 72.9% on a build decision** without its control. Same owner, same reason.
- `engine/rollout_r1.js` **should call** `engine/run_stamp.js` instead of its inline copy. SEARCH holds that file for the explore sweep. The shapes are identical today; two copies is how they stop being identical.

5. The possibly-stale artifacts, and the one class the checker could not see

`node engine/provenance.js` lists them; `node engine/provenance.js --graph` now prints the derived artifact graph itself, which is the part of that tool that could be silently wrong. Most entries are ordering artefacts inside a single run and are already annotated as such. The ones to actually chase are those older than `policy-weights.json`, those recording no game count at all, and — new — those carrying a **CORPUS DRIFT** note.

THE CANONICAL READER WAS HIDING ARTIFACTS FROM THE CHECKER. `provenance.js` derived an artifact's inputs by looking for a filename beside a read verb. A generator that loads the store the *recommended* way — `loadGames()` / `load_games()`, which resolve the path inside `engine/quality.js` and `engine/store.py` — never names `games.ladder.jsonl`, so it recorded **no dependency on the store at all** and was reported `ok` forever. Doing the right thing was the thing that made you invisible. Store derivation is now detected by the LOADER CALL (or an import of the reader), which is what a generator actually does.

Three more attribution faults surfaced with it, each of which had the same effect of exempting real artifacts from every corpus check:

- **A read `open()` looked like a write.** `pokemon-roles.json`, `role-matchups.json` and `roles-eval.json` were credited to `engine/build_roles_js.py`, which READS them to build a browser bundle, instead of `engine/roles.py`, which computes them from the store. `build_roles_js.py` touches no games, so all three were classed not-store-derived. A write test at line scope now requires a mode string.
- **The Python `OUT = os.path.join(...)` idiom hid a writer entirely.** `data/guru-matchups.json` — the source file at the centre of the `guru.js` divergence — had **no detected generator and was absent from the audit**. One level of variable indirection is now resolved.
- **Following into `engine/quality.js` classified everything as open-sheet.** quality.js names every store by construction, in its comments and in the error message that tells a caller how to pick one. `data/winrate-backtest.json`'s 6,886 **ladder** games were being judged against the 8,173-game open-sheet ceiling. It is a named exception now, with that reason.

The graph went from 76 artifacts (49 store-derived) to **84 artifacts (57 store-derived)**. One of the eight newly visible files, `data/counters.json`, was **older than the quality filter** — the UNSAFE condition this tool exists to catch, invisible for nine days. Regenerated (15 s); `provenance.js --strict` is green.

5g. AND IT WAS HIDING SEVEN MORE — the write detection had the same hole in the other language

84 artifacts → 91, 57 store-derived → 60, and two of the seven were UNSAFE. Found 2026-08-04 while writing the missing `docs/MODELS.md` entries: the roster guard reported `data/move-priors.json` as generated by `engine/state_encoder.py`, which only **reads** it. Three defects, each the same shape as §5's and each found by chasing the previous one.

- **`const` broke the path-indirection arm.** §5 taught this file the Python idiom `OUT = os.path.join(...)` … `json.dump(…, open(OUT,"w"))`. The JavaScript spelling is `const OUT = process.argv[3] || path.join(…)`, and the capture took the KEYWORD and then failed on the `=`. Every generator using it scored zero. The cost is the §5 cost exactly: `engine/policy.js` loads the store through `quality.js`, `state_encoder.py` opens no game file, so **the behaviour clone that nine files read was classed not-store-derived and exempt from every corpus check here.** It is 2026-07-31 vintage — the 5,269-game era — and nothing could say so.
- **A READ assignment is not a writer, and accepting `const` proved it immediately.** `const r = JSON.parse(fs.readFileSync(…'regulations.json'…))` followed later by an unrelated `fs.writeFileSync(file, r.body)` credited `engine/fetch_smogon_stats.js` with generating the format registry — a one-letter identifier matching `\br\b` inside any later write. An assignment whose own right-hand side is a read verb now never establishes a writer.
- **`named()` was a substring test, and the comment claiming that was safe was FALSE for the most-read file in the repository.** `ladder.json` is a substring of `games.ladder.jsonl`, so every generator that opens the game store was recorded as naming `data/ladder.json` — which is how `engine/refresh-site-data.NOARCH.py` was credited with generating **MACHAMP's hill-climb artifact**, whose real writer is `engine/ladder.js` (the on-disk keys are `ladder.js`'s to the letter). The same fault

hung a **phantom** `ladder.json` **input** on every store reader, and `roles.js` inside `pokemon-roles.json` did it again across eight more artifacts. Those show up as *"older than its input"* notes about dependencies that do not exist. An occurrence now only counts when the name is not the prefix of a longer one.

Corrected attributions, each verified against the generator rather than trusted: `move-priors.json` → `engine/policy.js`, `ladder.json` → `engine/ladder.js`, `dynamics.json` → `engine/dynamics.js`, `rollout-r4.json` → `engine/rollout_r4.js`, `smogon-priors.json` → `engine/smogon_priors.js`. Newly visible: `bring-bias.json`, `bring-priors.json`, `brood.json`, `core-matchups.json`, `exploitability.json`, `playstyle-matchups.json`, `smogon-priors-bo3.json`.

Two of the seven were UNSAFE, and the split between them is the point.

- `data/bring-priors.json` **was genuinely UNSAFE** — five minutes older than the quality filter, so computed under a different definition of which games count. It reads the store through `quality.js`. Regenerated (30 s), and it moved a figure a long way: `n_sides` **5,368 → 14,456**, and the format's **mega rate had been measured on 62 sides and is now measured on 12,442**, `p_side_megas` **0.9355 → 0.8785**, `p_mega_is_lead` **0.5345 → 0.5159**. `CLAUDE.md` sets a domain RATE floor there — *"a game without a mega should be rare"* — and the floor was being checked against 62 sides.
- `data/exploitability.json` **is a FALSE POSITIVE of the filter rule and a TRUE negative anyway, and it is left RED.** `engine/exploit.js` reads no game store at all — it plays self-play games from `policy-weights.json` — so the quality filter has no bearing on it, and the honest fix is a `not_store_derived` declaration, which only a re-run can write. **I did not add one, deliberately.** Stamping it would make a **genuinely unquotable** artifact look clean: it is PRIORITIES #18, WOBBUFFET's 63.2% fitted on **17 features against the 53 we ship**, and it is rendered on `web/stadium.html` and `app/stadium.html` today. Re-running it is a ~4,000-game adversarial search against a mid-flight MAG, which the engine release boundary forbids.

So `node engine/provenance.js --strict` **exits 1 on one artifact, and** `tests/run-all.js` **gates on it. That is stated, not filed.** The artifact has been invalid since 2026-07-26; the only thing that changed today is that something can see it. It needs Will's call between re-running WOBBUFFET after the release boundary and pulling the number off the two stadium pages.

5a. CORPUS DRIFT — and the answer to "two definitions of clean games"

There are not two definitions. There is one, and the other number is four days old.

`data/live.js` and `data/winrate-backtest.json` said 6,943; `data/meta-usage.json`, `data/roles-eval.json` and `data/guru-matchups.json` said ~5,269. Measured rather than argued:

figure	written	what it is
6,943	2026-08-04 03:09	`load_games(clean=True)` over the store as it stood then
6,890	2026-08-04 **02:52**	the same predicate, 17 minutes earlier — `backtest_winrate.js` began its run then and the collector appended **exactly 53** clean games while it ran
6,886	—	6,890 minus 4 games whose `winner` matches neither player's name. A genuinely narrower question, and it is already NAMED: `scorable` / `dropped_no_label`
5,269	2026-07-31 16:42	the same predicate over a store holding 29,117 collected instead of 38,587
5,265	2026-07-31 16:43	5,269 minus the same 4 unlabelable games

Collected grew ×1.325 and clean grew ×1.318 over those four days. A changed predicate does not scale with the corpus; a snapshot does. And `tests/test-quality.js` run tonight has the JS and Python readers selecting the **identical** 6,943 ids, sha `60aab8e1978e7554` on both sides. So renaming anything would have been wrong: **the**

defect was a date, not a word.

Why nothing caught it: mtime cannot. The store is append-only and its mtime moves every hour, so an mtime rule would mark every store-derived artifact stale within an hour of being rebuilt — a gate that cries wolf. `provenance.js` now compares the **declared count** against the clean corpus and warns past 10%, which the measured growth rate (~7%/day) makes "roughly a day and a half behind". Thirteen artifacts are flagged, including all three named above at 24.1–24.2%.

Two supporting fixes, both of which were the reason the headline artifact escaped:

- `declaredGames` now reads an explicit corpus claim first — `provenance.funnel.clean`, `provenance.usable`, `corpus.clean_games`. `data/meta-usage.json` states its population more carefully than any other file in the repository and had **no key the checker looked at**, so the file that started this question was the one it could not see a count for.
- The drift check ignores a bare `games` key, and that is deliberate: `rollout_r2.js` published `games` as the GAMES environment **cap**. Until every writer says whether `games` is a corpus or a sample, a drift figure computed on it is a guess, and the fix belongs in the generator.

Do not touch `data/quality-filter.json` to record the new funnel. Its mtime is `FILTER_MT`; bumping it marks every older artifact UNSAFE and turns `--strict` red across the repository.

5b. `data/guru.js` said 0 where `data/guru-matchups.json` said 6 — and both were misleading

`build/build_guru_js.js` read `g.decisive`. `engine/guru.py` writes the list as `decisive_matchups`. A missing key gave `[]`, the generator then recomputed `n_decisive` from **its own empty fallback**, and shipped a provenance note asserting "ZERO statistically-decisive matchups on this population" as though it were a finding. The 144-cell matrix was byte-identical throughout, which is why nobody noticed. `venusaurmega` / `venusaur-mega`, in a new pair of files.

The true value is 6 directed = 3 distinct matchups, and the generator carries the source's count now instead of recomputing it. Three things stop it recurring, all derived: every source key must be projected or named in `DELIBERATELY_UNUSED` with a reason; the source must agree with itself (`decisive_matchups.length === min(n_decisive, 20)`); and `build_guru_js.js --check` rebuilds the bundle in memory and diffs it, run by `tests/test-guru-derived.js` on every suite run.

And the measurement underneath it: 3 of 66 pairs is what chance produces. Each cell is its own 95% test. Over 66 unordered pairs the expected number clearing that bar with no real effect is **3.3**, and **3** clear it. The smallest exact two-sided binomial p-value in the matrix is **6.1e-3** against a Benjamini-Hochberg threshold of **7.6e-4**, so **zero survive FDR at q=0.05** and zero survive Bonferroni. The bundle publishes both counts (`n_decisive`, `n_decisive_corrected`) plus the arithmetic. The old file's "ZERO decisive" string was accidentally right and arrived there by a bug — which is worse than being wrong, because it cannot be checked.

`web/index.html:1845` gates a panel headed *"These are the matchups we can actually trust"* on `GURU.decisive.length`, so it will now render three matchups that do not survive multiplicity. It should read `decisive_corrected`. WEB's file; flagged, not edited. Note the same issue already affects `isSig()` in the matrix and the "statistically significant loop" claim, independently of this fix.

`data/guru-matchups.json` is itself 24.2% behind the corpus. Regenerating it is a separate, deliberate refresh — it moves every number the GURU booth renders — and is not done here.

5c. The thirteen drifting artifacts — TRIAGED, and NONE of them is a silent refresh

`node engine/provenance.js` flags thirteen artifacts 10.6–47.2% behind the clean corpus. The question that decides what to do with each is *does regenerating it move a published figure*, and it was measured rather than guessed: every scalar in each artifact was matched against the living docs and the site pages, at headline

depth, with the universal constants (0.693, 0.25, 50%) excluded because they appear for reasons that have nothing to do with the artifact.

The answer is that there are zero safe silent refreshes in the set. Nine carry a verdict string or an interval-based claim that regenerating could flip; the other four have headline figures typed into `MODELS.md`, the white paper or `SUMMARY.md`, which `engine/sanity_check.py` §5 cross-checks. By this project's own living-docs rule, regenerating any of them is a docs pass, not a refresh.

artifact	behind	what regenerating moves	act
`war.json`	**47.2%**	verdict *"WORSE THAN A COIN AT EVERY REGULARISATION STRENGTH TESTED"*; `held_out.log_loss` 0.694 in MODELS + white paper	**STOP** — a null on a corpus that has since doubled is the most interesting one here
`policy-eval.json`	43.8%	verdict *"phase-conditioning did not help; species-only prior retained"*	**STOP**
`pory-eval.json`	33.4%	`log_loss.pory` 0.6298 in white paper + SUMMARY, gated by sanity_check	**STOP** — restamped instead, see §5d
`pory-nn.json`	29.4%	**Blast radius OVERSTATED in this row and corrected 2026-08-04.** `val_logloss` and `auc` are **not keys in the file** — it holds an `arms` array with per-arm `logloss` / `acc` / `auc`. And the `71.6%` in MODELS.md and the white paper is the **policy clone's top-3 accuracy**, a different measurement that happens to match. **No living doc cites PORY-NN**, so regenerating moves zero published figures. Regenerated	done
`xatu-belief.json`	29.3%	`n_games` 4,910 and `top1_accuracy.belief` 31.2% in MODELS; an improvement CI clear of zero	**STOP**
`guru-matchups.json`	24.2%	every number the GURU booth renders; `log_loss_matchup_prior` 0.712 in the white paper	**STOP — and explicitly not in this pass**, WEB is in that booth
`roles-eval.json`	24.1%	headline *"0.6935 vs a coin 0.6931 and rating 0.6967"* — a knife-edge that regeneration can flip either way; six figures in MODELS	**STOP**
`pokemon-roles.json`, `role-matchups.json`	24.1%	same generator as roles-eval (`engine/roles.py`); all three move together or not at all	**STOP**
`vocab-usage.json`	24.1%	`role_coverage_of_battle_usage` 97.2% in MODELS	**STOP** (one-line docs pass)
`xatu-context.json`	24.1%	improvement CI [0.022, 0.042] rendered on the site	**STOP**
`meta-usage.json`	24.1%	**nothing typed** — the closest thing to a clean refresh, and PRIORITIES #16 names `node engine/analyze.js data/games.ladder.jsonl` as its closing command	**ASK** — `engine/mag_bot.js` and `engine/mew.js` read it, so it is the live bot's meta prior, and moving that is not MEASURE's call with an engine release boundary pending. It is **not** a refit trigger: `engine/feature_fixture.js` excludes it by name and `board.js` never reads it
`counterplay.json`	10.6%	`result.mean_coverage_gap` 0.0321, CI [0.0086, 0.0563] — an interval that currently excludes zero	**STOP**

A false positive in the drift check itself, measured not argued. `pory-eval.json` is reported 33.4% behind, and it cannot be less than ~21% behind however often it is regenerated. Its population is not *clean ladder games*; it is *clean ladder games whose raw log is present and names a winner*, a strict subset. Running the generator over the whole current corpus reaches **5,456 games, not 6,943**, so its true drift is 15.3%. Every artifact reading `games.ladder.raw-logs.jsonl` has this. `provenance.js`'s existing escape hatches (`gate`, `games_requested`, `sampled`) do not cover it, because this is neither a gate nor a deliberate sample — the artifact needs to declare the ceiling its population can reach, in the same style. Not fixed here: `provenance.js` was built tonight and a second hand on its drift arithmetic is how two files come to disagree about one fact.

5d. PORY — the artifact restamped, and the coefficients were wrong for ten days

The verdict was not stale. The generator was answering the wrong question, correctly, every run. `data/pory-eval.json` still read *"a real, calibrated value net"* ten days after PORY was retracted, and `engine/pory.py`'s gate was `hi < coin and hi < material_heuristic` — which is TRUE on this sample (hi 0.6456 against 0.6931 and 0.6550). Restamping the file alone would have been undone by the next run. `material_heuristic` is a crude 0.75/0.25/0.5 **sign** rule; beating it is arithmetic.

The tie is now measured, not inferred. Against a logistic on `[alive_diff, hp_diff]` alone — same gradient descent, same standardisation, same temporal split — PORY scores **0.629799 to 0.629778**: paired difference **+0.000021 (PORY worse), 95% CI [−0.000013, +0.000056]** clustered by game over 925 held-out games. On the current corpus (5,456 games) it is **−0.000001, CI [−0.000031, +0.000030]**. The retraction is robust to the corpus growth.

The reduction is structural, so no amount of data changes it. Every state is emitted from both perspectives with the label flipped, so the gradient on any column identical across the two rows cancels exactly: intercept and `turn/10` are pinned to `0.000000000`, not shrunk to it. `my_alive` and `foe_alive` swap and come back exactly antisymmetric (sum `0.000000000`). Five features, two degrees of freedom.

`engine/pory.py` **reproduces its own artifact bit-for-bit** — replayed on the identical first-4,623 clean-game sample it returned this file's weights, `feat_std` and log-loss exactly. So the fault was never the arithmetic. The gate now reads the paired difference, the withdrawn string travels under `withdrawn_verdict`, and `reduced_form` is derived from the file's own weights.

> ***REGENERATED 2026-08-05 on the current corpus, deliberately (the dispatch Will approved).*** `data/pory-eval.json` *now describes* ***5,883 games / 97,732 board-states*** *(was 4,623 / a 925-game test split) and* ***declares*** `population_ceiling: 5883` *— the §5f hatch, written by the generator on its first deliberate run since the hatch existed. Everything below survives the growth: paired difference vs the two-feature logistic* ***+0.000001, 95% CI [−0.000026, +0.000029]*** *clustered over 1,177 held-out games; the verdict string is unchanged. The numbers a document would quote moved: log-loss* ***0.6298 → 0.6236*** *[0.607, 0.6387], sign-rule heuristic* ***0.655 → 0.6428****, two-feature baseline* ***0.629778 → 0.623623****, reduced form* ***0.9943 / 1.4080 → 0.9809 / 1.4093****, accuracy 0.6264 → 0.630, ECE 0.017 → 0.0138. Doc locations quoting the old figures are listed in §13b; propagation is the router's pass, not this file's.*

The documented coefficients had no artifact behind them — the P1 class. `1.256 / 1.544` in `docs/MODELS.md` and `web/stadium.html:342` is commit `44e0fb0` (2026-07-24, `n_games` 7,381), the last run fitted on the **unfiltered store with bot games in it**. `7f74236` put every model behind the clean filter on 2026-07-26 and the coefficients moved to 1.0259 / 1.4347, then 0.9946, 0.9962, **0.9943 / 1.4080**. The retraction has been citing bot-contaminated coefficients as its evidence ever since. MODELS.md is corrected with the history; `web/stadium.html:342` **still says 1.256 — WEB's file, flagged not edited.**

5e. `tests/test-site-data-fresh.js` — two rules in it were wrong

It kept a second definition of stale, and it was the one `provenance.js` had already rejected. The verdict-input check compared each artifact's mtime against the newest `games.*.jsonl` and failed past a day. The store is append-only and the collector runs hourly, so that clock cannot be beaten: five artifacts were red and **four of them are clean** by the canonical rule. It delegates to `provenance.js` now, the same way `status.js` does. The founding case survives the change — `chomp-ev.json` four days behind is ~28% drift, well past the 10% threshold.

What delegation loses is stated rather than dropped: drift can only see an artifact that declares a corpus, and `chomp-ev.json`, `eval-report.json`, `policy-weights.json`, `policy-weights-joint.json` and `damage-validation.json` declare none. They are **listed every run without failing**, so the pressure is on the generator to record a count — the shape `tests/test-timestamps.js` already uses.

`--fix` would have refitted two models to make a freshness check go green. The guard that stops it detected a publisher by the filename suffix `-eval.json`, which is not a property of anything. It caught `engine/pory.py`. It did **not** catch `engine/nmf_roles.py` (writes `nmf-roles.json`) or `engine/xatu.py` (writes `xatu.json`) — both fitted models quoted in MODELS.md, both on the auto-run list. The rule now is that a bundle writes only browser files and a generator that also writes a `data/*.json` is a publisher; checked against all ten generators this test names.

Seven of the ten stale bundles were regenerated. **Four were byte-identical apart from a date stamp** (`mew.js`, `move-effects.js`, `mega-formes.js`, `status.js`) and two entirely so (`abra-meta.js`, `roles.js`) — pure mtime, the check crying wolf. **Two had really rotted:** `mag.js` was serving standard errors from before the last weight change (0.02452 against `policy-weights.json`'s 0.02363) and `scoreboard.js` was rendering superseded weights (1.1887 against 1.0884). That is the class this check exists for and it was real.

Also found: `build_mag_data.js` and `build_scoreboard.js` **crash without** `SHOWDOWN_PATH`, so `--fix` fails on them in any shell that has not exported it, and the test does not say so. Same shape as P0 #40 — two ratchets that crashed rather than failed for the same reason.

Still red, not filed: `data/pory-nn.json` at **29.4% corpus drift**. The command is `python engine/pory_nn.py`; it is a neural-net train and it republishes `val_logloss` 0.612 and `auc` 71.6%, which MODELS.md, the white paper and SUMMARY.md all quote. That is a stop-and-ask, not a refresh.

5f. IS THE DRIFT THRESHOLD A TREADMILL? Yes, and the unit is wrong — DECIDED 2026-08-04

`data/pory-nn.json` was regenerated on the current corpus and `tests/test-site-data-fresh.js` immediately reported **CORPUS DRIFT 15.7% — declares 6,008, 7,123 clean now**. The store grew during the retrain. That is not a bug in either tool; it is what a percentage of an unbounded append-only corpus does.

The verdict: a percentage is the wrong unit, and it is not a fraction at all — it is an age. The collector runs hourly and clean games grew 5,269 → 7,123 in four days. For an artifact of age Δt, drift is $1 - n(t_0)/n(t)$, which depends only on elapsed time. A 10% threshold is therefore "about a day and a half old", stated in a unit that hides the fact that it is a clock — which is exactly what §5e removed from `test-site-data-fresh.js` and put back through the front door. A freshly-regenerated artifact failing its own freshness check on the day it was made is the shape of a check that gets filed as *known*, and CLAUDE.md names normalisation, not invisibility, as how the docs-currency guard rotted.

The unit that answers the real question is absolute power, and `provenance.js` now prints it. Every drift note carries a POWER line beside it:

- `ci_gain` — how many percentage points narrower the 95% interval would be. Precision goes as `1/√n`, so `1.96 × 0.5 × (1/√n_dec − 1/√n_now)`.

- `max_shift` — how far the pooled point estimate could move if every missing game arrived. The pooled mean shifts by `(m/n_now)(x̄_new − x̄_old)` and `se(x̄_new) = sd/√m`, so a 2sd bound is `2 × 0.5 × √m / n_now`. Worst case, not expected case.

Measured across all thirteen drifting artifacts, the percentages span 4× and the power spans 2×:

artifact	drift	missing	CI gain	max shift (2sd)
`war.json`	**48.6%**	3,460	0.46 pts	**0.83 pts**
`policy-eval.json`	45.2%	3,220	0.41	0.80
`pory-eval.json`	35.1%	2,500	0.28	0.70
`xatu-belief.json`	31.1%	2,213	0.24	0.66
`guru-matchups.json`	26.1%	1,858	0.19	0.61
`roles-eval.json` and family	26.0%	1,854	0.19	0.60
`pory-nn.json`	15.7%	1,115	0.10	**0.47**
`counterplay.json`	12.8%	915	0.08	**0.42**

No artifact in this repository has enough missing data to move a proportion by one percentage point. `war.json` is missing *half its corpus* and can move 0.83 points. The smallest split-half noise floor this division has published is **0.43 points** (R1's cuts run 0.43–2.01; R4's three run 0.2 / 1.3 / 3.9), so `counterplay.json` is already **below the noise floor** and the rest sit inside a factor of two of it. "24% behind" and "the games it lacks cannot move it past its own noise floor" are different statements and only the second one is actionable.

And it self-extinguishes, which is the property the percentage lacks. `max_shift = √f/√n`, so the same 15.7% drift that moves 0.47 points at n=7,123 moves 0.33 at n=14,000 and 0.24 at n=28,000. The treadmill stops on its own as the corpus grows instead of being switched off by hand.

What was NOT changed, deliberately: the 10% trigger. Two reasons, and the second is the honest limit of this work.

1. Lowering the bar changes thirteen artifacts' status and that is not a call to make inside a measurement pass.
2. `max_shift` **still cannot see the thing that decided every row of §5c's hand triage** — the DISTANCE from an artifact's headline estimate to its decision boundary. `roles-eval.json` publishes 0.6935 against a coin's 0.6931; that 0.0004 margin is flippable by any new data at all, while `war.json`'s null is not flippable by 0.83 points. That margin is not computable from `n`, and the artifact is the only thing that knows it. `max_shift` is also stated for a **proportion** at sd = 0.5; a log-loss lives on another scale and the number is not directly comparable there. The next rung is a declared `decision_margin`, in the same convention as below.

A grace period measured in regenerations is rejected. It is a clock with extra steps, it cannot tell `chomp-ev.json` from `pory-nn.json`, and a fixed window is a licence for a genuinely flippable artifact to sit quiet inside it.

The `pory-eval.json` false positive is fixed on the reader side and still needs its generator. `provenance.js` now honours a declared `population_ceiling` (`j.population_ceiling`, `provenance.population_ceiling` or `corpus.population_ceiling`) and measures drift against it — the same declaration convention as `not_store_derived`, `raw_store_ok`, `gate` and `games_requested`. `pory-eval.json` is a strict subset (clean ladder games whose raw log exists AND names a winner: 5,456, not 7,123), so it can never get below ~21% against the wrong denominator. **This is a separate defect from the unit question and the answer above does not fix it by itself**: the hatch exists, and `engine/pory.py` must write the key

on its next deliberate run. **DONE 2026-08-05** — the deliberate run happened (§5d addendum) and the generator now writes `population_ceiling` with a note naming its predicate. Every artifact reading `games.ladder.raw-logs.jsonl` has the same shape.

WHAT THE NEW RULE WOULD AND WOULD NOT HAVE CAUGHT, plainly.

- `data/counters.json`**, older than the quality filter, UNSAFE for nine days — CAUGHT, and it was never a drift case.** That is the `FILTER_MT` check: the PREDICATE changed, so the artifact answers a different question, and no amount of power makes it valid. It is untouched, it is still `bad` rather than `warn`, and it still fails `--strict`. Confusing the two checks is how a volume rule gets credit for a correctness rule's catch.
- `data/chomp-ev.json` **four days behind, publishing "does not beat a coin" about a model with a directional edge — CAUGHT, and better than before.** That verdict sits *at* its boundary, so its decision margin is ≈ 0 and any `max_shift` exceeds it. The percentage caught it at 28% > 10%; the power rule catches it for the right reason.
- **An artifact recording a corpus it did not use — NOT CAUGHT, by either rule, and this file already says so on every run.** Only re-running the generator can.
- `data/slowking-playstyle.js`**, a GURU run written under the playstyle name — NOT CAUGHT by anything, and this is why the crude mtime rule in** `test-site-data-fresh.js` **was left alone.** See §9.

6. The noise floor is not a standing artifact

Split one arm in half and measure the spread. An effect smaller than that is not an effect. This gets re-derived by hand every time somebody needs it, which means it usually is not derived at all.

Two consumers now emit their own and neither is general: `rollout-r4.json` carries three split-half cuts of the H2H, and every block of `winrate-backtest.json` carries a `noise_floor` on Brier and on accuracy. That is the right shape — the floor belongs to the measurement, not to a global constant — but there is still no A/A run for the H2H, and a floor computed inside the arm being judged cannot see between-run variance.

7. All four rungs carry `n_measured` / `n_unit` — CLOSED 2026-08-04

`engine/rollout_r1_artifact.js` and `engine/rollout_r4.js` now write the pair R2 and R3 already carried, and both artifacts were regenerated from committed evidence (no rollouts): `data/rollout-r1.json` **9,201 scored positions**, `data/rollout-r4.json` **535 decisive pairs**. Choosing which of R4's four numbers goes in the common slot is the whole point of having one — the SPRT is computed on decisive pairs and nothing else, and the handoff quoting "5,248 games" (the line count of a store that writes two lines per game) is what the slot is for.

Still **not** called `n`: `data/rollout-r3.json` has published `n` as the rollout BUDGET since 2026-08-03, and one key meaning a sample size in one rung and a budget in the next is worse than no common key.

`tests/test-rollout-gates.js` derives the rung list from the filenames `engine/rollout_r*.js` write, asserts every generator emits both keys, and then permits exactly one artifact state beyond "carries them": *its generator does, awaiting a re-run.* `data/rollout-cost.json` is in that state and cannot leave it here — it is a set of TIMINGS, and R2 is re-run or it is nothing. What the test forbids is the state that actually goes wrong: missing in the artifact **and** in the generator, which is nobody having done it.

8. Naive timestamps — CLOSED 2026-08-04, and it was FIVE writers, not one

`engine/rollout_r1_join.py` was the reported case. The real answer to "is one occurrence a typo or a pattern" is that `datetime.now().isoformat(timespec="seconds")` appeared in **five** generators — `rollout_r1_join.py`, `lookahead_bound.py`, `lookahead_clock_control.py`, `nmf_rank.py`, `porygon2.py` — which makes it the house style rather than a slip. Eight committed artifacts carry one, and all eight come from exactly those five.

Correct the diagnosis, not just the bug. JavaScript does not misparse it. ECMA-262 gives the two ISO forms opposite defaults — date-TIME with no offset is read as LOCAL, date-ONLY is read as UTC:

```
new Date('2026-08-03T04:14:10')  ->  2026-08-03T08:14:10.000Z   (local, this box is UTC-4)
new Date('2026-08-03')           ->  2026-08-03T00:00:00.000Z   (UTC)
```

So the four-hour figure is the RENDERED string, not the parse, and on this machine the value round-trips. The defect is that the stamp means something different to every reader, that the two forms this project already uses side by side follow opposite rules, and that it is wrong by the reader's UTC offset the moment it is compared against a `Z` stamp — which is what every JavaScript writer here emits and what `status.js` and `provenance.js` exist to do.

`engine/isotime.py` is the single home (`utc_now()` , `utc_today()`); all five call it. `data/rollout-r1-withdrawn-join.json` is deliberately NOT regenerated — it is a withdrawn result kept so the withdrawal can be checked. `tests/test-timestamps.js` gates the WRITERS, asserts the two ISO forms really do disagree on the running machine rather than quoting a comment about it, and lists the artifacts still carrying a naive stamp without failing on them, because an artifact is fixed by re-running its generator and that pressure is how "KNOWN FAILURE" gets typed.

9. The two "stale bundles" — one was a no-op and the other was never the file it claims to be

Both were regenerated with the verify-before-trusting step first. That step is the entire finding.

`data/engine-data.js` **— BYTE-IDENTICAL. Nothing was landed.** `SHOWDOWN_PATH=… node build/rebuild_sets_from_sheets.js` reports 318 species, 195 rebuilt from real sheets, 123 left alone under 10 sheets, **materially changed 0, illegal abilities fixed 0**. Run with `--write` and diffed against a preserved copy: **identical to the byte**. The generator reproduces its own artifact and the 0.9-day staleness was mtime and nothing else.

The original mtime was then RESTORED, and that is the point of the entry. Writing the identical file moved `engine-data.js` forward and immediately turned `counterplay.json` , `scoreboard.js` and `winrate-backtest.json` — this division's own leaf-calibration artifact — into *"older than its input engine-data.js"*. Three false staleness flags manufactured by a regeneration that changed nothing. A restamp with negative information content is still a restamp; `status.js` 's refit edge is hash-based (`feature_fixture --check`) and was never at risk, but `provenance.js` 's input-ordering rule is mtime-based and was.

> ***REPAIRED AND LANDED 2026-08-04. The section below is kept as the diagnosis; this is the result.*** *Run with* ***both*** *variables set —* `TAG=playstyle MATRIX_FILE=data/playstyle-matchups.json` *— every figure predicted below reproduces to the digit: n_games 5,265 →* ***2,860****, archetypes 12 →* ***8****, mixture →* ***Rain 0.8079 / Setup 0.1657 / FakeOutBalance 0.0255****, greedy–Nash 0.0409 [−0.0001, 0.1735] →* ***0.026 [−0.0001, 0.1498]****, uniform 0.0761 →* ***0.0338****, triples 1,320 →* ***336****, cycle →* ***TailwindOffense → Sand → TrickRoom*** *(legs on 40,* ***5*** *and 140 games, still* `supported: false` *), and* ***the verdict flips*** *to "no material exploitability gap … close to transitive at this granularity." It reproduces itself on a second run, byte-identical, and is no longer byte-identical to* `data/slowking.js` *.* ***The GURU arm was re-run first and reproduces its own artifact bit-for-bit*** *— every shared key unchanged, which is what licensed trusting the playstyle run from the same code.* ***THE FIX IS THE DEFAULT, NOT THE FILE.*** `engine/slowking_preview.py` *now REFUSES to write a* `TAG` *-named artifact from the default matrix and prints the two-variable command. The rule is narrow on purpose: a TAG names a NON-default run, so TAG-set-with-default-matrix is the one combination that cannot mean anything; the ordinary GURU run and the correct playstyle run are both untouched. A relative* `MATRIX_FILE` *now resolves against the repo rather than the shell's cwd — the documented command only*

> *worked from the repository root, and a path that works from one directory and not another is how the wrong matrix gets reached for.* ***A second half of the same bug, found on the way:*** `source_matrix` *was the hardcoded literal* `"data/guru-matchups.json"`*. So even a CORRECT playstyle run would have stamped the GURU matrix as its source — the one field that could have exposed the clobber was pinned to agree with it. It is derived now, and* `tag` *is recorded beside it.* ***What moves on the page*** *(*`app/index.html:907-923` */* `web/index.html`*, WEB's files, not edited): games 5,265 → 2,860; the cycle legend from three species pairs to TailwindOffense → Sand → TrickRoom; leg edge 10% → 5%; the mixture chips from Gengar-Incineroar 66% / Charizard-Garchomp 22% / Pelipper-Archaludon 12% to* ***Rain 81% / Setup 17% / FakeOutBalance 3%****; greedy exploitability 4% → 3%.* ***And two TYPED literals in that paragraph are now wrong on both pages*** *— "these matchups rest on 49, 37 and 15 games" (really 40, 5 and 140) and "the strongest of 1,320 candidate triples" (really 336).* ***Worse than the numbers: the room's whole thesis is now contradicted by its own artifact.*** *The panel is headed "The meta looks like rock-paper-scissors" and argues "picking one single playstyle is exploitable while a mixture isn't — the reason to mix", while the artifact it renders now says mixing buys little here. That is a WEB pass, and it is a rewrite rather than a number swap.* ***Three consumers checked.*** `engine/sanity_check.py` *passes, and its §5 check "site mixture top == report top" now reads* ***Rain == Rain****; before the repair it compared two copies of the same wrong file and passed for that reason.* `tests/test-docs-current.js` *§1b likewise now reads the real playstyle artifact (0.026, CI [−0.0001, 0.1498]) where it had been reading GURU's numbers under the playstyle name — a guard built to track this artifact was tracking the other one. And* `engine/build-status.js:18` *reads* `slowking-playstyle-eval.json` *into a variable* `ex` *that* ***nothing in the file ever uses****; it is a consumer in name only.*

`data/slowking-playstyle.js` **— STOP. It is not stale; it is the wrong file, and has been since 2026-08-03 15:15.**

`engine/slowking_preview.py` takes its OUTPUT NAME from `TAG` and its MATRIX from `MATRIX_FILE`, which **defaults to** `data/guru-matchups.json`. Run with `TAG=playstyle` and `MATRIX_FILE` unset it writes a GURU result under the playstyle name. Measured:

- `data/slowking-playstyle.js` has a payload **byte-identical** to `data/slowking.js`;
- `data/slowking-playstyle-eval.json` is a **byte-identical file** to `data/slowking-eval.json`;
- both read 5,265 games / 12 species-pair archetypes / 1,320 candidate triples — GURU's shape. `data/playstyle-matchups.json` holds **2,860 games over 8 playstyles**.

Regenerating it correctly moves published figures, so it was **restored and not landed**: n_games 5,265 → **2,860**, archetypes 12 → **8**, mixture Gengar-Incineroar 0.66 / Charizard-Garchomp 0.22 / Pelipper-Archaludon 0.12 → **Rain 0.81 / Setup 0.17 / FakeOutBalance 0.03**, greedy–Nash 0.0409 [−0.0001, 0.1735] → **0.026 [−0.0001, 0.1498]**, uniform 0.0761 → **0.0338**, cycle Charizard-Garchomp→Kingambit-Garchomp→Incineroar-Whimsicott → **TailwindOffense→Sand→TrickRoom**, triples searched 1,320 → **336**, and the verdict string flips from *"substantially less exploitable… the meta is non-transitive here (rock-paper-scissors)"* to *"no material exploitability gap between Nash and greedy — this meta is close to transitive at this granularity."*

The corrected numbers are the ones `docs/MODELS.md` **already publishes** in the MACHAMP entry — 336 triples, a leg on 5 games, 0.026, [−0.0001, 0.1498] — to the digit. So the docs are right and the artifact is wrong, which is the rare direction, and the 2026-08-02 withdrawal of the SLOWKING cycle still rests on measured evidence. `engine/build-status.js:18` and `engine/sanity_check.py:32` both read the clobbered file, and `app/index.html:907-923` renders its mixture and cycle legs. The repair is one command with **both** variables set; it is a WEB pass, not a refresh. The generator should refuse to write a `TAG`-named artifact from the default matrix.

Two things this costs the checkers, and both are recorded rather than fixed here.

- `provenance.js` reports `slowking-playstyle.js` as `ok`, correctly by its own rules — it is co-generated with `slowking.js`, so the ordering carries no information. Provenance sees ordering and declared counts; it cannot see that a file's CONTENT came from the wrong input. This is why the crude mtime bundle rule in `tests/test-site-data-fresh.js` was **left alone** despite §5f: delegating it to drift today would have marked this file clean.
- `tests/test-site-data-fresh.js` printed the repair as `node engine/slowking_preview.py` — the wrong interpreter, in the STALE table only; the `--list` path already derived it from the extension. Fixed. The command it names is still incomplete, which the test's own comments already admit, and running it with `TAG` set and `MATRIX_FILE` unset is a plausible route to the clobber that is on disk.

10. `train_value.py` was discarding a fifth of the corpus — PRIORITIES #13, FIXED 2026-08-04

`idn()` normalises punctuation and nothing else, so the event stream's `charizardmegay` never matched the bring list's `charizard`, `side_of` returned None, and the event was thrown away without a word. **Measured on 4,000 clean games before the fix: 21.7% of faints, 22.7% of damaging events, 20.8% of all damage, at least one discard in 96.5% of games, and 97.6% of discarded targets are megas.** The visible symptom is the one to remember: **88.9% of clean games ENDED with both sides still holding bodies.** The value net was learning from trajectories in which almost nobody ever loses their team.

The fix is a verb on the one resolver, not a fourth copy of it. `engine/mc_key.js` gained `mcKey.base` (which body is this) and `mcKey.bases` (the whole map, for a caller that cannot call into JavaScript per name), reading the `base` field the generator already wrote from the dex into `MC.mons`. Three properties were deliberate:

- **It is not a string strip.** `re.sub(r'mega[xy]?$','',s)` is the obvious three lines and it is wrong — `mc_key.js` already records that the identical JavaScript strip answered Victreebel for Victreebel-Mega. The table carries the answer; this reads it.
- **It returns a flat BODY id, not a table key.** `MC.mons` holds `floette-mega` with `base: "floette"` and holds no `floette` row, so a version resolving the base back through the table returned null for exactly the 1,613 events this exists to rescue, one layer down. Being in our damage table is a fact about our table; the body is a fact about the game.
- **It touches no dex, so it cannot need** `SHOWDOWN_PATH` — the crash-instead-of-fail mode PRIORITIES #40 records for two other ratchets. `train_value.py` shells it once per run and **fails loudly** if it cannot, rather than reverting to the behaviour above.

After: 22.7% → 1.7% of damaging events dropped, 21.7% → 1.5% of faints, and games ending with both sides intact 88.9% → 26.3%.

What it moved, and the honest size of it. Paired on identical held-out states — 1,445 games, 10,120 states, both arms fitted on the same split:

	before	after	paired difference
log-loss	0.6634	0.6520	**−0.0114, 95% CI [−0.0183, −0.0041]** (bootstrap clustered by game)
accuracy	59.72%	61.47%	**+1.75 pts, 95% CI [0.50, 2.94]**

The mechanism is legible in the weights: `hpDiff` moved **0.169 → 0.377**, because a fifth of all damage had never been applied and the feature was attenuated toward zero. The shipped artifact (ladder + self-play, as `main()` defaults) moved `test_logloss` **0.6638 → 0.6536**.

Both intervals clear zero and the effect is still inside the noise floor for an unpaired comparison. Twenty split-half cuts of the fixed arm alone spread by a **median 1.87 accuracy points** (range 0.30–5.37) against a 1.75-point effect. The pairing is what buys the resolution; two runs on different samples could not tell these value nets apart. And the ceiling is unchanged — 61.4% sits below the **66.92%** in-sample ceiling for this feature class and below the live leaf's **67.97%**. **This is a correctness fix, not a capability change, and it was worth making on the first ground alone.**

Residual, measured rather than assumed: 1.7% still drops, and 1,613 of the 1,625 are one species. `MC.mons` carries `floette-mega` → `floette`, the store's bring lists hold `floetteeternal`, and no `floette` row exists — the chain does not close. The rest are in-battle formes the mega table does not cover (`mimikyubusted` 274, `morpekohangry` 48, `castformsnowy` / `castformrainy` 11). **Filed to ENGINE, not patched here:** closing them means reaching for the Showdown dex, which would make `train_value.py` produce different numbers depending on whether `SHOWDOWN_PATH` is set. That is *fitting environment and playing environment must match* in a new place, and it is not worth 1.5% of events.

11. THE THIRD COPY OF THE WEATHER MAP IS IN `board.js`, IT IS WRONG, AND THE FIXTURE CANNOT SEE IT

> ***LANDED 2026-08-04, with the two fixture boards it needed, and refitted. The diagnosis below is kept because it is the reason the fixture grew; the result is here.*** `engine/board.js` *no longer keeps a weather map. Both reads —* `dmgFractions` *and the* `punishExposure` *call in* `featuresFor` *— go through a* `weatherKind(board, D)` *helper that calls the damage engine's exported* `weatherId`*, the same consolidation ENGINE made in* `tag_dex.js`*. A damage engine that cannot answer is counted in* `dmgFailures.weatherUntranslated` *rather than defaulted to clear skies; measured over 234,873 candidate vectors it is* ***0****, on both the node and the browser export path (* `tests/test-board-browser.js` *: 58 of 58 features agree to 6 dp).* ***THE PRE-LANDING MEASUREMENT REPRODUCES ON THE CURRENT ENGINE TO THE ROW.*** *Re-run before relying on it, because the engine had moved underneath it:* `fit_policy.decisionsFor` *over the first 1,200 open-sheet corpus games,* ***32,054 decisions / 234,873 candidate vectors****, one process holding both builds of* `board.js` *: | | measured 2026-08-04 (pre) | re-measured after the ENGINE band | |---|---|---| | candidate vectors that move | 1,768 (0.75%) |* ***1,768 (0.75%)*** *| | | decisions that move | 892 of 32,054 (2.78%) |* ***892 (2.78%)*** *| | | columns that move | 14 of 58 |* ***14 of 58*** *| | | games containing a moved vector | not measured |* ***238 of 1,200 (19.83%)*** *| The 19.83% is the new number and it is the one that matches the census: 18.5% of games carry sand or snow at some turn.* ***THE FIXTURE NOW SEES IT — 0 columns before, 10 after.*** `sand-is-up` *and* `snow-is-up` *join* `SCENARIOS`*. Tyranitar takes special hits under sand (the Rock special-defence 1.5x) and Ninetales-Alola and Weavile take physical hits under snow (the Ice defence 1.5x); Hippowdon and Ninetales-Alola both carry Weather Ball, whose TYPE resolves off the same field, so a translation that mapped one weather and not the other cannot pass both. A Rock body and an Ice body sit on each bench, because the switch family prices a body that is not on the field and would otherwise be untouched — that one choice took the detection from 6 columns to 10. Scored against the pre-fix map:* `koTarget`*,* `dmgFrac`*,* `killIsRoll`*,* `killsThreat`*,* `koFirst`*,* `switchSurvives1`*,* `switchKOFast`*,* `switchDiesFirst`*,* `benchRisk`*, and the joint* `partnerCoversMe`*.* ***State the limit rather than the win: the fixture catches 10 of the 14 columns the corpus moves.*** `protectThreatened`*,* `diesBeforeMoving`*,* `screenValue` *and* `switchKOSlow` *move on corpus boards and not on these ten. A fixture is evidence for the restamp rule, never proof of it, and this is the measured size of the gap rather than a caveat in prose. Coverage did not regress: 40 slots, 324 candidates, 1,309 pairs, and* ***0 features that never fire****. Two properties of the landing worth keeping.* `weatherId` *still does not know* `desolateland` */* `primordialsea` *; on 339,483 corpus turn-boards that costs nothing and adding them is ENGINE's call on*

> `SD2WEATHER`*, not a fourth table here. And adding scenarios re-stamps every hash, so it went in the same pass as the refit — a fixture change and a restamp cannot be separated.*

This is a refit trigger. It is measured, it is NOT landed, and the gate for landing it is the P0/P1 band, which is not met.

`engine/board.js:1190` carries `WEATHER_KIND`, a third private copy of the Showdown-weather → engine-weather translation that `medicham2-browser.js` owns as `SD2WEATHER` / `weatherId`. It is read at two sites — `dmgFractions` (`:1247`, every damage-derived MAG feature) and the `punishExposure` call in `featuresFor` (`:2937`, `clickCost`). What it holds:

```
{ sunnyday: 'sun', desolateland: 'sun', raindance: 'rain', primordialsea: 'rain' }
```

It maps the two weathers this format cannot produce and misses the two it does. `desolateland` and `primordialsea` are primal weather: **0 occurrences in 339,483 corpus turn-boards**. `sandstorm` and `snowscape` are not in the table at all, so `WEATHER_KIND[board.weather]` is `undefined`, `|| ''` makes it clear skies, and **every damage feature under sand or snow has been computed in no weather**. The engine reads `field.weather === 'sand'` for the Rock special-defence 1.5×, `=== 'snow'` for the Ice defence 1.5×, and Weather Ball's type comes off the same field.

Exposure, re-measured rather than inherited — a census of `board.weather` at every turn-board across `games.ladder` + `games.bo3` + `games.ots` (52,441 games):

weather at a turn-board	share	`WEATHER_KIND` gives
clear	64.15%	`''` correct
`sunnyday`	14.90%	`'sun'` correct
`raindance`	10.23%	`'rain'` correct
`snowscape`	**5.43%**	`''` **— WRONG**
`sandstorm`	**5.29%**	`''` **— WRONG**

10.72% of turn-boards, and 18.5% of games contain at least one.

THE FIXTURE PASSES ANYWAY, AND THAT IS THE FINDING. The patch was applied, measured and reverted. `engine/feature_fixture.js --check` returns `feature semantics OK` on both `policy-weights.json` and `policy-weights-joint.json` **before and after** — because the fixture's only two weather scenarios are `RainDance` and `SunnyDay`, the two `WEATHER_KIND` already gets right. All 58 columns are hash-identical while the feature function has moved.

What actually moves, measured on the fit's own rows — `fit_policy.decisionsFor` over the first 1,200 open-sheet corpus games, **32,054 decisions / 234,873 candidate vectors**:

candidate vectors that move	**1,768 (0.75%)**
decisions that move	**892 of 32,054 (2.78%)**
columns that move	**14 of 58**

`dmgFrac` (1,182), `koTarget` (238, max |Δ| 0.93), `killIsRoll`, `killsThreat`, `diesBeforeMoving`, `benchRisk`, `koFirst`, `protectThreatened`, `switchKOFast`, `switchKOSlow`, `screenValue`, `switchDiesFirst`, `switchSurvives1`, `switchSurvives2` — several flipping a full 0→1.

So `feature_fixture --check` is necessary and not sufficient, and `status.js`'s `refit edge: CLEAN` inherits that limit. The hash covers the boards the fixture builds; the feature FUNCTION lives over every board the corpus contains. This division's own rule — *a restamp is only valid if the feature FUNCTION is*

unchanged — is the binding one, and a green fixture is evidence for it, not proof of it. **The fixture needs a sand board and a snow board.** Adding them is itself a fixture change that re-stamps every hash, so it belongs in the same pass as the refit, not before it.

The patch is small and is recorded here rather than left on disk: replace both reads with a `weatherKind(board, D)` helper that calls the damage engine's exported `weatherId`. Do not write a fourth map. Note that `weatherId` does not know `desolateland` / `primordialsea`; on the measured corpus that costs nothing, and adding them is ENGINE's call on `SD2WEATHER`, not a second table here.

11a. `position_features.js` — the same defect at the second boundary. LANDED, no refit owed.

`engine/position_features.js:292` built its field object as `B.norm(board.weather || '')` — the board's *move name* — and handed it to `M.dmgRange` and `M.effSpeed`, which compare against the engine's words. Truthy and meaningless, exactly as at the leaf boundary. Now `M.weatherId(...)`.

No refit is owed and that was checked, not assumed: nothing in the repository fits, reads or renders these columns. The only callers of `positionFeatures` are four tests, and no artifact contains any of its feature names.

Exposure re-measured on the boards this module is actually asked about — the `joint_rows.build` walk, 400 open-sheet games, **3,202 mid-game boards / 6,404 scored positions**. **49.94% carry a weather**, higher than the 35.85% turn-board figure because weather accumulates as a game runs.

- **1,962 of 6,404 positions moved (30.64%)**: sun 73.6%, rain 66.7%, sand 49.5%, snow 29.5%.
- 7 of 16 columns: `raceEdge` (29.67%, max |Δ| 0.42), `killFirstEdge`, `iKillNext`, `theyKillNext`, `benchAnswersDiff`, `speedEdge`, `pinnedDiff`.
- **0 of 3,206 clear-weather positions moved** — the control that says this is the weather.

`:296`'s terrain now goes through `M.terrainId` as well. That one is a **confirmed no-op**: 16 of the 3,202 boards carry a terrain, 11 of them under clear weather, and all 22 positions scored on those are bit-identical. Every downstream reader already calls `terrainId` itself. It is translated here so the field object leaves in one vocabulary rather than two, which is the condition that let the weather half go unnoticed. The four probed keys are a list of BOARD KEYS, not a translation table — the same justification `rollout_leaf.terrainOnBoard` records.

ENGINE's *"0 of 400 boards"* for this file was terrain-only and is reproduced (0.50% here). It was being read as though it covered the weather too, and the weather number is 49.94%.

11b. `git checkout -- <file>` INVALIDATES EVERY CONTENT STAMP ON THIS MACHINE

Found by doing it. `core.autocrlf=true`, the committed blobs are LF, and the worktree files are LF — so a checkout REWRITES them as CRLF. The bytes change, nothing in git notices (`git diff` is empty), and `sha256(worktree)` moves. `winrate-backtest.json`'s `measured_against` and `run_stamp.js`'s `source_digests` both hash worktree bytes, so `engine/board.js` immediately began printing `PRE-CHANGE — measured against a different build of: … engine/board.js` after a revert that changed no code. Converted back to LF; the digest returns to `bcf2dab9dc6f`, which is the stamp exactly, and board.js drops out of the PRE-CHANGE list.

This is the mirror of the warning already in *Reading a stamp — never compare* `source_digests` *to* `git.blobs`*, they differ by line-ending translation*. The new half is that an ordinary git operation can move one of them. **After any** `git checkout --` **or** `git stash pop` **on this box, check** `node engine/status.js` **before believing a PRE-CHANGE line.**

12. `tests/test-no-silent-failure.js` — the 32 MEASURE entries, and what two of them were hiding

Worked through without `--update`, which would have laundered the SEARCH, OPS and WEB entries in the same command. **NEW since the baseline: 52 → 20.** Every MEASURE-owned entry is cleared; the 20 that remain are 13 SEARCH (`miltank.js`, `rollout_leaf.js`, `rollout_r1.js`), 3 OPS (`mag_bot.js`) and 4 in `tests/test-{site-data-fresh,stadium-roster,web-parses}.js`.

Two were hiding something real.

`engine/run_stamp.js:92` **recorded "the tree was clean" whenever git refused to answer.**

```
const porcelain = git(['status', '--porcelain', '--'].concat(watched)) || '';
```

`git()` returns `null` when the command throws — an index lock, an interrupted rebase (CLAUDE.md documents this repository reaching one 43 commits into a 45-commit replay), git not on `PATH`. An EMPTY porcelain is git's way of saying CLEAN. `|| ''` collapsed the two, so a stamp written while git was unavailable published `dirty: false` **beside a commit id that described nothing on disk** — and this module's own header says *"a clean commit id over a dirty tree is a lie of exactly the kind this module exists to stop"*, while `docs/MEASURE.md`'s *Reading a stamp* tells every reader to trust the commit when `dirty` is false. `rev-parse HEAD` was already guarded; `status --porcelain` was not. Now a third state: `dirty: null` with `git_errors`, and `status.js` renders it as **DIRTINESS UNKNOWN** rather than as the clean case.

`engine/backtest_winrate.js:71` + `engine/status.js:176` **composed into a false clean bill.** `stampOf` returns `{mtime: null, error}` with no `sha256_12` when a source cannot be read. `status.js` then did `if (!st || !st.sha256_12) continue;` and, finding nothing in `moved`, printed **"CURRENT — every engine source the leaf reads still hashes to what it was measured against"**. With every stamp failed, that sentence was printed over **zero comparisons**. Two silent catches, neither wrong on its own, producing a clean provenance line on this division's headline number. The count is now stated (`all N engine sources`), unstamped sources are named, `NOT DERIVED` is printed when N is zero, and a source that has been DELETED is reported as gone rather than as "a different build of".

The rest were latent rather than active, and the honest answer is that they were hiding nothing today — which is a measurement, not an absence of one.

- `engine/rollout_r4.js:279` — the split-half scan that produces the NOISE FLOOR discarded a torn line in silence, while `countLine`, reading the same file for the header counts, keeps `torn` and publishes it. A row lost here shrinks an arm and moves the spread, and the spread is the entire output. Counted; the split now refuses to report if anything was lost. **Measured on `games.r4-decided.jsonl`: 0 torn, 0 bad-seed, across all three cuts.** Before the counter existed, 0 and 500 looked identical from there. A second hole found while adding it: a non-numeric seed put every such record on side B, because `NaN % 2 !== 0` — now rejected rather than piled up. Incidental: the `seed hash parity` cut splits **1,382 / 1,242**, an 11% imbalance, and it is the cut that produced the largest of the three spreads (3.9 pts) quoted as this run's noise-floor range.
- `tests/test-timestamps.js:49/54` — a directory that would not list and a file that would not read were both skipped with `continue`, so *"no Python generator writes a naive datetime"* was also the answer when **zero generators had been looked at**. That is CLAUDE.md's *a capability that cannot prove it ran* inside a guard written for a different failure. Now asserts a floor on files scanned (39 in `engine/`, 1 in `build/`; `tools/` has no `.py` and `scripts/` does not exist) and fails on anything skipped unread.
- `tests/test-timestamps.js:92` — an artifact that would not parse was silently excluded from the published *"N artifacts still carry a naive stamp"* list, so the files most likely to be broken were the ones the survey

could not see. Now counted and named: **0 of 108** `data/*.json` **fail to parse**, so the list of 8 is complete — a statement that could not previously be made at all.

- `tests/test-web-status.js:181` — `catch { return false }` on the freshness filter means "not newer than the board", the same answer a perfectly fresh artifact gets. A source that had been **deleted or renamed** read as up to date, in the test whose job is that every rendered figure traces to an artifact. Missing is now its own failure. None are missing today.
- `tests/test-rollout-gates.js:81` and `engine/rollout_r1_artifact.js:228` — both collapsed "no such file" into "will not parse". The first then granted a CORRUPT gate artifact the one tolerated state (*"awaiting a re-run"*); the second made a broken sidecar indistinguishable from a run nobody ever stamped, which is the exact distinction §4 and §7 exist to preserve.
- `engine/status.js` (9), `engine/rollout_explore_sweep.js` (3), `engine/rollout_r1_artifact.js` (3 more), `engine/run_stamp.js:60`, `tests/test-web-status.js:58/112`, `tests/test-guru-derived.js:56` — each conflated *absent* with *unreadable*. `status.js` now carries a `DIAGNOSTICS` block, printed on screen and deliberately **outside** the section bodies so `--write` never stamps a transient into a ledger.

Two defects in the ratchet itself, filed not fixed:

- `--update` **is all-or-nothing, so the tool's own guidance cannot be followed.** It says *"if a silent fallback is genuinely right here, say why in the code and re-baseline with* `--update` *so the exception is deliberate and visible"* — but `--update` re-baselines every silent catch in the repo, including other divisions'. There is no way to bless ONE. It needs a per-entry allow with a reason string, in the shape `build_guru_js.js`'s `DELIBERATELY_UNUSED` already uses.
- `isSilent` **cannot see a recorder it does not recognise by name.** `error: e.message` inside a returned object is a colon, not an `=`, so an artifact that carries its own reason still reads as silent; and a named helper that pushes onto a list looks like nothing from inside the catch body. Four surviving entries are this. Widening the regex would launder real ones, so the code was moved to the documented convention instead — `status.js`'s recorder is named `logUnreadable`, and two locals are named `errWhy` / `errBundle`.

`--all` was added to the ratchet: the 25-line cap is right for a gate, but *"… and 27 more"* is how the tail of a list stops being anybody's job.

13. THE REFIT RAN — and it moved nothing measurable. That is the result, not a preamble to one.

`node --max-old-space-size=4096 engine/fit_policy.js` then `engine/fit_joint.js`, on the weather landing in §11. **8,759 clean open-sheet games, 229,339 usable decisions** (up from 8,414 / 220,613), 183,679 train / 45,660 held out, lambda selected on held-out likelihood at 0. Both weight files carry a fresh `featureHashes` over the 10-scenario fixture, and `feature_fixture --check` exits 0 on both.

The before/after in the artifacts is not the comparison to read, because the two fits have different corpora and different held-out sets — 44,033 decisions against 45,660. Quoting `heldOut.boardAware` 32.269% against 32.271% would be comparing two samples, which is the confound this division keeps finding in other people's work. The comparison that means something scores the SAME held-out decisions three ways, with the split reproduced exactly (`hash(game) % 5 === 0`):

arm	what it is	logL/decision	top-1
A	old weights + old features	−1.732548	32.204%
B	old weights + NEW features	−1.732200	32.252%
C	NEW weights + NEW features — what ships	−1.732276	32.178%

1,772 held-out games, 46,162 decisions, paired per decision, bootstrapped over 10,000 resamples of GAMES (decisions inside one game share a team and a board):

paired difference	logL/decision	top-1 points
B − A the weather fix alone, weights frozen	**+0.000348** [0.000075, 0.000623]	**+0.048** [0.009, 0.093]
C − B the refit, given the fixed features	−0.000076 [−0.000172, +0.000021]	−0.074 [−0.155, +0.004]
C − A everything, against what shipped	+0.000273 [−0.000010, +0.000556]	−0.026 [−0.117, +0.064]

Read plainly, three statements:

1. **The correctness fix is detectable and it is a quarter of the noise floor.** B − A clears zero on both metrics, and twenty split-half cuts of arm C alone spread by a **median 0.192 top-1 points** (range 0.005–0.770) against an effect of 0.048. The pairing is the whole reason it resolves at all; two runs on different samples could not tell these builds apart. Same shape as §10's value net, one order of magnitude smaller.
2. **Refitting the weights on the corrected features bought nothing.** C − B contains zero on both metrics and its point estimate is NEGATIVE. Only **1 of 58 weights moved more than 2 SE** (`dmgFrac` +0.0592, 2.45 SE — the column the fix touches most), 6 moved more than 1 SE, and the L2 norm of the whole weight change is **0.216**. The joint file moved less: largest term `terrainSetupHelpsPartner` +0.102, L2 **0.128**.
3. **The combined change is indistinguishable from zero on held-out human-click prediction.** C − A contains zero on both. The fix was worth making because it is a fact about the game that the feature function was getting wrong on 10.72% of turn-boards — that is the whole justification and it does not need a metric to support it.

What this does NOT say. Top-1 agreement with a human click is not a win rate. Nothing here measures whether MILTANK plays better; that is an H2H and it belongs to SEARCH. And the leaf is a separate model from MAG — §1's finding that the leaf is worse than a coin is untouched by any of this.

13a. THE FIT IS NOT RUN IN THE ENVIRONMENT THE BOT PLAYS IN, and it is 20x the weather defect

This is the headline of the refit, not a caveat on it. CLAUDE.md's rule is *fitting environment and playing environment must match*, recorded because MAG's weights were once fitted with the sheet visible while the bot played without it. **The mismatch is back, pointing the other way, and nothing was watching for that direction.**

```
engine/fit_policy.js:376   board.setSheet(side, m.species, { nature, item })
engine/magnemite.js:522    this.board.setSheet(m[1], sp, { nature, item, ability, moves })
```

`Board.switchIn` copies all four onto the active mon, and `dmgMon`, `effAbility` and `movePriority` read them. So the LIVE player sees a sharper board than the fit ever did: the fit prices every opponent on the dataset's representative moveset and on Smogon's per-species ability odds, while an open-sheet game hands the player the declared four moves and the declared ability.

The sheets carry it. 14,400 sheet entries over 1,200 corpus games: 100.0% declare an ability, 100.0% declare four moves. This is not information the fit lacks — it is information the fit is handed and drops.

Measured the same way §11 was, one process holding both builds of `fit_policy`, identical games and identical decisions:

	weather defect (§11)	the sheet-channel gap
candidate vectors that move	1,768 (0.75%)	**37,460 (15.95%)**
decisions that move	892 (2.78%)	**16,177 of 32,054 (50.47%)**
columns that move	14 of 58	**20 of 58**
games containing a moved vector	238 (19.83%)	**1,197 of 1,200 (99.75%)**

`switchDiesFirst` (10,013), `diesBeforeMoving` (9,466), `switchSurvives1` (6,632), `dmgFrac` (4,188), `killsThreat`, `switchKOSlow`, `switchSurvives2`, `switchKOFast`, `protectThreatened`, `priority` (1,958 — the declared ability reaching `movePriority`), `screenValue`, `movesFirst`, `koTarget`, `benchRisk`, `killIsRoll`, `koFirst`, `clickCost`, `passTurnAccrues`, `switchFaster`, `deadNoLastMove`. The choice set is unchanged — the row counts match game for game — so this is purely what the board KNOWS, not what it offers.

It is NOT landed and no second refit was started. Landing it is a one-line change to `fit_policy.js:376` plus a full refit of both files, and it would invalidate the refit reported above on the day it was published. It also needs a question answered first that this measurement does not answer: the fit's decisions come from games where the sheet was public, but MAG must also play the ~half of ladder games where the opponent declines OTS, and a model fitted on four channels degrades differently from one fitted on two when a channel goes missing. That is the Focus Sash lesson — *replacing a hedge with a certainty is only an improvement if you also track what invalidates the certainty* — and it is a decision, not a refresh.

Filed with its size stated, which is the part that was missing: **half of every decision the fit trains on is priced against a board the player does not see.**

13b. THE JOINT LAYER IS REFITTED ON FOUR CHANNELS, AND THE CHANNELS ARE WORTH A LIKELIHOOD GAIN, NOT AN ACCURACY GAIN — 2026-08-05

The other half of §13a's debt, run under Will's go. Three results, each with its instrument named.

The joint refit (`engine/fit_joint.js`, four-channel `joint_rows.js`): 8,856 clean open-sheet games, 101,459 joint turns → 95,886 usable, 77,975 train / 17,911 held out by game, lambda 0 on held-out. The artifact now carries a `fitEnvironment` block and it says `matches_player: true` by measurement: the declared ability and moves reached the board on **202,343 of 202,918 scored slots (99.7%)** and **395,130 of 396,288 live foe actives (99.7%)**. Held out, predicting the pair: separate decisions logL −3.3294 / top-1 10.3%, refit with joint terms zeroed −3.3199 / 9.8%, with the joint terms **−3.2308 / 12.2%**. The chosen pair fell outside the top-6 menu on 11.1% of kept turns. `feature_fixture --check` passes on the new artifact. No before/after against the presheet joint vector is quoted because none was measured — the presheet run published no held-out table to its artifact, and comparing two logs would be comparing two samples.

What the two extra channels are worth at the marginal layer (`engine/sheet_channel_value.js`, arm A = release `d3d04b669e18`'s two-channel incumbent, 44,982 paired held-out decisions over 1,789 games, 10,000 game-bootstrap resamples). The first run **VOIDED itself** — ENGINE saved `engine/medicham2-browser.js` mid-run and the instrument recorded `void: true` — and the second run is clean, with every deterministic figure identical between the two:

paired difference	logL/decision	top-1 points
B – A the information alone, weights frozen	**+0.002853** [0.001611, 0.004072]	+0.009 [−0.140, +0.157]
C – B the refit, given the information	**+0.002234** [0.001638, 0.002831]	**+0.165** [0.029, 0.299]
C – A everything vs what shipped	**+0.005087** [0.003854, 0.006331]	+0.173 [−0.011, +0.360]

Split-half noise floor of the shipping arm, 20 cuts: **median 0.331 top-1 points** (range 0.012–1.385; the earlier refit's floor was 0.192 on a smaller paired set). Read plainly: **the sheet channels buy a real likelihood gain — every logL interval clears zero — and no demonstrable top-1 gain.** The one top-1 interval that clears zero (C – B, +0.165) is half its own noise floor and resolves only because the comparison

is paired; the total effect against what shipped contains zero. Same shape as §13: correctness and information first, metric second, and the honest metric statement is "better calibrated per decision, not measurably more often right on the argmax."

The degradation budget did not move, and cannot move by this lever. `fit_joint.turnsDropped` is **5.4929% (5,573 of 101,459) against a 5.49% ceiling — still red**. The dropped turns are unmatched clicks (5,555) and ambiguous mirrors (18); the chosen pair is kept regardless of its rank, so the four-channel w1 changes which ALTERNATIVES are on the menu, never which turns are kept. The rate crept from 5.4811% when the ceiling was ratcheted (86,242 turns) because the newly ingested games unmatch at 5.56%. The ceiling is untouched; the call on it is Will's.

PORY family regenerated, and `tests/test-site-data-fresh.js` **is GREEN (7/7)** — §5d addendum has the pory-eval numbers; `data/nmf-roles.json` moved 13,258 → 14,808 team-docs, 258 → 263 moves, recon-err 0.8346 → 0.8356, rank 10 unchanged, and both site bundles (`data/pory.js` , `data/nmf.js`) were rewritten by their own generators in the same runs. `data/pory-nn.json` retrained at 6,289 games / 106,782 states (was 6,008 / 102,296): every arm ordering holds — N6 0.6201, NR 0.6132 and LR 0.6064 all still beat the two-feature bar at 0.6229, nonlinearity is still worth ~0.003 and representation ~0.016 — and no living doc quotes these figures (§5c). The first retrain immediately re-red the drift check at 15.1%, the §5f false-denominator class to the letter: its population is the raw-logs subset. Both `engine/pory.py` and `engine/pory_nn.py` now declare `population_ceiling` (the artifact's own generator wrote it; the retrain reproduced every arm to the digit under its seeds), which is what turned the check green rather than a threshold being moved.

Doc and site locations quoting superseded PORY figures, for the propagation pass (grep-verified, historical HANDOFF files excluded as history): `docs/ABRA-whitepaper.md:113` (0.6298 [0.6125, 0.6456]), `docs/SUMMARY.md:77` (same + 0.655), `docs/MODELS.md:358/360/364` (0.9943/1.4080, 0.629799/0.629778, +0.000021 [−0.000013, +0.000056], 925, 4,623), and WEB's `web/stadium.html:506,:728` + `app/stadium.html:506,:728` (the `kadabra` data object and its prose), which render every one of those numbers and are flagged, not edited.

14. THE OUTPLAYED TURNS — 1,336 recorded actions were not clicks, and the model was learning from every one of them. LANDED 2026-08-05.

`docs/CLICK-CENSORING-FIX.md` is the spec, ordered by Will: *"i def dont like just tossing turns because they got outplayed with a move liek encore or follow me, these are the basis of vgc man."* Four artifacts: `data/click-censoring-census.json` , `data/partial-label-em.json` , `data/censoring-value.json` , and the refitted `data/policy-weights{,-joint}.json` .

LEAD WITH THE RESULT, INCLUDING THE HALF THAT DID NOT WORK. Two headline classes were measured and only one moved:

held-out class	what changed, after − before	verdict
COERCED (n=284) — Encore replaced the click, or the mon was dragged in	P(model picks the action no human chose) **−0.002614, 95% CI [−0.003663, −0.001637]**	the poison is unlearned, and it is the only headline that moved
PARTIAL (n=643) — a redirector soaked the attack	mass on the true candidate set **+0.000109 [−0.000286, +0.000491]**; logL on the set **−0.002646 [−0.004037, −0.001377]**	**no improvement. The likelihood is very slightly WORSE.**
CONTROL, CLEAN (n=46,268)	logL **+0.000447 [0.000142, 0.000743]**; top-1 **+0.002 [−0.094, 0.098]**	as the spec predicted: no top-1 change

47,195 paired held-out decisions over 1,809 games, 10,000 bootstrap resamples **clustered by game**, `engine/censoring_value.js` . The spec disclaims a corpus-wide top-1 improvement in advance and none is claimed here; the CLEAN row is a control.

> ***RE-MEASURED 2026-08-05 on the current engine and a corpus grown to 9,230 games — every figure in this section reproduces inside its interval.*** *The table above is the 3.42.0 run and is kept as published; the artifact on disk now holds* ***n=48,274 over 1,851 held-out games****, COERCED* ***−0.002613 [−0.003650, −0.001672]****, PARTIAL mass* ***+0.000122 [−0.000261, +0.000514]****, CLEAN logL* ***+0.000485 [0.000189, 0.000777]****. §17 has the full comparison and the reason the engine move could not have touched it.*

Say the negative result plainly: Stage C bought nothing measurable, and the reason was predicted by Stage C's own validation before the refit ran. The EM harness recovers **97.4%** of a planted censoring bias when the censoring is heavy, and at the rate the corpus actually censors, the bias in weight space is **−0.0030 against a 0.2600 noise floor** — unmeasurable. The redirection correction is right in principle, and the class is 1.35% of actions with a candidate set of exactly two, so there was almost nothing to recover. Both instruments agree, which is the only reason to believe either.

Stage A — the census. 241,927 recorded human actions over 8,942 clean open-sheet games (the FIT corpus; the census artifact has since been re-run twice with the store, at 9,022 and then **9,230** games, and the shares are stable to a hundredth of a point — see §17):

class	n	share	mechanism		
CLEAN	229,555	94.886%	—		
PARTIAL	3,260	1.3475%	Follow Me / Rage Powder 3,231; Lightning Rod 29. Every candidate set is size 2		
COERCED	**1,336**	**0.5522%**	Encore's application turn 1,116; a `\	drag\	` 220 (Roar 184, Dragon Tail 33, Whirlwind 3)
dropped, not a censoring class	7,776	3.214%	unmatched 6,937, trivial 809, ambiguous 30		

The mechanism list is read from the running format, never typed — moves with `condition.onOverrideAction`, moves with `forceSwitch`, abilities with `onFoeTryMove`, items assigning `switchFlag` / `forceSwitchFlag` (**empty here**: Eject Button, Eject Pack and Red Card are all `isNonstandard: 'Past'`), plus `data/tags.json`'s `redirects` / `redirectsType`. Every set refuses to be empty, and a zero on either counter is fatal in both fitters.

THE CLASSIFIER WAS SCORED AGAINST THE PROTOCOL, NOT ASSERTED. The census has a second arm that reads `data/games.*.raw-logs.jsonl` and compares per (game, turn, slot):

class	protocol says	classifier flagged	both	recall	precision
Encore application	619	642	617	**99.68%**	**96.11%**
drag	86	86	83	**96.51%**	**96.51%**

The 25 Encore false positives are 0.01% of all actions and the asymmetry is the right way round: a false positive deletes one real click, a false negative keeps a poisoned one, and there are two of those. Most likely cause is an Encore blocked by Protect, which the extractor records no failure flag for. Stated, not chased — the classifier was frozen while the refit that depends on it ran.

Two corrections to the spec, both measured. (1) §1's first row is wrong and `engine/redirect_audit.js` said so on 2026-08-02: redirection does **not** drop the turn. The redirector is a legal candidate target, so the matcher matches it and the click enters the fit with a CONFIDENT WRONG TARGET. It is label noise, not censoring — which makes Stage C a poison fix as much as a recovery. (2) A `\|drag\|` is a third coerced class the spec does not list. `engine/durable-ingest.js:67` parses `\|switch\|`, `\|drag\|` and `\|replace\|` with one regex, and `fit_policy`'s `forcedSlot` guard only knows about faints, so every phazed arrival was fitted as a voluntary switch decision.

WILL'S FARIGIRAF CASE IS ANSWERED: PARTIAL, NOT ERASED.

```
|cant|p1a: Farigiraf|ability: Armor Tail|Aqua Jet|[of] p2b: Basculegion
```

The blocker is named first, the attempted **move** is named, and `[of]` names the **attacker**. **284 of 284 priority-block lines carry the attacker slot (100.0%)**, so the user and the move are exact and only the target is ambiguous — between the blocker and its ally, and nowhere else, because the ability blocks nothing aimed elsewhere.

It is counted and NOT recovered, and that is a judgement with a reason. Showdown emits no `\|move\|` line for a blocked attempt, so the class leaves no event and lives only in the raw logs — which cover **66.17% of the fit corpus (5,917 of 8,942 games)**, and the gap is one SOURCE, `data/games.ots.jsonl`, an external archive with no log file. Recovering these 284 clicks, and the 126 more that `\|cant\|` states outright (Taunt 59, Disable 58, Heal Block 5, Imprison 4), would add outplayed turns from two stores and none from the third. That is a corpus reweighting wearing a bug fix's clothes. Closing it means re-ingesting the ots archive with its logs — OPS work, filed.

A FOURTH THING, FOUND ON THE WAY, AND IT IS A WRONG DENOMINATOR RATHER THAN A WRONG LABEL. `engine/board.js`'s `candidates()` narrows the choice set for a **Choice item**, derived from the dex's `isChoice`, with its own comment saying why: *"that is not a scoring error, it is a WRONG DENOMINATOR. A conditional logit divides by the sum over the choice set."* It does nothing about the other family that shrinks a menu — the `onDisableMove` set. **2,280 of 139,769 logged actions (1.6313%) were taken with a menu-sealing volatile up**: Encore 1,276, Throat Chop 375, Taunt 329, Disable 239, Heal Block 94. A human left one legal move by Encore is priced as having chosen it over nine. **NOT FIXED HERE** — narrowing the menu moves every feature row and owes its own refit, and it is a different defect from the one this dispatch was for. Counted so the decision has a size.

Stage C — the estimator, shown failing on known-bad input before it was believed. `engine/em_validation.js`, 31,940 real corpus feature rows over 1,200 games with SYNTHETIC labels drawn from a known planted vector, 3 seeds, the real censoring process applied to the planted labels:

regime	rows censored	oracle	naive	EM	noise floor	verdict
amplified	20.961%	0.9978	**1.8913**	**1.0208**	0.2600	bias 0.8935 clears the floor; **EM recovers 97.4%**
observed	0.439%	0.9978	0.9948	1.0021	0.2600	bias **−0.0030 — inside its own noise floor**

Distances are $\|\hat{w} - w^*\|_2$. The noise floor is the spread of the ORACLE arm across the three seeds, so it carries no information about the contrast. The **first** amplified regime censored EVERY eligible row and EM recovered only 45% — correctly, because with every same-move row collapsed there is nothing left to identify the target features from. That is Cour et al.'s identifiability condition failing, not the estimator; the eligibility is now exogenous and the collapse label-dependent, which is what the corpus does. `engine/em_validation.js --check` re-verifies the recorded verdict AND re-hashes every source, so editing `engine/click_class.js` turns the gate red instead of leaving a stale PASS; it is registered in `tests/run-all.js`.

Stage D — what the refit moved, and the confound stated rather than buried. `data/policy-weights.json`: **8,942 games, 232,815 usable decisions of 241,927 seen** (186,494 train / 46,321 held out), lambda 0 on held-out, reweighted vector ships. $\|new - old\|_2 = 0.8030$ and **9 of 58 weights moved more than 2 SE**. The mechanism is legible in which ones:

feature	before → after	
`deadStall`	−1.3114 → −1.4763	5.44 SE
`stallIntoEncore` — *"I am about to Protect and something across from me can Encore me for it"*	**−1.0502 → −1.6281**	3.10 SE, the largest single movement

feature	before → after	
`deadSide`	−2.7606 → −3.1414	4.01 SE

That is the predicted direction. The poisoned rows were victims "choosing" their last move under an active Encore; deleting them makes clicking into an Encore threat look worse, and the Encore/stall family is exactly where the vector moved. Same shape as §10's `hpDiff` 0.169 → 0.377.

THE CONFOUND, NAMED: the two vectors differ in four ways, not one. The incumbent was fitted on 8,856 games and the new one on 8,942 — the collector never stops — so the Stage D contrast carries the coerced removal, the partial-label EM, 86 extra games and the refit itself. The weight-movement pattern above is evidence for attribution and is not proof of it. `CENSORING=off` now exists in `engine/fit_policy.js` for exactly this: it fits the OLD way on the NEW corpus, and it records `censoring: "off (CONTROL ARM — not shippable)"` in its own artifact so a control can never be mistaken for a ship. **That arm has not been run** — it is a second full refit and free RAM was 1.3 GB with the joint fit in flight. It is the next thing this section owes.

AND EVERY EFFECT HERE IS SMALLER THAN ITS OWN CLASS'S NOISE FLOOR. The COERCED contrast is 0.002614 against a split-half floor of 0.007635; the CLEAN logL gain is 0.000447 against 0.007855. They resolve only because the comparison is **paired per decision** — two runs on different samples could not tell these builds apart. This is the same statement §13 and §13b make, and it must travel with the numbers.

Stage B — the budgets are RE-DERIVED, not renumbered, and `turnsDropped` is retired. `fit_joint.turnsDropped` was `(turnsSeen - kept)/turnsSeen` and sat at 5.4929% against a 5.49% ceiling. Stages B–C change what "dropped" MEANS: coerced turns used to be inside `kept`, carrying a wrong label, and now leave the labelled set — so the old total would have gone UP while the artifact got strictly better, and a ceiling that may only tighten would have gone red for an improvement. **Raising or lowering the number would have been the wrong move in either direction.** Three counters now, each with its granularity stated in `data/degradation-budgets.json` and its ceiling ratcheted from a measured run:

counter	what it counts	denominator
`fit_policy.decisionsUnreadable` / `fit_joint.turnsUnreadable`	the click existed and could not be recovered. **A LOSS.** Successor to the old totals, and directly comparable because it is the same quantity minus a term that was misfiled as kept	human actions seen by `fit_policy` / joint turns seen by `fit_joint`
`fit_policy.coercedActions` / `fit_joint.coercedTurns`	the recorded action was **not a click** and was removed. **A CORRECTION, not a loss** — it should track the metagame's use of Encore and phazing and nothing else	same
`fit_policy.decisionsDropped` / `fit_joint.turnsDropped`	**RETIRED.** Carried in a new `superseded` block with its old ceiling intact, so the history is not deleted	—

`measured_at` also used to read *"over 120 corpus games"* on every row, which is true of the three `board.js` counters and **false of every fitter rate** — those come out of an artifact written over the whole corpus. A ceiling whose denominator is misdescribed cannot be re-derived by anyone.

What this does not say. Top-1 agreement with a human click is not a win rate; whether MILTANK plays better is an H2H and belongs to SEARCH. The COERCED class has no ground-truth label by construction, so its contrast measures a change in the MODEL, not an improvement in accuracy — it cannot be otherwise, and inventing an agreement number for it would have been the dishonest option.

15. THE TWO LEAF ARTIFACTS DO NOT CONTRADICT EACH OTHER. RESOLVED 2026-08-05, and the answer is a decomposition, not a winner.

`data/winrate-backtest.json` says the in-game leaf ranks at **50.99%** and is worse than a coin. `data/rollout-r1-explore-sweep.json` says the same leaf ranks at **69.84%** with a monotone reliability curve. The sweep flagged the conflict itself in `reading_against_the_leaf_calibration` and refused to treat "explore=1.0 spent the signal" as established while a second measurement disagreed. **It was right to refuse, and both artifacts are correct.** They score the same function on positions of very different difficulty, and the gap decomposes cleanly.

The sweep named two differences — rollout budget and horizon — and **those are the two that do not matter.** There are six, and the three that carry the gap are position, corpus and sheet, in that order.

`engine/leaf_position_contrast.js` holds five of the six fixed at a time: one leaf (`rolloutWinProb` , explore=1.0, n=40, horizon 20), one frozen release (**6b0e4117d964**), the same seeds, and both accuracy definitions on every arm. `data/leaf-position-contrast.json` , with `data/leaf-position-contrast-rows-6b0e4117d964.jsonl` beside it so any cut can be re-derived without re-running the rollouts.

arm	corpus	position	sheet	n	maj. class	accuracy	Brier vs coin (paired)	ECE	MCE	curve slope
D = the sweep	open-sheet bo3	mid-game	yes	9,201 pos / 2,500 g	52.5%	**69.83%** [68.6, 71.1]	**−0.0440** [−0.0513, −0.0360]	0.0925	0.162	**0.703**
C	open-sheet bo3	mid-game	no	9,201	52.5%	68.73% [67.5, 69.9]	−0.0401 [−0.0470, −0.0329]	0.0939	0.146	0.693
B	open-sheet bo3	**turn 0**	yes	2,500	52.4%	58.20% [56.4, 60.2]	+0.0101 [0.0020, 0.0182]	0.1120	0.332	0.402
A	open-sheet bo3	**turn 0**	no	2,500	52.4%	55.92% [54.0, 57.8]	+0.0166 [0.0088, 0.0243]	0.1284	0.351	0.331
E = the backtest	closed ladder	**turn 0**	no	1,499	52.2%	**51.17%** [48.6, 53.6]	**+0.0456** [0.0344, 0.0567]	0.1793	0.458	**0.068**

Intervals are game-clustered bootstraps, because 9,201 mid-game positions come from 2,500 games and an unclustered interval on them is too narrow by about √3.7.

The decomposition telescopes exactly. 69.83 − 51.17 = 18.66 points:

term	contrast	points	how measured
POSITION	A → C	**+12.81**	mid-game vs turn 0, sheet off, same 2,500 games
CORPUS	E → A	**+4.75**	closed ladder vs open-sheet bo3, turn 0, sheet off, same config
SHEET	C → D	**+1.10** [0.31, 1.88]	paired McNemar, same 9,201 boards from two walks

C and D come from two passes of `joint_rows.build` over the same games, the second with `Board.prototype.setSheet` disabled — so suppressing the sheet changes what the leaf KNOWS and must not change which boards are scored. **The original run asserted that and it PASSED**: all 9,201 positions agree across the two walks on gid, turn, label, `aliveDiff` and the continuous HP witness, and the run aborts rather than report a pairing it did not check. The artifact on disk is a re-cut of that run's rows, so its `pairing_check` says the result is carried rather than re-performed — a process that did not do the check does not get to say PASSED.

The sheet at turn 0 is worth **+2.28** [0.57, 3.99] (B − A, paired), so taking the other path through the square gives position +11.63 instead of +12.81. Either way position is two-thirds of it and the sheet is the smallest of the three.

Three independent things say the config is not the explanation. Arm E re-runs the backtest's condition at the SWEEP's budget and horizon (n=40, h=20) and lands on 51.17% / Brier +0.0456 / ECE 0.1793 against the published 51.66% / +0.0466 / 0.1827 — inside E's own split-half floor of 1.54 points. The sweep re-ran itself at h=60 and got 69.86% against 69.84%. And §1 already recorded that 40 and 200 rollouts give the same turn-0 answer. **The horizon and the budget are settled: they move nothing.**

Two independent routes reach the same turn-0 number, which is why I believe the decomposition. Cutting the sweep's own committed dump down to `turn ≤ 1 AND aliveDiff == 0 AND |hpDiff| < 0.02` — its nearest thing to a preview board, sheets on — gives **55.70%** on n=237. Arm B measures a real turn-0 board on the same corpus with sheets on and gives **58.20%** on n=2,500. The subset's split-half spread runs 0.47 to 21.47 points across ten random by-game cuts (median ≈ 4.2), so those two agree.

THE HEADLINE 50.99% IS THE UNDERPOWERED READ, and the better number is not better news. It is the held-out fifth at n=200. Re-cut from `data/winrate-backtest-rows.jsonl`, the same leaf at n=40 over the **full** 6,886-game clean corpus ranks at **51.66%** (51.80% on 6,570 decisive calls) — real by p, and its majority class is 51.25%, so its edge over *always say p1* is **0.41 points against a median split-half floor of 0.75.** LESSONS §9: an effect smaller than the noise floor is not an effect. **On the closed-sheet ladder at turn 0 the leaf does not beat the majority class.** "Cannot rank at all" was reported off the wrong n and happens to survive the correction.

Now the answer to the three options, plainly.

- **(a) "fine mid-game, broken at turn 0" — the largest term, and "fine" is too kind.** Mid-game the leaf is genuinely not broken: it beats a coin on Brier by 0.044 [0.036, 0.051], its curve is monotone with slope 0.703, and on boards where the material baseline has *collapsed to the majority class* (aliveDiff 0, |hpDiff| < 0.02, n=411) it scores 62.29% against material's 51.09% — **+11.19 [5.05, 17.34] over counting.** It is reading real non-material structure. But it still puts **31.4%** of positions in the two extreme bins, and its top bin predicts 97% and wins 86%. A slope of 0.70 is not calibration; it is a leaf that ranks well and lies about how sure it is.
- **(b) "broken everywhere, the sweep measures something easier" — right that it is easier, wrong that it is only easier.** The +11.19 over a collapsed material baseline is not an artefact of easy positions. The sweep is not measuring material with extra steps.
- **(c) and there is a term nobody named: the CORPUS, +4.75 points — bigger than the sheet channel at either position.** The open-sheet corpus is `fit_policy.loadCorpus()`, which on its first 2,500 games is **99.9% our own** `gen9championsvgc2026regmbbo3` **scrape**, not the OTS archive as the generator's own comment implies. Its pool is lower-rated (median 1,174 against the ladder held-out fifth's 1,266) and it plays under **forced** open sheets, so both humans had full information and the outcome may be more determined by the matchup. Turn counts and forfeit rates are the same in both. **Neither mechanism is tested here** — the term is measured, its cause is not, and it is not a sampling artefact of the held-out slice, because the full-corpus backtest agrees with E.

DISCRIMINATION AND CALIBRATION FAIL SEPARATELY AND THE SPLIT WIDENS AS INFORMATION IS REMOVED. Arm A ranks at 55.92% against a 52.4% majority — its interval's lower bound is 54.0, clear of both the majority class and its 2.24-point split-half floor — and its Brier is still **worse than a coin**, with an MCE of 0.351. So a turn-0 leaf can carry real ranking signal and still be a liar about its confidence, which is the failure mode that matters to an argmax. By arm E even the ranking is gone and only the confidence is left.

A SIGN FLIP WORTH A RE-RUN, EXPLICITLY NOT ESTABLISHED. Exploration helps mid-game and may hurt at turn 0. Paired on the backtest's own 6,886 turn-0 ladder games at horizon 60, the **greedy** playout ranks at 53.09% against explore=1.0's 51.66% — **+1.44 [0.10, 2.77]** — while paired on the sweep's 9,201 mid-game boards explore=1.0 wins by **+3.20 [2.24, 4.15]**. The turn-0 lower bound is 0.10 against a median split-half floor of 0.75, so it is **inside its own noise floor and is not a result**; and greedy's Brier there is *worse* (0.3240 against 0.2966), so the two playouts differ in which failure they have rather than in quality. The two arms are also not the same code path (`battleInit` + `chooseAction` against `rolloutWinProb`). It needs `mew.js --miltank-explore`, which §3 already filed to SEARCH, and it needs the position held fixed.

WHAT THIS MEANS FOR PORYZ, since that is what the question was for. `docs/PORYZ-spec.md`'s representation is per-Pokémon HP fraction, status, every stat stage, revealed item and ability, and a threat matrix — and it is the leaf of `EV(a) = Σ P(reply) × V(board after)`. **Every one of those inputs is constant across games or absent at turn 0:** all eight bodies are at 1.0 HP, no status, no stages, and the closed-sheet ladder has revealed no items. The only feature that survives to turn 0 is the threat matrix over the brought four a side. So **PORYZ cannot move the turn-0 number, by construction of its own feature list** — if the target was "the leaf that reads 100% and loses", that leaf is the PREVIEW one and this spec is not aimed at it.

Aimed at what PORYZ-spec's engineering section actually says — making the mid-game EV sum affordable — it is well aimed, and this run hands it a bar measured on the same positions rather than quoted from another sample: **69.83% accuracy, Brier 0.2060, ECE 0.0925, slope 0.703 on 9,201 positions at release 6b0e4117d964**, with the rows on disk. PORYZ's premise sentence, "the whole learned value function is worth 3.4 points over counting", is about PORY2. The rollout leaf is worth **+4.58 [3.47, 5.68]** over the same graded material baseline mid-game and **+11.19 [5.05, 17.34]** where material has nothing to say. That is the incumbent PORYZ has to beat, and it is a harder incumbent than the spec assumed. This is a measurement, not a build decision; the decision is SEARCH's.

Filed, not fixed.

- `engine/rollout_r1.js:436` **puts a prose** `note` **key inside** `source_digests`**.** `engine/provenance.js:648` calls `digestOf()` on every key in that map, so a key that is not a readable path marks the whole artifact `unverifiable` — which is why `data/rollout-r1-explore-sweep.json` cannot be digest-verified. This file made the same mistake and moved the prose to a sibling key; the artifact went from `stale?` to `ok`. SEARCH's file.
- `engine/rollout_r1.js:26-29`**'s corpus comment is misleading about what it samples.** `loadCorpus()` reads bo3, OTS and ladder in that order, and the first 2,500 games — the whole R1 and sweep sample — are 2,497 bo3 and 3 smogtours. Every R1 number ever published is a **bo3 open-sheet** number, and §15 measures that this corpus is worth 4.75 accuracy points at turn 0. The published figures are not wrong; what they are *about* is narrower than the comment says.
- `data/censoring-value.json` **and** `data/click-censoring-census.json` **trip the provenance ratchet** — their generator ships without recording what content it read. Another division's files, written this session. Reported, not touched.

Re-cutting this artifact costs seconds, not half an hour:

```
RECUT=data/leaf-position-contrast-rows-6b0e4117d964.jsonl node engine/leaf_position_contrast.js
```

It opens the release named in the filename rather than the newest, refuses a dump that cannot name its engine, and never rewrites the dump it read. Verified: the re-cut reproduces every figure above bit-for-bit and leaves the row file byte-identical.

§16 — `censoring-value.json` is UNSAFE, and re-running it is not a repeat

> ***ANSWERED IN §17, 2026-08-05 — and by none of the three options below.*** *The confound was measured instead of argued: all 58 feature columns are identical across the engine bundles on all 1,751,688 corpus rows, so the fitting environment and the playing environment are the same FUNCTION here. Both artifacts were re-run against the live tree and both are* `ok`*. The section below is kept as what was true before that was measured; do not read its three options as open.*

2026-08-05. `provenance.js` flags it: `medicham2-browser.js` was `e2bcff0db96f` when it was measured and is `80fe43fba1a9` now, because WIRES 114–116 landed underneath it. The flag is correct and the artifact should not be quoted.

Two things had to be fixed before it could even be re-run, and both are worth more than the number.

The comparison baseline lived in a session scratchpad. The run compares the pre-censoring incumbent against the post-censoring fit, and the incumbent existed only as a copy in a temp directory that gets cleaned. A published figure whose input is in `%TEMP%` is not reproducible by anyone, ourselves included, one cleanup later. It is now `data/policy-weights-pre-censoring.json`, sha12 `01bc43936324` — the digest the artifact itself records, verified to match.

The artifact was invisible to provenance despite recording more than most files that pass. It stamped `source_digests_before` and `source_digests_after`; `provenance.js` reads `source_digests` and nothing else, so it fell to "rests on mtime alone" while carrying better evidence than the files around it. Recording something correctly under a name the checker cannot see has the same outcome as not recording it. The generator now writes the canonical key too — and the moment it did, the artifact stopped being `ok` and became `UNSAFE`, which is the whole point.

THE RE-RUN IS BLOCKED ON A JUDGEMENT, NOT ON COMPUTE. Both weight vectors were fitted under the pre-WIRE-114 engine. Scoring them through the current one breaks the rule in `CLAUDE.md` that the fitting environment and the playing environment must match, and it would measure *the censoring change plus three wires* as one quantity. The options, none free:

- **Refit both vectors under the current engine, then re-measure.** Correct, and the expensive one.
- **Re-measure through a release frozen at `e2bcff0db96f`.** Reproduces the original honestly, but the artifact deliberately reads the live tree — `no_engine_release` says freezing it would measure the thing being tested — so this changes the design of the measurement.
- **Leave it UNSAFE until the next refit lands anyway**, and do not quote it. Cheapest, and the status quo, but only honest while nothing downstream depends on it.

Not chosen here. `engine/censoring_value.js` refuses to run without `WEIGHTS_OLD` and now points at the preserved baseline and at this section, so whoever picks it up is choosing rather than guessing.

§17 — THE CONFOUND WAS MEASURED AND IT IS EMPTY. Both artifacts re-run, both `ok`. 2026-08-05.

None of the three options in §16 was taken, and the reason is a measurement rather than an argument. The blocking question — *"the vectors were fitted under one engine and would be scored through another"* — is a claim about the FEATURE FUNCTION, and a feature function is a function from a board to a number. Two versions of it are the same function if they agree on every board. So they were run against each other on every board the fit actually uses.

Result: all 58 feature columns are hash-identical across the three engine bundles, over 1,751,688 candidate feature vectors from all 9,230 clean open-sheet games.

bundle	`medicham2-browser.js`	`data/tags.json`	what read it	58 column hashes
release `09acd3b404ef`	`e2bcff0db96f`	`c0bb781f47a8`	`censoring-value.json`	identical
release `032b4a2979dd`	`80fe43fba1a9`	`c0bb781f47a8`	`click-censoring-census.json`	identical
live	`0cb911437fed`	`73c81e6421b8`	the re-runs below	identical

The three bundles were loaded from the frozen releases and registered under the live module paths, so `board.js`, `fit_policy.js` and `click_match.js` are the same bytes in every arm and only the simulator and the tag dex move. `engine/quality.js` is deliberately NOT swapped — a snapshot copy of it resolves the store inside the release directory and the walk would have had no rows to disagree about.

A null result from an instrument that cannot see is worth nothing, so the instrument was shown seeing. Under a Psychic Terrain with a Levitate body, the two frozen engines return `0` — priority refused — and the live one returns `Infinity`. The harness reads the same call the feature code reads, so a difference of that kind would have moved a column.

Why the change is real in the simulator and invisible in the features: across the whole corpus `board.js` makes **173,478** guarded calls to `priorityRefusedAbove`, of which **424** are under a Psychic Terrain, and in **0** of them is every live defender airborne. WIRE 117 can only change an answer when no grounded body is left to hold the bar up.

Both artifacts were then re-run against the live tree, and both reproduce. The corpus had grown 8,942 → 9,022 → **9,230** clean open-sheet games in between, so this is a fresh measurement on a superset rather than a replay — which makes the agreement evidence rather than tautology:

held-out class	published 3.42.0 (n=47,195, 1,809 games)	re-run (n=48,274, 1,851 games)
COERCED P(the coerced action), lower is better	−0.002614 [−0.003663, −0.001637]	**−0.002613 [−0.003650, −0.001672]**
PARTIAL mass on the candidate set	+0.000109 [−0.000286, +0.000491]	**+0.000122 [−0.000261, +0.000514]**
PARTIAL log-likelihood of the set	−0.002646 [−0.004037, −0.001377]	**−0.002662 [−0.004002, −0.001368]**
CONTROL, CLEAN log-likelihood	+0.000447 [0.000142, 0.000743]	**+0.000485 [0.000189, 0.000777]**
CONTROL, CLEAN top-1	+0.002 [−0.094, 0.098]	**−0.008 [−0.107, 0.085]**

Every verdict in §14 stands, including the negative one: the redirection correction still buys nothing measurable, and **every effect is still smaller than its own class's split-half floor** (COERCED 0.002613 against 0.011909; CLEAN logL 0.000485 against 0.004820). They resolve because the comparison is paired per decision, and that sentence must keep travelling with the numbers.

The census moved with the corpus and its shares did not: **249,404 actions over 9,230 games — CLEAN 94.9111%, PARTIAL 1.3344% (3,328), COERCED 0.5545% (1,383: Encore 1,152, `|drag|` 231)**, against 94.8916 / 1.3467 / 0.5509 at 9,022 games. The classifier still scores against the raw protocol at **encore recall 99.69% precision 96.31%, drag 96.74% / 96.74%** on the 67.23% of games that have a raw log.

`node engine/provenance.js --strict` **exited 0 at that point: 0 UNSAFE, 1 declared VOID (** `exploitability.json` **), 57 ok.** Both files carry `source_digests` over the tree they were computed on, so a next engine move flags them again by CONTENT rather than by mtime — **and one did, forty minutes later. See §17b, which is the more important half of this section.**

What this does NOT license. It says the four wires moved no feature on THIS corpus — it does not say the engine did not change, and it is not a general permit to score old weights through a new simulator. The next engine move gets the same treatment: run the columns, then decide.

The harness is `engine/feature_engine_contrast.js` **and it is in the repository, not in a session scratchpad — which is §16's own lesson applied to §17's evidence.** It writes `data/feature-engine-contrast.json` with `source_digests`, and it costs about four minutes per bundle over the whole corpus:

```
SHOWDOWN_PATH=… BUNDLES=live,09acd3b404ef,032b4a2979dd node engine/feature_engine_contrast.js
```

Each bundle runs in its own child process, because a module-cache swap cannot be undone in one. Two properties are worth more than the number it prints:

- **It refuses to report agreement unless its positive control disagreed.** `BUNDLES=live,live` returns *NOT A RESULT — the positive control did not separate the bundles*, verified before this was believed. A harness that silently loaded the same bytes twice would otherwise publish a confident "identical", which is the exact shape of every failure in this project's history.
- **It is not** `engine/feature_fixture.js` **and does not replace it.** The fixture hashes ~50 frozen boards so a weight file can *carry* the hashes, and its own header states the limit: a guard only guards what it exercises. This runs the same question over every board the fit actually uses, so a branch no fixture board stands on cannot hide in it. Both were green here, which is the first time they have been asked the same question on the same day.

§17b — AND THEN THE TREE MOVED AGAIN, AND THIS TIME THREE COLUMNS MOVED WITH IT. A REFIT IS OWED.

The instrument built in §17 found a real feature change forty minutes after it was written, and `engine/feature_fixture.js --check` **— the guard** `status.js` **prints the refit edge from — is BLIND to it.** That is the finding of this session, and it outranks everything above.

Between 15:40 and 15:44 on 2026-08-05, while this division was measuring, three files moved:

file	was	is	what it did
`engine/fit_policy.js`	`45f545425420`	`caeeec21c560`	`loadCorpus()` went **9,230 → 6,055** clean open-sheet games
`engine/medicham2-browser.js`	`0cb911437fed`	`82bed8cdcf6b`	—
`engine/board.js`	`54e3d2ca9f85`	`5bdaa3923958`	the feature file itself

Re-run with the sample pinned — **1,136,845 candidate vectors over the same 6,055 games, identical** `row_key_hash` **in all three arms** — the verdict is no longer IDENTICAL:

> ***MOVED —*** `deadNoLastMove` **,** `movesFirst` **,** `diesBeforeMoving` ***differ on identical rows. This is a REFIT, not a restamp.***

Both frozen bundles (`e2bcff0db96f` , `80fe43fba1a9`) agree with each other and disagree with the live tree in the same three columns, which is what a single new change looks like — **and it is: CHANGELOG 3.49.0, *"There were two implementations of who moves first. One is deleted, and the survivor is dynamic."*** Speed

order is now re-sorted mid-turn, so `movesFirst` and everything downstream of it answers a different question than the weights were fitted against. The columns name the change without anyone having to guess, which is what a per-column hash is for.

** `node engine/feature_fixture.js --check data/policy-weights.json` says *"feature semantics OK — agrees with board.js on every fixture board"* on that same tree. **Both instruments are working; they are answering the question on different boards, and the ~50 frozen fixture boards do not stand on the branch that moved. The fixture's own header says a guard only guards what it exercises — this is the first time that limit has been shown with a number rather than stated.** `status.js` prints `refit edge: CLEAN` from that check, so the refit edge is currently reported clean and is not.** Two consequences, in order:

1. **A refit is owed on the 15:43–15:44 change** — three of the 58 columns changed meaning under weights fitted against the old ones. That is not WIRES 114–117; those were measured empty above.
2. **The refit edge needs both instruments.** The fixture is what a weight file can CARRY, and it should stay; the corpus contrast is what can DETECT. Wiring `feature-engine-contrast.json` into `status.js` beside `feature_fixture --check` is the obvious next move, and it is deliberately not done in this pass — a status line added at the end of a session that watched three files move is a line nobody has watched behave.

`data/click-censoring-census.json` **and** `data/censoring-value.json` **are therefore UNSAFE again**, now through `engine/fit_policy.js` rather than through the simulator, together with `data/partial-label-em.json`, which is the same cause and was not touched here. `node engine/provenance.js --strict` **exits 1 with 3 UNSAFE.** That is stated, not filed: the re-runs in §17 were valid photographs of the tree at 14:26–14:40 and they say so in their own digests; the tree they photographed no longer exists.

They were not re-run a third time, deliberately. The corpus definition changed by a third (9,230 → 6,055 open-sheet games) inside the same twenty minutes, so a third run would publish a different population under the same headline, attributable to neither the engine nor the censoring change. Re-run both against a still tree — the loader digest is in every artifact — and the numbers in §17 are the ones to compare against.

A measurement cannot be taken while the lens is being changed, and `engine_release.js` **does not cover this case.** A release freezes 23 files; it does not freeze `engine/fit_policy.js`, and it cannot freeze the store. That is why `feature_engine_contrast.js` pins its sample by game id and refuses when one goes missing: the first version of it reported **all 58 columns moved** purely because the corpus shrank between two children, which is a REFIT verdict manufactured out of somebody else's edit.

§17a — the `board.js` partial-body over-refusal is worth 0 rows, and here is the number

ENGINE filed it rather than fixing it: `engine/board.js:2565` and `engine/position_features.js:231` map their priority defenders to `{ability, fainted}`, so `isGrounded()` sees no type list and no item and a Flying-type foe is still over-refused **in the feature vector**. Widening that signature moves the feature vector, which is a refit, which is why it came here. Measured on the fit's own decisions over all 9,230 games, rebuilding every defender twice — once the way `board.js` does it, once with the types and item the board already holds:

	n	**of**
candidate feature vectors	1,751,688	—
with a priority move	332,030	19.0% of candidates
aimed at a body, i.e. reaching `board.js:2560`'s guard	135,552	40.8% of those
under a Psychic Terrain	**362**	**0.27%** of guarded priority candidates
where a complete body changes the answer	**0**	—

The artifact on disk carries the same measurement over the post-15:40 corpus (6,055 games, 1,136,845 vectors, 220,932 with priority, 91,240 reaching the guard, **273** under a Psychic Terrain, **0** changed, upper bound 5). Two corpora a third apart give the same answer, which is the strongest thing that can be said about it without more Psychic Terrain in the metagame.

The only five rows in the entire corpus where types and item flip the bar are `protect` ×4 and `ragepowder` ×1 — **self-targeted moves, which** `board.js` **never routes through** `priorityRefusedAbove` **at all**, because the branch is guarded on `cand.targetMon`. Counting them as exposure would have overstated it by five rows out of 1.75 million; both counts are recorded here so the guard is visible rather than assumed.

So: NOT WORTH A REFIT, and the exposure is 0 rows in 1,751,688 (upper bound 5, of which 0 are reachable). Two things keep it from being closed. `fails.groundedBodyIncomplete` fires on **100% of 173,478** calls — every single feature-path call is made with a body that cannot answer — so the defect is total and only its consequence is nil; and the consequence is a property of THIS corpus, where 0.27% of guarded priority candidates stand on a Psychic Terrain. A metagame that pairs Psychic Surge with Flying bodies moves that number without anything in the code changing. The right time to widen the signature is the next refit, when the feature vector is moving anyway and the change is free. `engine/position_features.js`'s copy is a separate call site and is NOT measured here.

Reading a run

```
node engine/sprt.js <file>
```

Cat the shards together first. **Never read an interim SPRT** — 66.7% became 44%, 57.7% became 50%. The bound exists precisely so you do not have to look. SPRT is valid under continuous monitoring because its boundaries were derived for it; a Wilson interval read repeatedly is not the same thing and does not inherit that property.

The unit is the **decisive pair**, not the game. In a paired run a 1-1 split means the team decided it, not the policy.

Reading a stamp

```
node engine/run_stamp.js --show        data/rollout-r3.json
node engine/run_stamp.js --reconstruct data/rollout-cost.json
```

Every gate artifact has a `<name>.meta.json` beside it saying which configuration produced it. `status.js` prints the headline under the gate line, so the absence of a stamp is on the same screen as the number — R1's +2.91 was quoted for a day against a dump that could not say which of two runs four accuracy points apart it was, and nothing was hidden then either. The fact simply lived in a file nobody opened.

Three things to check before quoting any of it:

- `reconstructed: true` means **inferred from a commit, not observed**. Read `confidence`, which publishes the gap in seconds between the artifact's own timestamp and the commit that carried it.
- `git.dirty: true` means the commit id does not describe what ran. Trust `source_digests`.
- `source_digests` hashes **worktree bytes**; `git.blobs` names git objects. On Windows those differ by line-ending translation — `data/engine-data.js` does — so never compare one to the other.

`writeStamp()` is the only mode worth trusting, because only the run knows its own settings. `reconstruct()` exists for the artifacts that predate it and labels itself on every line.

18. THE PORYGON2 SEPARATION GATE — PRIORITIES #23. PASS, and the interesting number is the one that is not in the verdict. 2026-08-06

`engine/porygon2_separation_gate.py` → `data/porygon2-separation-gate.json`. **The MILTANK leaf redesign (#24) is buildable.** PORYGON2 does not collapse a subtree to one number.

39,843 same-game position pairs two turns apart, across 6,328 clean HUMAN ladder games, every interval bootstrapped with the GAME as the cluster. Thresholds were written to disk at **05:59:44Z**, the run wrote at **06:56:06Z**, and `--run` refuses to start unless the declaration on disk matches the block in the generator character for character.

	measured	declared bar			
T1 separation median |	Δscore|	over 2 turns	**0.1628** [0.1600, 0.1653]	≥ 0.02	PASS
T2 locality same-game 0.1985 vs unrelated 0.2801; D	**+0.0815** [0.0786, 0.0845]	CI lower > 0.0043	PASS		
T2 locality ratio R = same / unrelated	**0.709** [0.700, 0.718]	≤ 0.75	PASS		
T3 direction agrees with the material sign	**85.58%** [85.16, 85.98]	CI lower > 50, point ≥ 60	PASS		
T3 secondary, moves toward the eventual winner	61.59% [61.12, 62.07]	reported, not gated			

All eight PORYGON2 arms pass — 17 and 19 features, plain and weighted, k=50 and k=200 — with R between 0.684 and 0.739. The verdict is read off **17f weighted k=50**, which is what `docs/MODELS.md` headlines.

THE NEGATIVE CONTROLS DID THEIR JOB, AND THE SECOND ONE IS THE ONE THAT MATTERS. A constant 0.5 leaf fails all three (median 0, R undefined, direction 0%). That was the required control and it is the weaker one. A **uniform-random** leaf **PASSES T1 with a median of 0.2924 — nearly twice PORYGON2's separation** — and fails T2 (R = 0.995 [0.985, 1.004], D CI [−0.0012, +0.0049] straddling zero) and T3 (50.28% [49.72, 50.87]). So separation alone cannot tell a value function from noise, which is precisely why T2 was written as the deciding test. A gate proved only against a constant would have been passed by static.

AND THE FINDING THE VERDICT DOES NOT CONTAIN. A bare material count — `0.5 + 0.15·alive_diff`, the same rule `porygon2.py` scores itself against — was run through the identical pipeline as a BASELINE rather than a control. At a two-turn gap **it passes the gate too**: R = 0.703 [0.692, 0.715], statistically indistinguishable from PORYGON2's 0.709. Read alone, that says the 17 features buy no locality at all.

It is not read alone, because the addendum below settles it. **At the ONE-turn gap the search actually operates at, the material count goes flat: its median \|Δ\| is 0.000 and it returns the identical number on 58% of adjacent positions**, while PORYGON2 moves on 99.3% of them with a median of 0.1154 and its locality gets *better*, R = 0.5464 [0.5392, 0.5534]. Every branch a material leaf cannot separate is a branch the argmax decides by tie-break. That is the case for #24, and it is a different case from the one the headline makes.

Two comparisons in that block that look like findings and are not, stated so nobody quotes them:

- the material baseline's *toward-the-eventual-winner* rate (64.77%) is **higher** than PORYGON2's (61.59%) — but its score moves on only 21,975 of 39,843 pairs, i.e. only where a Pokemon actually fainted. It is scoring the easy subset. The two rates are computed on different populations and are not comparable.
- the gate produces properly-intervalled accuracies for free: **17f weighted k=50 at 63.11% [62.32, 63.81]** against the material sign's 61.02% [60.06, 61.96] on the same 52,501 positions. These are **separate** game-clustered intervals, **not a paired test**. `docs/MODELS.md`'s 63.59% is still marked **NOT**

MEASURED and this **supersedes nothing** — a paired difference with a split-half floor is what would close it, and nobody has run one.

WHAT WAS FROZEN, AND WHAT COULD NOT BE. The gate is stamped to engine release `4c73f9cafa4b` and that stamp is honest about its own limits: **none of PORYGON2's sources are in the frozen set** — not `engine/porygon2.py`, not `data/porygon2-species.json`, not either corpus. PORYGON2 is a Python model and `REL.require` is a JavaScript shim, so it cannot be loaded through a release at all. What the release *did* supply, through `REL.require`, is the thing that decides the population: the frozen `engine/quality.js` + `data/quality-filter.json`. For the rest the generator takes its own photograph — sources copied into a private tree and imported from the copy, live originals re-digested afterwards (none moved) — and the two append-only stores are pinned by the **clean id set** (7,992 ids, sha256 `4ccc0afc…`) rather than by a whole-file digest, because the collector appends hourly and a file digest would void any run longer than an hour.

A DEFECT FOUND ON THE WAY, AND IT IS THIS REPOSITORY'S SIGNATURE SHAPE. The first artifact carried `"R_same_over_unrelated": NaN` — Python's `json.dump` writes a bare `NaN`, which every Python reader accepts and which **is not valid JSON**. `JSON.parse` throws on it. The effect was that `engine/provenance.js` **could not read one field of the file and reported it** `ok`: a clean bill of health issued over a document it had never parsed, including the `void` flag that exists precisely so a generator can condemn its own run. Both dumps now pass `allow_nan=False`, so it raises instead of shipping. Worth a sweep: any Python generator here can emit this, and the artifact still looks fine from Python.

Three smaller things the gate needed and now does:

- it writes `corpus.clean_games` and `corpus.population_ceiling` **spelled the way** `provenance.js` **reads them**. Its prose `population` block was invisible to `declaredGamesFrom()`, which is the §5e state where an artifact "records no game count".
- the declaration timestamp survives every re-run. `--run` overwrites the file, so reading `generated` would report the last run as the moment the thresholds were fixed — drifting later than the numbers, every time.
- a `--run` re-run used to silently delete the `--addendum` block. It is carried forward now, each block keeping its own timestamp and digests.

Disclosed rather than omitted: a 150-game smoke run of this pipeline executed at 06:02Z, after the declaration and before the headline sample, to find bugs. Its numbers were seen first. No threshold changed — the equality check enforces that — but the smoke run's R landed at 0.735 against a 0.75 bar, close enough that saying nothing about it would be the omission this division exists to prevent.

What this gate does NOT establish, and #24 should not be read as having it:

- it says the leaf **separates**, not that swapping it in **wins**. The unit that answers that is the decisive pair, and it needs an SPRT against the incumbent playout.
- the pairs are consecutive positions from *real games*, not **sibling branches from one node**. Siblings differ by one action from an identical board and are more alike than anything measured here. The lag-1 addendum is the closest available proxy and it is a proxy.
- T3's ground truth is `alive_diff + hp_total_diff`, which are two of PORYGON2's own inputs (`alive_diff` carries a learned weight of 5.12 against a mean of 1.0). It asks whether the model respects its strongest features. A k-NN guarantees no such thing, so it is not vacuous — but it is not independent, which is why the outcome-anchored secondary is reported beside it.
- **no split-half was run.** The noise floor here is built into the design instead: T2's unrelated-pair arm *is* the floor for the effect claimed, and every interval is game-clustered. The estimator is deterministic given the game set, so a split-half would re-measure what the bootstrap already reports. The one stochastic input

— how the unrelated partner is drawn — was checked by a second mechanism: the any-turn control gives R ≈ 0.74 against the turn-matched 0.709, so the conclusion does not depend on the draw.

19. THE STRONG-PLAYER BASELINE — the cutoff gradient exists and is free; §1.3's "real humans" column cannot be compared to it; and "flat in rating" is NOT MEASURED. 2026-08-06

`data/strong-player-baseline.json`, written by `build/strong_player_baseline.js` (one process, ~2 minutes). Built for task #46 out of `data/smogon-stats/` and this repository's own stores. **It reads no simulator, no leaf, no feature layer and no policy weights**, so it is not invalidated by an engine release or by a MAG refit and does not need re-running when either happens. It is invalidated by a new Smogon month or by a change to `data/quality-filter.json`.

The generator exists because of §19e. The claim this section retracts — *"measured move quality is close to flat in rating"* — has been quoted in a fitted model's own caveat block for weeks with no generator behind it. Shipping an artifact with the same defect would have been the same mistake in a new file. It lives in `build/` rather than `engine/` for a dated reason: it was written on 2026-08-06 while an ENGINE agent was rewriting the simulator, and this division does not add files to another agent's directory mid-flight. If that reason expires, `engine/` is the better home.

The question it was built for is Will's, 2026-08-06, reading `docs/ROADMAP.md` §1.3: *"'outright failed' could be incompetence or a high level play and we dont know the difference."*

19a. What the 1630 weighting can and cannot support — stated before it is used

It CAN support what strong players BRING and RUN. Species usage at four skill weightings, and ability / item / spread / move frequencies within a species.

It CANNOT support what strong players CLICK. The files are team-composition aggregates. There is no turn, no board, no opponent and no click in them. All three of §1.3's metrics — *moves that outright failed*, *moves that hit an immune target*, *moves that were super effective* — are per-turn rates and **have no Smogon counterpart at any cutoff**. A 1630 column added to that table would look comparable and would not be, which is worse than a missing column.

"Cutoffs are weightings, not subsets" is now MEASURED rather than quoted. Across both months and both formats, every species' `Raw count` is identical at all four cutoffs — **3,117 of 3,117 species-cutoff pairs**, and 310 of 310 rows of the usage table's `Raw` column. Only `Avg. weight` moves. So no cutoff describes a *set of players*; each describes the same battles seen through a different lens.

And no rating number is mapped onto a cutoff number anywhere in this work. Nothing in this repository or in the Smogon files establishes that `1630` is the same ruler as the Showdown `|player|` rating field. The corpus is located on the cutoff axis by **composition**, which is scale-free.

19b. The gradient exists, and it was free

Four cutoffs (0 / 1500 / 1630 / 1760) × two months (2026-06, 2026-07) × two formats (B01, B03) are already on disk: 16 usage files and 16 moveset files. Nothing was collected.

Effective sample size is `Raw count × Avg. weight` per species per cutoff. Smogon weights lie in [0,1], so `Σw² ≤ Σw` and the true effective sample `(Σw)²/Σw²` is **at least** `Σw` — the intervals below are therefore too WIDE, not too narrow. The offsetting hazard is stated and not corrected for: the independent unit is a PLAYER, one strong player contributes many battles, and the files cannot measure that.

The 1760 column costs almost everything. Effective team slots, 2026-07 B01:

cutoff	avg weight/team	effective team slots	share of raw
0	1.000	3,529,372	100%

cutoff	avg weight/team	effective team slots	share of raw
1500	0.512	1,807,038	51.2%
1630	0.068	239,997	6.8%
1760	0.002	7,059	**0.2%**

The noise floor is the same cutoff across two months, which is an *upper* bound because it contains real metagame drift as well as sampling. Total absolute species-usage difference, summed over 310 ranked species, 2026-07:

contrast	L1 (points)	vs the cutoff-0 month floor of 140.9
cutoff 0 vs 1500	69.4	**inside it — 1500 is not distinguishable from the whole ladder**
cutoff 0 vs 1630	151.6	above
cutoff 0 vs 1760	195.0	above
cutoff 1630 vs 1760	70.6	inside

83 of the 104 species at ≥1% base usage have a 0→1760 usage change whose 95% interval excludes zero, and the large ones are monotone across all four cutoffs. Charizard-Mega-Y 15.69% → 18.75% → 23.46% → **30.58%** (Δ +14.89 [13.81, 15.96]); Kingambit 23.40 → 36.79 (+13.39 [12.27, 14.52]); Sableye 6.52 → 3.36 (−3.17 [−3.59, −2.74]). *(Population: Smogon 2026-07* `gen9championsvgc2026regmb` *, 1,764,686 battles, reweighted.)*

Within a species, the gradient is real for **moves and spreads**, marginal for **items**, and absent for **abilities**. Mean total-variation distance over the top 20 species by raw count, cutoff 0 vs 1760, computed over the **intersection** of the two listed key sets:

section	cutoff gradient	month noise at cutoff 0
moves	**17.89**	11.52
spreads	**9.17**	3.72
items	6.98	4.99
abilities	1.63	5.12

The abilities row is the honest negative, and it needs one species named: the 5.12 month-noise is dominated by **Sneasler**, whose Unburden/Poison Touch split really did move 15.3 points between June and July. Excluding it, abilities barely move with skill either — the format's abilities are near-locked at every cutoff.

A correction made mid-run, recorded because a first pass shipped it. A key absent from a Smogon moveset list is **not 0%** — the file lists the top few plus `Other`, so an absent key is below that list's reporting floor. Treating it as zero manufactured an 18-point "gradient" on Venusaur/Energy Ball. Every distance here is over the intersection, with the unlisted mass reported separately.

19c. Where our corpora sit on that axis — and there are THREE of them, not one

They must never share a sentence with only one population named.

corpus	store	filter	rated slots	median	≥1400	≥1500
clean **closed** ladder	`games.ladder.jsonl`	clean, 8,047 games	11,852	**1266**	26.2%	14.4%
Bo3 **open sheet**	`games.bo3.jsonl`	none	14,539	**1175**	5.33%	0.72%
Bo1 open sheet	`games.ots.jsonl`	clean, 2,860 games	3,414	**1087**	0.23%	0.09%
unfiltered ladder	`games.ladder.jsonl`	none, 45,006 lines	83,668	1130	6.21%	3.09%

Two things fall out of that table.

The dispatch's figures are confirmed and they belong to the Bo3 store. 14,465 rated slots at p10 1043 / median 1175 / p90 1355 / max 1707, ≥1400 5.3%, ≥1500 0.7% is `data/games.bo3.jsonl`, which now holds 14,539 slots at exactly those quantiles. Same file, 74 slots of growth.

The clean filter moves the population a long way, and that is the population §1.3 benchmarks against. Unfiltered ladder median 1130 with 6.21% ≥1400; clean ladder median **1266** with **26.2%** ≥1400. The bot and behavioural-bot rules remove a large low-and-flat block. So §1.3's "real humans" are not the median-1175 population — that is the corpus MAG was *fitted* on. Any sentence of the form "the ladder is median X" has to say which of the two it means.

On the cutoff axis, by composition (L1 between the corpus's team-preview species vector and each cutoff, mega formes collapsed to base on both sides):

corpus	vs cutoff 0	1500	1630	1760	own split-half floor
clean closed ladder, vs 2026-07	**138.6**	174.9	230.9	274.6	36.9 – 75.8 (median 53.4)
Bo1 open sheet, vs 2026-06	211.2	184.0	159.2	**157.1**	45.9 – 91.8 (median 67.3)

The closed ladder is nearest cutoff 0 and moves monotonically away from every higher one. Its 138.6 is about one month of drift at cutoff 0 (140.9) and its games run 2026-07-22 to 2026-08-06 — later than the newest published month — so cutoff 0 is a fit and 1630/1760 are not. **The Bo1 open-sheet corpus cannot be placed at all**: its four distances span 157.1–211.2 against its own split-half floor of 45.9–91.8, so the cutoff ordering is inside its noise. NOT MEASURED for that store, and the apparent preference for 1760 is not a finding.

19d. The Fake Out / Armor Tail case, answered as far as it can be

Farigiraf runs Armor Tail on **97.80%** of sets at cutoff 0 [97.77, 97.83] and **99.11%** at 1760 [98.59, 99.45] — a real +1.31-point gradient [0.89, 1.73] against a month noise of 0.11, and **completely useless for the question**, because the ability was already near-universal everywhere. Incineroar carries Fake Out on 98.89% of sets at cutoff 0 and 99.82% at 1760. Expected Fake Out carriers per team of six: 0.728 at cutoff 0 → 0.751 at 1760.

So the collision is **at least as available** at the top as at the bottom. The aggregate can say the two sides are both brought slightly more by strong players; it cannot say who clicked, because there is no turn in the file.

What this implies for task #44 part 1, which is owned elsewhere and NOT attempted here. The denominator of a failed-move rate is composition-confounded, and the split has to condition on it. Protection is the largest single source of a `|-fail|` line — a repeated Protect fails by rule — and it moves with the cutoff: expected protection carriers per team of six run **4.00 → 4.14 → 4.34 → 4.39** across the four cutoffs, and Detect alone runs 0.103 → 0.144 (+39% relative). A raw failed-move rate therefore rises with how much protection the population runs, independently of anybody playing better or worse.

19e. `fit_policy.js:1264`'s "flat in rating" — NOT MEASURED, not false

The claim: *"Open-sheet players also average ~185 rating points lower, though measured move quality is close to flat in rating."* `docs/DEFENSE.md` §1 gives the numbers — failed moves 2.59% under 1100 against 2.30% at 1400–1600; blocked actions 4.66% to 3.43%.

Neither figure has a generator in this repository. `engine/realism_report.js` counts the same protocol lines but **pools both players of a game and never bands by rating**, and no `data/*.json` carries a rating-banded rate. The claim has sat in a fitted model's own caveat block with nothing behind it that can be re-run. That is the P1 class this division already named for PORY's coefficients.

Recomputed here from the protocol, attributed to the **acting** side, on the clean closed ladder — 8,047 clean games / 7,040 raw logs matched / 14,078 player-slots / **171,801 moves**. Intervals are a game-clustered bootstrap, 400 resamples.

rating band	moves	failed % [95%]	immune % [95%]	super-effective % [95%]
<1100	21,098	2.218 [1.93, 2.57]	2.318 [2.01, 2.63]	20.84 [20.02, 21.59]
1100–1199	27,390	2.472 [2.08, 3.03]	2.234 [1.98, 2.47]	20.62 [19.97, 21.33]
1200–1299	22,699	2.291 [1.99, 2.59]	2.071 [1.83, 2.31]	20.62 [19.90, 21.32]
1300–1399	22,577	2.197 [1.95, 2.48]	2.210 [1.98, 2.46]	20.84 [20.16, 21.56]
1400–1599	24,669	2.165 [1.83, 2.53]	2.027 [1.81, 2.25]	21.24 [20.55, 21.91]
1600+	7,486	2.658 [1.93, 3.52]	2.151 [1.72, 2.57]	19.16 [17.95, 20.36]

The direction reproduces. <1100 against 1400–1599 on failed moves: **−0.054 points, 95% CI [−0.539, +0.403]**. Nothing here contradicts "close to flat".

But the design was powered for a 0.674-point difference at 80% power on a 2.2% base rate — a 31% RELATIVE change. Anything smaller was never detectable, so *"flat"* and *"an effect up to 30% of the base rate"* are the same observation in this corpus. Immunity, same two bands: −0.291 [−0.679, +0.057], MDE 0.517 on a 2.32% base. Super effective: +0.396 [−0.606, +1.461], MDE 1.527 on a 20.8% base. All three contrasts contain zero.

And the whole between-band spread is inside the within-band noise floor. Failed-move rate across all six bands spans 2.165% to 2.658% — 0.49 points. Cutting a *single* band eight ways produces spreads from **−0.489 to +0.806** points. The observed effect *is* the noise floor.

The binding constraint is not the rating range, which is what the dispatch expected. The clean closed ladder holds 32,155 moves at ≥1400, 17,551 at ≥1500 and 7,486 at ≥1600. The binding constraint is that the metric is a ~2% event: 25,000 moves is only ~530 failures, and separating two bands by a fifth of a point on that needs far more.

So the verdict is NOT MEASURED. The claim is not shown false and it should not be repeated as though it were established. `fit_policy.js:1264` and `docs/DEFENSE.md` §1 should say *not measured at this power* — filed, not edited, because `engine/` is being rewritten in parallel.

What would settle it is not more games at this metric. Either a metric with a higher event rate (super-effective is 21%, so its 1.53-point MDE is **7.3% relative** against failed-move's 31% — four times better), or a paired design that holds the board fixed, which is what a click-level model gives and a rate does not.

19f. What §1.3 should say instead

The gap §1.3 reports between MAG and humans is **3.87 points** (6.34% against 2.47%). The entire measurable rating effect inside the human population is at most **0.67 points** and its point estimate is **0.05**. The gap is ~5.7× anything skill does to this metric within the corpus. **Closing it makes MAG resemble a human; it cannot make MAG resemble a strong human, because on this metric strong and weak humans are not separated at all.**

Five changes, in order:

1. **Name the population on the same line as every figure.** The column is *clean closed-sheet ladder games, both players pooled, no rating condition*. It was 1,905 clean games when written; the same predicate now selects 8,047 clean games / 7,040 raw logs / 171,801 moves at median rating 1266 with 26.2% of rated slots ≥1400. It is **not** the open-sheet corpus MAG was fitted on.
2. **Give an interval.** A bare percentage invites a comparison it cannot support.

3. **Say the human column is a REALISM target, not a skill target** — and give the reason rather than the assertion: within this corpus the same rate does not separate a sub-1100 player from a 1400–1599 player at a detectable size (−0.054, 95% CI [−0.539, +0.403], MDE 0.674).
4. **Do not add a Smogon 1630 column to that table.** It has no per-turn rate at any cutoff.
5. **If a strong-player column is wanted, it belongs in a different table about bring and build** — where 1630 and 1760 genuinely answer the question. That table is what this artifact provides.

For reference, the human column recomputed on the current corpus with attribution to the acting side (*clean closed ladder, 171,801 moves over 7,040 logs, 2026-08-06*): failed **2.275%**, immune **2.074%**, super-effective **21.005%**, against §1.3's 2.47 / 1.91 / 21.37 on 1,905 games. Close, and stated so the population and the count travel with the numbers — not as a substitute for re-running `realism_report.js`, which pools rather than attributes.

19g. Filed, not fixed — a real defect found on the way

`data/smogon-priors.json`**'s** `teammates` **array is polluted on 275 of its 284 species.** Kingambit's holds 32 entries where the source file lists 10, and the extras are `intimidate`, `blaze`, `sitrusberry`, `passhoberry` — the *next* species' Abilities and Items rows. The cause is `engine/smogon_priors.js:160`: `grab('Teammates', 'Checks and Counters')` terminates on a section these moveset files **do not contain**, so the regex falls through to `$` and swallows the rest of a 14-chunk window that already spans into the following species block. The same file's `abilities`, `items`, `spreads` and `moves` are unaffected — each has a section that really does follow it.

Blast radius today: zero. Nothing in the repository reads `teammates` out of that file; the only occurrence is the writer. It is latent, not live. Not fixed here because `engine/` is being rewritten in parallel and this division does not patch another agent's file mid-flight.

`provenance.js` **classes this artifact's corpus as open-sheet, and it is closed.** `provenance.js:268` is `/games\.(ots|bo3)\.jsonl/.test(withDeps) ? 'opensheet' : 'ladder'`, and the generator names both open-sheet stores because it reads their rating summaries. Its **primary** corpus is the clean CLOSED ladder, so drift is judged against the wrong ceiling and the artifact prints *CORPUS DRIFT — declares 8,047 games; 9,177 are clean open-sheet now, 12.3%*. That is the same class as the named exception §5 already records for `winrate-backtest.json`, and it needs the same treatment in `provenance.js`, which is `engine/` and was in flight.

It is left flagged on purpose. `provenance.js` honours a declared `population_ceiling`, and declaring one here would set the ceiling equal to the count and silence this artifact's drift check permanently — which is §5e's `--fix` failure exactly: refitting the world so a check goes green. A false-positive warning that says why is better than a real check switched off.

19h. What this artifact does and does not stamp

`source_digests` is at the **top level**, because `provenance.js:685` reads `j.source_digests` and not `j.provenance.source_digests` — a first run put it in the wrong place and broke the content-digest ratchet in `data/provenance-stamp.json`, which may fall and may never rise. It is closed now; the artifact is one of the **2 of 93** verified by content rather than by mtime.

Only the stable inputs are stamped, deliberately. The generator, `engine/quality.js`, `data/quality-filter.json` and all 32 Smogon monthly dump files — every input that is supposed to be frozen, so a change in one is a real event. The three game stores are **not** stamped and the artifact lists them under `provenance.unstamped_inputs` with the reason: they are append-only, the collector runs hourly, and their digest changes every hour by construction. Stamping them would hang a permanent mismatch on the artifact, which is §5a's "mtime cries wolf" wearing a hash. The instrument for an append-only corpus is the declared count, and that is what `corpus.clean_games` is for.

One more small trap, recorded because it is generic: `run_stamp.sourceDigests()` adds a prose `note` key to the map it returns, and `provenance.js` iterates every key of `source_digests` as a path to re-hash. Left in, it prints *"stamped input note cannot be read to verify"* on every run. The generator deletes it and carries the prose in `source_digests_note` instead.

> ***FIXED IN THE READER, 2026-08-07 (§20).*** *Working around it in each generator is a fix that has to be remembered once per generator, and the next one to follow* `provenance.js`*'s own printed advice pays the same tax —* `data/wire-ladder.json` *did, on its first run.* `run_stamp.js` *now EXPORTS* `STAMP_NOTE_KEY` *and* `provenance.js` *imports it and skips exactly that key. One place knows which key is prose, which is the FACTS-ARE-GLOBAL rule applied to a two-line string. Same shape as the frozen-release false positive already handled twenty lines below it in that loop: a checker that penalises the workflow it recommends gets ignored.*

Living-docs obligations this pass did NOT discharge, because the dispatch scoped the write to two files and forbade `status.js --write`: the CHANGELOG entry and version bump, and whether `SUMMARY.md` or the white paper should carry the §1.3 correction. `docs/ROADMAP.md` §1.3 itself is unedited — the rewrite is specified in 19f and is the router's call to place.

20. THE RELEASE LADDER — what one night of WIRE fixes bought, controlled. MEASURED 2026-08-07.

`engine/wire_ladder.js` → `data/wire-ladder.json`. Nine frozen releases plus a repeat of the baseline, **1,995 games each**, one pinned census (`data/wire-ladder-census.pin.json` , `f63179105d3c`) and one team pool (`3d0112fce455`). `arms_comparable.compare()` cleared **all nine** arms against the baseline; the eleven watched inputs were byte-identical before and after; the planted-divergence proof and all seven equivalence rules passed on every arm. This replaces the pairwise before/afters retracted in CHANGELOG 3.62.1, and it replaces WIRE 6's own artifact, which pinned its census to a path inside an agent scratchpad.

The instrument is deterministic and that is now demonstrated, not assumed. The pre-WIRE-1 baseline ran first and last with eight arms between: every measured field identical and the per-game divergence depth identical game for game. The whole ladder was then run three times end to end and reproduced. So every difference in the table is the engine change.

The headline is a negative and it should be read first. The median game still parts after ONE completed turn, at every rung. Six wires did not move it. Whole-game agreement went **2 → 22 games of 1,995** — 98.9% of games still diverge. The divergence rate is saturated and says almost nothing; what moves is the DEPTH, and the useful unit there is the protocol line, not the turn:

	baseline	after six wires
median first-divergence line	13	**14**
mean	15.01	**23.97**
p90	30	**57**
games that never diverge	2	**22**
median completed turns	**1**	**1**

Paired over the same 1,995 games, the top rung parts **later on 742, earlier on 141, at the same line on 1,112**. More than half the sample is untouched by the entire night, and the median delta is zero.

Per rung, the one number each is worth (paired against the rung before it; "net" is games parting later minus games parting earlier):

rung	net later	its own class
mega resolution order (unpublished intermediate)	+73	`ordering` 247 → 170

rung	net later	its own class
Knock Off base-power truncation (intermediate)	**0**	4 games reclassified, nothing else
WIRE 1 Protect/crash	+65	`-miss field 3` **18 → 1**
WIRE 2 stall counter	+156	`unrelated event mismatch` **700 → 562**
WIRE 3 refused stat drops	+99	`event missing` 673 → 636
WIRE 4 fixed-point damage	+16	`-damage field 3` 216 → 179
WIRE 4 recoil/drain rounding	+48	`-damage field 3` 179 → 141
WIRE 6 priority brackets	**+287**	`turn order` **85 → 3**

Three things in that table are worth more than the ladder itself:

- **An unpublished intermediate outranks a named wire.** The mega-resolution-order cut (`28e66a7c9ab8`, 02:36) is worth **more than WIRE 1 on every measure here** — net +73 against +65, and `ordering` 247 → 170 (−77 games) against `-miss field 3` 18 → 1 (−17). It sits between the baseline release and WIRE 1, so a pairwise baseline→WIRE-1 comparison credits WIRE 1 with all of it and reports WIRE 1 as more than twice its true size. (The largest rung of the night is WIRE 6 at +287; the point is the misattribution, not a new champion.)
- **An unambiguously correct arithmetic fix can be worth zero at the whole-game level.** The Knock Off base-power truncation moved the divergence position in **0 of 1,995 games** and reclassified four. It is right, it is not measurable here, and those are different statements.
- **A class count can fall because a game parts EARLIER on something else.** `-damage field 3` RISES 170 → 216 over WIREs 1-3 and then falls to 141 — the earlier wires push games deeper, which exposes damage divergences that had been masked. So a per-class delta is only readable beside the depth column, and `event missing from medicham2` (604 → 627) growing is not a regression.

141 games part EARLIER than the baseline after six correct fixes, most of it appearing at the mega and WIRE 1 rungs. That is not a contradiction: changing a trajectory surfaces a different pre-existing bug sooner. It is 7.1% of the sample and it is the reason "net later" is reported rather than "later".

Coverage, controlled: distinct moves connected 224 → 261 (+37). The wires' own reports claimed 173 → 197 on ~346-game uncontrolled arms. The absolute levels are not comparable — a different census steers a different sample — but the controlled delta is **larger** than the claimed one, which is WIRE 4's pattern again: the findings were real and the numbers were wrong.

What remains at the top rung, in cause order, because a ladder should end in something actionable: `[from] hospitality` heals medicham2 emits and Showdown does not (127 games across two classes, the single largest cause in the file), Illusion (`zoroarkhisui` on every `switch: a different body`), `-prepare` for two-turn moves, `-activate|feint`, and `-end|throatchop`. All of it is `data/wire-ladder.json` → `what_remains_at_the_top_rung`, and all of it is ENGINE's.

Carried forward, not fixed here: every arm reports `trace_body_off_field` 54-69 — a `??` identifier reaching the medicham2 stream, which `tests/test-protocol-trace.js` PART 6 says must read **0**. It is present in all nine releases including WIRE 6, so it is not something the night introduced. ENGINE's.

What the ladder still cannot see, restated rather than implied: an uncommitted edit inside `SHOWDOWN_PATH`. The other two blind spots `arms_comparable.js` declares — the driver itself and `data/protocol-events.json` — are digested before and after every arm and recorded in the artifact.

21. THE VIEWER WAS AN INSTRUMENT AND NOBODY HAD AUDITED IT. 2026-08-13

`engine/divergence_cards.js` renders diverging games for a human to read. It computes nothing by design, which is exactly why it was never checked — and it carried three claims that were false.

what the page said	what was true	why
"Sixty diverging games out of 209"	80 of 655	typed into the template weeks ago
"209 / 815 diverged"	top 309 / 1,539, bottom 346 / 1,539	same
"80 of 160" (after the first fix)	80 of 655	`of_diverged` tallied the POOL, which is capped at 2× the dump

The third is the interesting one, because it survived a fix. Deriving a number is not enough if the field you derive it from is itself a cap wearing a population's name. It is the same error as the coverage credit before ROADMAP #91: a figure that is SMALLER than the truth still misleads, because it is read as the truth.

And the sample itself was one corner. The dump drew from the primary arm alone, so the page answered "where do the engines part when every sub-100 move misses" while presenting itself as "where do the engines part". Both arms now feed it, interleaved and labelled.

Rule this leaves behind: a page that renders a measurement is part of the measurement. It gets the same treatment as an artifact — every number derived, every narrowing declared, and the population distinguished from the sample by name.

Running the release ladder

```
SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown node engine/wire_ladder.js --write
```

About 7 minutes, one process, ten arms. `--keep-arms <dir>` keeps each arm's full differential artifact; without it they go to a temp directory and the ladder file carries the numbers. It exits non-zero if any arm is refused as incomparable or if the two baseline runs disagree. **It does not run `tests/test-mechanics.js` and neither should anything measuring beside it** — that regenerates the census, which is the steering input the ladder pins.

Running the backtest

```
SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown node engine/backtest_winrate.js
```

About 13-18 minutes, one process, on 6,886 clean games (809s and 1,046s on two runs). `MAXG=200` thins it for a smoke run, and the artifact records the n it actually scored, so a thinned run cannot be mistaken for a published one. It writes `data/winrate-backtest.json` and the per-game rows beside it. Every seeded configuration reproduces bit-identically across runs; the unseeded legacy `winProb2` arm moved 0.26 accuracy points between two full runs, which is its run-to-run floor.

It stamps the sha256 of every source the leaf reads. `status.js` re-hashes those and prints `CURRENT` or `PRE-CHANGE`, which is a comparison rather than an mtime inference — a checkout moves an mtime without moving code, and the 2026-08-02 artifact was quoted for two days against an engine that had gained "one mega per side" in between.

Done looks like

- `status.js` prints a leaf calibration line that is fresh, adequately powered, and states the reliability curve rather than a verdict string. **Done 2026-08-04** — and the answer is that the leaf is worse than a coin.
- `provenance.js --strict` exits zero.

- Every gate R1–R4 has an artifact. **Done 2026-08-04** — and R1's turned out to disagree with the prose it replaced. An artifact per gate is the floor, not the goal.
- Every gate artifact says which configuration produced it. **Done 2026-08-04 for R2 and R3** via `engine/run_stamp.js`; R1's dump has its own inline copy of the same shape and should call the module. An artifact that records its build still is not enough on its own: R3 records its build and **not its control**, and a divergence rate without the self-disagreement floor beside it is a headline, not a result.
- `REFIT OWED` is either clear or has a dated reason next to it.